



ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY

College of Electrical and Mechanical Engineering

Department of Software Engineering

AI-Driven Personalized Learning Roadmap Generator And Tutor

Ontology-Guided Adaptive Learning Roadmap Generator And Tutor

Members: ID:

Yegeta Taye ETS1665/14

Yigermal Abebe ETS1680/14

Yihun Alemayehu ETS1685/14

Advisor Name: Tesfaye Meshu

Signature_____

Acknowledgement

We wish to express our sincere gratitude to the Faculty of Computing, Department of Software Engineering, Addis Ababa Science and Technology University (AASTU) for providing the institutional resources and academic environment necessary to conduct this research.

We extend our deepest appreciation to our academic advisor, Tesfaye Meshu, for their invaluable guidance, constructive feedback, and mentorship throughout the conceptualization and development phases of this final year project. Their expertise in software engineering methodologies and educational technology has been instrumental in refining the scope and technical direction of AI-Driven Personalized Learning Roadmap Generator And Tutor.

We are grateful to our peers and colleagues who engaged in technical reviews and provided insightful critiques during project development workshops. Additionally, we acknowledge the open-source community, particularly the developers of the frameworks and libraries leveraged in this project (React, Flutter, Python, PostgreSQL).

Finally, we recognize the educational content providers whose free, high-quality resources — FreeCodeCamp, GeeksForGeeks, MDN Web Docs, and others — form the foundation of the resource aggregation strategy that makes AI-Driven Personalized Learning Roadmap Generator And Tutor viable in resource-constrained environments like Ethiopia.

Abstract

The proliferation of online educational content has created unprecedented opportunities for self-directed learning, yet it has simultaneously introduced a critical problem: the fragmentation of high-quality resources and the absence of structured guidance. In the Ethiopian context, where access to premium mentorship and formal course subscriptions is economically prohibitive, students often find themselves trapped in “tutorial hell” — scrolling through disconnected YouTube videos, blogs, and documentation without a coherent learning path or performance feedback mechanism. Existing solutions (e.g., roadmap.sh) provide static, one-size-fits-all curricula that fail to adapt to individual learning pace, gaps, or learning preferences.

This project presents AI-Driven Personalized Learning Roadmap Generator And Tutor, a full-stack web platform that synthesizes verified ontology-based curriculum design with intelligent adaptive tutoring across five technical domains: Frontend Development, Backend Development, Data Science, DevOps Engineering, and Python. The core innovation is a hybrid artificial intelligence architecture that eliminates hallucination risks inherent in pure Large Language Model (LLM) generation through a “Skeleton & Flesh” strategy. Offline, a verified domain ontology (the skeleton) encodes the curriculum as a directed acyclic graph of learning nodes with prerequisite relationships and learning outcomes. At runtime, a three-tier LLM stack—Phi-4-Multimodal (primary, zero-cost), Ollama/Qwen2.5 (local CPU fallback), and Gemini 2.5 Flash (cloud last resort)—generates context-specific explanations and adaptive quizzes (the flesh) grounded exclusively in the verified skeleton, preventing factual deviation.

AI-Driven Personalized Learning Roadmap Generator And Tutor implements five tightly integrated mechanisms: (1) the *Gatekeeper Pattern*, requiring $\geq 80\%$ quiz performance before advancing, with tiered remediation (resource adaptation, prerequisite review) for underperforming learners; (2) *Mastery Decay*, operationalising Ebbinghaus’s Forgetting Curve by transitioning mastered nodes through time-aware states (Mastered $\rightarrow$ Review Needed $\rightarrow$ Relearn) and triggering micro-quizzes automatically; (3) *Multi-Path Branching*, enabling learners to select specialised trajectories (Frontend, Backend, Data Science) while maintaining prerequisite integrity; (4) a *Streaming AI Instructor*, a real-time conversational tutor delivering word-by-word explanations and answering node-specific questions via Server-Sent

Events; and (5) *Verifiable Certificates*, issued on roadmap completion with cryptographically unique identifiers, downloadable as PDF or PNG, and publicly verifiable via a shareable link.

The system is delivered as a microservices architecture comprising three services (API Gateway, Learning Service, AI Service) behind an Nginx reverse proxy, fully containerised with Docker Compose. The frontend is a responsive React 18 and TypeScript single-page application. Authentication supports email/password, Google OAuth, and GitHub OAuth with JWT-based stateless session management. Role-based access control provides distinct interfaces for learners, domain expert instructors, and administrators. Learning analytics, achievement tracking, and instructor dashboards provide visibility into learner progress and domain health.

This project addresses a critical gap in educational technology for developing economies, democratizing access to structured, mentorship-quality guidance at zero cost to end users. By combining a verified knowledge graph with intelligent adaptive assessment and a cost-optimized, locally-runnable AI stack, AI-Driven Personalized Learning Roadmap Generator And Tutor empowers Ethiopian students and self-directed learners globally to navigate the landscape of technical skill acquisition with structured confidence and measurable mastery.

Keywords: adaptive learning, ontology-based curriculum design, large language models, assessment-driven progression, educational technology, spaced repetition, mastery decay, personalized learning paths, microservices, developing economy context, Ethiopia.

Contents

Acknowledgement	2
Abstract	3
List of Figures	10
List of Tables	10
List of Abbreviations	12
1 Introduction	14
1.1 Statement of the Problem	14
1.2 Objectives	16
1.2.1 General Objective	16
1.2.2 Specific Objectives	16
1.3 Scope and Limitations	19
1.3.1 In-Scope Elements	19
1.3.2 Out-of-Scope Elements	21
1.3.3 Limitations	21
1.4 Methodology and Approach	22
1.4.1 Development Methodology: Agile-Scrum	22
1.4.2 System Architecture: Microservices with Layered Design	23
1.4.3 Hybrid AI Strategy: Skeleton & Flesh	23
1.4.4 Gatekeeper Pattern: Three-Tier Progression Logic	24

1.4.5	Spaced Repetition Integration: Mastery Decay Algorithm	24
1.4.6	Multi-Path Branching Logic	25
1.4.7	Resource Generation: Ontology-Grounded AI Content	25
1.4.8	Technology Stack and Justifications	26
1.4.9	LLM Selection Rationale	26
1.4.10	Development Phases	27
1.5	Plan of Activities	28
1.6	Budget Breakdown	29
1.7	Significance of the Project	29
1.7.1	Academic Significance	29
1.7.2	Regional and Socioeconomic Significance (Ethiopian Context)	30
1.7.3	Technical and Practical Significance	30
1.7.4	User and Stakeholder Significance	31
1.8	Outline of the Study	31
2	Literature Review	33
2.1	Study of Related Works	33
2.1.1	Evolution of Intelligent Tutoring Systems	33
2.1.2	Knowledge Space Theory and Ontology-Based Curriculum Design	34
2.1.3	Knowledge Representation: From Ontologies to Retrieval-Augmented Generation	35
2.1.4	Bayesian Knowledge Tracing and Mastery State Machines	36
2.1.5	Adaptive Assessment and Item Response Theory	37
2.1.6	Adaptive Learning, Mastery-Based Progression, and Assessment-Driven Adaptation	37
2.1.7	Spaced Repetition and Knowledge Decay	39
2.1.8	Personalisation, Adaptive Hypermedia, and Multi-Path Learning	41
2.1.9	Resource Aggregation and Content Curation	42
2.1.10	Gamification in Educational Technology	44
2.1.11	Large Language Models as Educational Tools	44

2.1.12	Learning Analytics and Progress Dashboards	45
2.1.13	Academic Foundations: Synthesis of Educational Psychology	46
2.2	Identifying Milestones of the Related Literatures and Finding the Gaps	47
2.2.1	Key Milestones	47
2.2.2	Critical Gaps in Existing Solutions	49
2.3	Lessons Learned from Literatures	50
3	Problem Analysis and Modeling	54
3.1	Existing System and Its Problems	54
3.2	Requirements Elicitation and Specification	57
3.3	System Modeling	63
3.3.1	Use Case Model	63
3.3.2	Functional Requirements Traceability	63
3.3.3	UML Class Diagram	63
3.3.4	Dynamic Models of a System	65
3.4	Model Validation	71
4	System Design	73
4.1	Overview	73
4.2	Specifying the Design Goals	73
4.3	System Design	74
4.3.1	Proposed Software Architecture	74
4.3.2	Subsystem Decomposition	76
4.3.3	Database Design	77
4.3.4	Deployment Diagram	87
4.3.5	User Interface Design	88
4.3.6	System Integration	94
4.3.7	Security Design	96
4.4	Verifying the Requirements in the Design	98
4.4.1	Requirements Traceability Approach	98

CONTENTS

4.4.2	Functional Requirements Verification Matrix	98
4.4.3	Non-Functional Requirements Verification Matrix	103
4.4.4	Verification Methods and Evidence	104
4.4.5	Discrepancy Identification and Resolution	105
4.4.6	Evidence Sources	105
5	System Implementation	107
5.1	Reviewing the Design Solution	107
5.2	Deciding on the Development Tools	109
5.2.1	Programming Languages	109
5.2.2	Frameworks and Core Libraries	109
5.2.3	AI and External Integration Tools	110
5.2.4	Frontend and Mobile Tooling	110
5.2.5	Testing and Quality Tooling	110
5.2.6	Infrastructure and Runtime Tooling	110
5.2.7	Development Environment Setup	111
5.2.8	Rationale Summary	113
5.3	Developing the Solution	113
5.3.1	Coding Practices and Development Workflow	113
5.3.2	Integration Strategy During Development	113
5.3.3	Major Functionality Code Excerpts	114
5.3.4	Integration Challenges and Resolutions	125
5.3.5	Summary	126
6	System Evaluation	127
6.1	Preparing Sample Test Plans	127
6.1.1	Functional Test Plans	128
6.1.2	Non-Functional Test Plans	137
6.1.3	Automated Test Suite Implementation	138
6.2	Evaluating the Proposed Design and Solutions	142

6.2.1	Functional Test Results	142
6.2.2	Non-Functional Test Results	149
6.2.3	Security Test Results	152
6.2.4	AI Provider Failover Test Results	152
6.3	Discussing the Results	152
7	Conclusions and Recommendations	155
7.1	Conclusion of the Study	155
7.2	Recommendations of the Study	158
	References	165
	Appendix A: Source Code and Supplementary Materials	166
A.1	Core Algorithm Pseudocode	166
A.2	Adaptive Difficulty Computation	166
A.3	Mastery Decay Transition Logic	167

List of Figures

3.1	UML Class Diagram	65
3.2	Quiz Attempt and Gatekeeper Flow	66
3.3	Mastery Decay and Spaced Repetition Trigger	67
3.4	Multi-Path Branching Selection	68
3.5	Learner Progression Activity Flow	69
3.6	Gatekeeper Decision Activity	70
3.7	Node Mastery State Machine	71
4.1	System Architecture Diagram	76
4.2	Entity-Relationship Diagram (Full Attributes)	87
4.3	Deployment Diagram	88
4.4	UI Mockups and Wireframe Navigation	89
4.5	UI Navigation Diagram	90
4.6	Login Page	91
4.7	Sign Up Page	91
4.8	Dashboard Page	91
4.9	Overview Page	92
4.10	Roadmap Page	92
4.11	Explanation Page	92
4.12	Quiz Page	93
4.13	Catalog Page	93
4.14	Enroll Page	93
4.15	Sequence Diagram (Auth + AI Flow)	96

List of Tables

1.1	Technology stack and justifications	26
1.2	Cost comparison: three-tier LLM architecture vs. original GPT-4 plan	27

1.3	Detailed weekly breakdown (24 weeks)	28
2.1	Summary reference: academic foundations of AI-Driven Personalized Learning Roadmap Generator And Tutor	47
2.2	Position of AI-Driven Personalized Learning Roadmap Generator And Tutor within the landscape	53
3.1	Primary Use Cases	64
3.2	Use Case to Functional Requirement Mapping	65
4.1	users table	78
4.2	domains table	78
4.3	ontology_versions table	79
4.4	learning_nodes table	79
4.5	node_prerequisites table	80
4.6	enrollments table	80
4.7	learner_node_progress table	80
4.8	quizzes table	81
4.9	quiz_questions table	81
4.10	quiz_attempts table	82
4.11	resources table	82
4.12	resource_ratings table	82
4.13	domain_whitelist table	83
4.14	adaptation_events table	83
4.15	challenge_projects table	83
4.16	notifications table	84
4.17	refresh_tokens table	84
4.18	certificates table	84
4.19	learner_velocity table	85
4.20	xp_events table	85
4.21	user_badges table	86

4.22	supplementary_nodes table	86
4.23	Functional requirements verification matrix	98
4.24	Non-functional requirements verification matrix	103
6.1	Functional test plans	128
6.2	Non-functional test plans	137
6.3	Test frameworks and coverage by layer	139
6.4	Test files and coverage areas	139
6.5	Test coverage summary	142
6.6	Functional test results	143
6.7	Non-functional test results	150
6.8	AI provider failover test results	152

List of Abbreviations

Abbreviation	Full Form	Definition
AASTU	Addis Ababa Science and Technology University	Institution hosting the research
AI	Artificial Intelligence	Systems designed to simulate intelligent behavior
API	Application Programming Interface	Software intermediary for application communication
LLM	Large Language Model	Deep neural network trained on vast text corpora
RAG	Retrieval Augmented Generation	LLM generation with external knowledge retrieval
SERP	Search Engine Results Page	Structured search result data returned by SERP API
SSE	Server-Sent Events	HTTP streaming protocol for real-time token delivery
MCQ	Multiple Choice Question	Assessment with predefined answer options

Abbreviation	Full Form	Definition
DAG	Directed Acyclic Graph	Graph structure for learning dependencies
JSON	JavaScript Object Notation	Lightweight data interchange format
REST	Representational State Transfer	Architectural style for web service APIs
MOOC	Massive Open Online Course	Large-scale online educational platforms
UX	User Experience	Quality of interaction between user and system
UI	User Interface	Visual elements of the application
SRS	Software Requirements Specification	Formal document detailing requirements
QA	Quality Assurance	Systematic testing and validation
Git	Version Control System	Distributed version control
PostgreSQL	Open-source Relational Database	Persistent data storage
React	JavaScript Frontend Library	Component-based UI framework
Flutter	Mobile Application Framework	Cross-platform mobile development framework
Node.js	JavaScript Runtime Environment	Server-side JavaScript execution
Python	Programming Language	High-level language for backend AI/ML
CI/CD	Continuous Integration/Deployment	Automated pipeline for code testing and release
JWT	JSON Web Token	Secure authentication token
SSL/TLS	Secure Sockets Layer/Transport Layer Security	Cryptographic protocols for secure communication
HTTPS	Hypertext Transfer Protocol Secure	Encrypted web communication protocol
ERD	Entity-Relationship Diagram	Conceptual model of database structure
DFD	Data Flow Diagram	Process-centric model showing data movement
RBAC	Role-Based Access Control	Permission management system

Chapter 1

Introduction

1.1 Statement of the Problem

The proliferation of free online learning resources—YouTube tutorials, blog posts, open-source documentation, and platforms like FreeCodeCamp and GeeksForGeeks—has democratized access to technical knowledge. However, this abundance has created a paradoxical challenge: fragmentation without structure. Ethiopian students and self-directed learners face a critical problem landscape characterised by six interconnected challenges.

Challenge 1: The Tutorial Hell Phenomenon. Learners navigate an overwhelming landscape of resources without a coherent learning path. A student seeking to learn web development might start with an HTML tutorial, then jump to a CSS playlist on YouTube, then read scattered JavaScript documentation, then discover React, without understanding how these components interconnect or progress logically. This non-sequenced, ad-hoc learning creates knowledge gaps, misaligned prerequisites, and wasted effort. Research on cognitive load theory [51] demonstrates that unstructured, fragmented information presentation significantly increases cognitive load, reducing learning efficiency and retention.

Challenge 2: Absence of Adaptive Feedback Loops. Existing free resources are predominantly passive. Learners consume content (watch videos, read documentation) but receive minimal performance-based feedback or adaptation. If a learner struggles with a concept, the resource does not diagnose why or adapt the explanation. Unlike human tutors who adapt in real-time based on learner confusion, static resources force learners to self-diagnose and manually seek alternative resources. This mirrors the broader finding in educational psychology that immediate, targeted feedback is essential for efficient learning [28], yet most self-directed platforms lack this feature.

Challenge 3: No Mastery-Based Progression Gates. Learners can mark topics as “complete”

based on time spent or arbitrary quizzes, regardless of actual competence. A learner might watch a JavaScript video, feel confident, and mark “JavaScript Fundamentals” as done—then struggle severely when prerequisites are required in an advanced topic. This leads to a false sense of mastery and downstream learning failures. Research on mastery-based progression [4] establishes that learning should be gated by demonstrated competence, not time or completion of content consumption.

Challenge 4: Cognitive Overload and Retention Decay. Self-directed learners often experience cognitive overload: learning multiple new concepts simultaneously without reinforcement, leading to rapid forgetting. Ebbinghaus’s forgetting curve [19] demonstrates that without strategic review, learned information decays exponentially within days. Yet self-directed learners have no automated system prompting spaced repetition; they must manually re-engage with material, which is cognitively demanding and often neglected. Consequently, skills learned weeks or months prior decay to unusable levels by the time they are needed in integrated projects.

Challenge 5: Generic Roadmaps Ignore Learner Heterogeneity. Static, one-size-fits-all learning paths (e.g., “Learn HTML → CSS → JavaScript → React → Node.js”) ignore that learners differ in preferences, prior knowledge, and career aspirations. Some learners prefer visual, design-oriented paths; others prefer backend logic and databases. A fixed sequence frustrates learners whose interests diverge from the prescribed path, reducing intrinsic motivation and increasing dropout rates. Research on self-determination theory [48, 13] shows that learner agency—the ability to make meaningful choices about learning—significantly impacts motivation and persistence.

Challenge 6: The Hallucination Problem in AI-Driven Recommendations. Recent tools leveraging Large Language Models (LLMs) for curriculum generation can produce fluent but factually incorrect sequences. An LLM might generate a learning path for Python that contains logical inconsistencies, missing prerequisites, or inaccurate technical descriptions [42, 58]. In technical education, where precision is non-negotiable, hallucination in curriculum design is catastrophic. Learners might internalize incorrect concepts, leading to buggy code or flawed system design.

Systemic Root Cause: Lack of Verifiable, Adaptive Knowledge Structures. Underlying these six challenges is a fundamental structural problem: the absence of a verified, machine-readable knowledge graph that combines the rigor of traditional Intelligent Tutoring Systems with the scalability and flexibility of modern LLMs. Current solutions fall into two inadequate categories:

1. Static resources (documentation, videos, blogs): high quality but passive, non-adaptive, and fragmented.
2. Pure LLM systems (ChatGPT tutoring): flexible and scalable but prone to hallucination and lack curriculum coherence.

Neither category addresses the integration of verified domain knowledge, adaptive progression, and

performance-based feedback at the system level.

1.2 Objectives

1.2.1 General Objective

To design, develop, and validate an ontology-guided, adaptive learning roadmap generator that integrates verified domain knowledge with intelligent tutoring principles to enable self-directed Ethiopian learners to achieve technical competence in programming and software engineering through structured, performance-driven, and personalized learning paths.

1.2.2 Specific Objectives

Objective 1: Design and Implement a Hybrid AI Architecture. Develop and operationalise the “Skeleton & Flesh” hybrid architecture that separates curriculum knowledge representation (the verified ontology skeleton) from runtime tutoring (the adaptive AI flesh). Specifically:

- Create domain-specific ontologies (DAG of learning nodes, prerequisites, learning outcomes) for five initial domains: Frontend Development (HTML/CSS/JavaScript/React), Backend Development (Node.js/Databases/APIs), Data Science (Python/Pandas/ML), DevOps Engineering (Docker/CI-CD/Cloud), and Python (core language fundamentals).
- Implement a Teacher Model (offline LLM-based generator) that produces the initial ontology with human expert verification.
- Implement a Tutor Model (runtime, three-tier AI stack) that generates explanations, adaptive quizzes, and conversational answers grounded in the ontology, preventing hallucination through skeleton-constrained generation.

Objective 2: Implement the Gatekeeper Pattern for Assessment-Driven Progression. Engineer and validate the Gatekeeper Pattern, an assessment-driven progression mechanism where learners cannot advance to subsequent learning nodes without passing performance thresholds on adaptive quizzes. Specifically:

- Design three-tier quiz outcomes: Pass ($\geq 80\%$ → unlock next node + challenge project), Marginal Pass (70–79% → unlock node + flag for spaced repetition), Fail ($< 70\%$ → trigger resource adaptation or prerequisite review).
- Implement adaptive content modality: if a learner fails ($< 70\%$), trigger a `resource_swap` adaptation event that instructs the Tutor Model to regenerate the node explanation in a more

tutorial-oriented, step-by-step style rather than a reference-documentation style, recognising modality mismatch in learning.

- Implement prerequisite validation: if a learner fails ($< 50\%$), recommend review of foundational nodes before retry.

Objective 3: Integrate Spaced Repetition with Decay-Aware Roadmap Evolution. Operationalise Mastery Decay—a system that integrates Ebbinghaus’s forgetting curve directly into the learner’s roadmap visualisation, making knowledge decay visible and triggering timely reviews. Specifically:

- Implement a decay algorithm where nodes transition from “Mastered” (green, 0–14 days) to “Review Needed” (yellow, 14–30 days) to “Relearn” (red, > 30 days) based on time since last visit.
- Generate micro-quizzes (2–3 questions) when nodes transition to yellow or red status, targeting core concepts of that node.
- Record and visualise decay patterns, enabling learners to identify their personal retention curves and adjust study frequency.

Objective 4: Develop Multi-Path Branching for Personalized Learning. Design and implement Multi-Path Branching where learners choose between alternative learning sequences based on interests and strengths, while ensuring semantic coherence and prerequisite integrity. Specifically:

- After foundational nodes, offer learners a choice: “Prefer visual design or backend logic?” recommending either Frontend (CSS $\rightarrow$ UI frameworks) or Backend (Databases $\rightarrow$ APIs) paths.
- Ensure paths reconverge at advanced, integrative topics (e.g., “Full-Stack Architecture”, “System Design”).
- Validate that paths maintain prerequisite dependencies and learning outcome alignment.

Objective 5: Develop AI-Grounded Resource Generation. Build a quality-assured content generation pipeline that produces node-specific learning resources and references anchored entirely within the verified ontology skeleton. Specifically:

- Generate node explanations and supplementary resource recommendations at runtime using the Tutor Model, constrained to the node’s verified learning outcomes so that no external hallucinated links are introduced.

- Attach curated reference links to ontology nodes during the offline authoring phase, verified by domain experts before publication.
- Enable domain expert instructors to review and update node resources through the administrator ontology builder interface, maintaining content quality without system downtime.

Objective 6: Develop Role-Stratified User Interfaces. Create intuitive, responsive interfaces for all three system roles—learner, domain expert instructor, and administrator—enabling end-to-end interaction with the platform. Specifically:

- Develop a full-featured web application (React 18 + TypeScript) providing learners with an interactive roadmap DAG, progress dashboards, quiz flows, streaming AI explanations, an AI Instructor chatbot, learning insights analytics, achievement tracking, and certificate download-/sharing.
- Develop an instructor portal enabling domain experts to monitor learner progress, view domain-level analytics, review flagged quiz events, and manage ontology content.
- Develop an administrative console for user management, domain lifecycle management, ontology construction, and system statistics monitoring.
- Develop a Flutter-based mobile application (Android/iOS) as a complementary access channel, providing roadmap navigation and quiz interactions for learners on mobile devices.

Objective 7: Validate System Effectiveness with Ethiopian Learners. Conduct empirical evaluation of AI-Driven Personalized Learning Roadmap Generator And Tutor’s learning efficacy and usability with real learners. Specifically:

- Recruit 50–100 Ethiopian students/engineers as test users.
- Measure learning gains: assess pre/post knowledge using domain-aligned assessments, comparing learners using AI-Driven Personalized Learning Roadmap Generator And Tutor vs. a control group using traditional resources.
- Measure engagement and retention: track time-on-task, completion rates, session frequency, and long-term return visits (6+ months).
- Collect usability feedback via surveys and interviews, identifying UI/UX improvements and cultural/linguistic adaptations needed.

Objective 8: Ensure System Scalability and Sustainability. Design and implement AI-Driven Personalized Learning Roadmap Generator And Tutor as a scalable, maintainable, open-source system capable of extension to additional domains and deployment in resource-constrained environments. Specifically:

- Deploy using Docker Compose with an Nginx reverse proxy, enabling reproducible, infrastructure-agnostic deployment on any VPS or cloud host without vendor lock-in.
- Implement circuit breakers on all three AI providers to guarantee graceful degradation and uninterrupted service when individual AI backends are unavailable.
- Maintain Redis-based response caching to minimise redundant AI API calls and reduce operating costs at scale.
- Create comprehensive documentation and developer guides enabling future contributors to extend the system to new domains or platforms.

1.3 Scope and Limitations

1.3.1 In-Scope Elements

The AI-Driven Personalized Learning Roadmap Generator And Tutor system encompasses the following capabilities and features:

1. Learning Domains: Five technical domains are in scope: (a) Frontend Development (HTML, CSS, JavaScript ES6+, React.js, responsive design), (b) Backend Development (Node.js, Express, RESTful API design, SQL/NoSQL databases), (c) Data Science (Python, Pandas, NumPy, basic ML algorithms, data visualisation), (d) DevOps Engineering (Docker, CI/CD pipelines, cloud infrastructure fundamentals), and (e) Python (core language fundamentals, OOP, standard library). These domains were selected as high-demand skills with well-defined prerequisite structures and representational breadth across modern software engineering.

2. Hybrid AI Architecture: Full implementation of the Skeleton & Flesh strategy, including (a) a Teacher Model that uses LLMs to generate the initial ontology DAG with expert verification, (b) a runtime Tutor Model implementing a three-tier provider chain (Phi-4-Multimodal → Ollama/Qwen2.5 → Gemini 2.5 Flash) that generates explanations, adaptive quizzes, and conversational answers constrained by the verified ontology, and (c) circuit-breaker-guarded provider failover to ensure continuous availability when individual AI backends are unreachable.

3. Gatekeeper Pattern: Complete implementation of assessment-driven progression, including (a) adaptive quiz generation for each node, (b) three-tier outcome logic (Pass/Marginal Pass/Fail), (c)

adaptive resource swapping triggered by failure modes, (d) prerequisite validation and enforcement, and (e) challenge project recommendations for learners who pass with high scores.

4. Spaced Repetition Integration: Full implementation of Mastery Decay, including (a) decay-state transitions (Green → Yellow → Red) based on time since last visit, (b) automated micro-quiz triggers, (c) visual roadmap representation of decay states, and (d) learner notifications prompting review.

5. Multi-Path Branching: Implementation of learner choice at key curriculum junctures, including (a) branching questions (e.g., “Prefer frontend or backend?”), (b) alternative learning sequences for each path, (c) prerequisites maintained within each path, and (d) reconvergence points at advanced topics.

6. AI-Grounded Content and Resource Generation: Full implementation of ontology-constrained content generation, including (a) streaming AI explanations delivered token-by-token via Server-Sent Events (SSE), (b) a conversational AI Instructor chatbot supporting real-time question-and-answer within each learning node, and (c) node-level resource references curated offline by domain experts and surfaced at runtime.

7. User Interfaces: Complete development of (a) a web application (React 18 + TypeScript) for desktop/laptop learners with roadmap DAG visualisation, progress dashboards, quiz flows, streaming AI panels, learning insights analytics, gamification achievements, and certificate management; (b) an instructor portal for learner monitoring, domain analytics, and flagged event review; (c) an admin console for user, domain, and ontology management; and (d) a Flutter mobile application (Android/iOS) as a complementary access channel.

7a. Certificates: Full implementation of verifiable completion certificates, including (a) server-issued certificates with Crockford base-32 public identifiers upon roadmap mastery, (b) client-side PDF and PNG download via html-to-image and jsPDF, and (c) a publicly accessible, no-authentication verification page at `/verify/:publicId` for sharing and employer validation.

8. User Base: Initial deployment targets Ethiopian software engineering students (university-level), self-taught engineers, and career-transitioning professionals. A target user base of 50–100 test users for initial validation, with potential to scale to thousands.

9. Deployment and Scalability: Deployment using Docker Compose orchestration with Nginx as a reverse proxy, providing infrastructure-agnostic containerized deployment suitable for any Linux VPS or cloud host. The microservices architecture (API Gateway, Learning Service, AI Service) with Redis caching enables horizontal scaling of individual services independently.

10. Methodology: Agile-Scrum development methodology with 2-week sprints, daily standups, sprint reviews, and retrospectives, ensuring iterative delivery and stakeholder feedback integration.

1.3.2 Out-of-Scope Elements

The following elements are explicitly excluded from this project:

1. **Spoken Language Learning:** The system does not support learning of spoken languages. It focuses exclusively on programming languages and technical content, where precision and verification are more tractable.
2. **Paid Content Integration:** The system does not aggregate links to paid courses or license proprietary content. It restricts to free, openly accessible resources, ensuring accessibility for Ethiopian learners with limited financial means.
3. **Video Content Scraping:** The system does not scrape, host, or re-distribute video content. While it may link to high-quality YouTube videos from trusted educational channels, it does not download or modify video content.
4. **Advanced Gamification:** While the system implements basic achievement tracking and progress visualisation, advanced competitive gamification elements (leaderboards, peer challenges, community rankings) are out of scope.
5. **Institutional Accreditation:** The system issues verifiable digital certificates of completion with publicly accessible verification links. However, these certificates are not formally accredited by academic institutions or professional certification bodies in the initial deployment.
6. **Collaborative Learning:** The system does not include peer collaboration features (discussion forums, pair programming rooms, group projects). Learning paths are individualised.
7. **LMS Integration:** Integration with existing Learning Management Systems (Moodle, Canvas, Blackboard) is out of scope. The system operates as a standalone platform.
8. **Advanced Personalisation Based on Learning Styles:** While Multi-Path Branching provides some personalisation, the system does not implement advanced learning style detection [45].

1.3.3 Limitations

The project acknowledges the following realistic constraints:

1. **Ontology Verification Bottleneck:** The quality of the skeleton ontology depends on expert verification. If domain experts are unavailable or provide incomplete feedback, the ontology may contain errors or omissions.

2. **Resource Variability:** The quality of AI-generated explanations and supplementary resources varies with LLM provider quality. If the primary and fallback AI providers are both unavailable, the system falls back to cached responses or static node content, which may be less tailored to the learner.
3. **Algorithm Calibration:** The Mastery Decay algorithm parameters (14-day yellow threshold, 30-day red threshold, 80% pass score) are initial estimates informed by research but may require calibration based on real learner data.
4. **Scalability Constraints:** At extreme scale, costs for LLM API calls, database queries, and computation may become prohibitive. Cost optimization may be needed.
5. **LLM Costs:** While the three-tier fallback architecture reduces LLM costs to approximately \$15 over the project duration—a 99.5% reduction from the originally planned GPT-4—the Gemini API still incurs costs at scale.
6. **Learner Motivation and Self-Discipline:** The system can structure learning and provide adaptive feedback, but it cannot directly motivate learners or prevent procrastination.
7. **Hands-On Verification:** The system recommends resources and provides quizzes but does not directly assess learners' ability to write functional code.

1.4 Methodology and Approach

1.4.1 Development Methodology: Agile-Scrum

AI-Driven Personalized Learning Roadmap Generator And Tutor is developed using Agile-Scrum methodology, emphasising iterative delivery, continuous stakeholder feedback, and adaptive planning. Key practices include:

- **Sprint Duration:** 2-week sprints, with each sprint focused on delivering incremental, testable features.
- **Daily Standups:** 15-minute daily synchronization meetings where team members report progress, identify blockers, and coordinate efforts.
- **Sprint Planning:** At sprint start, the team selects user stories from the product backlog, estimates effort, and commits to sprint goals.
- **Sprint Review:** At sprint end, completed features are demonstrated to stakeholders for feedback.

- **Sprint Retrospective:** The team reflects on process improvements, identifies what went well, and what needs adjustment.

1.4.2 System Architecture: Microservices with Layered Design

AI-Driven Personalized Learning Roadmap Generator And Tutor is architected as a modular microservices system with clear separation of concerns:

1. **Frontend Layer (Presentation):** Web application (React 18 + TypeScript + Vite) with Zustand state management, Tailwind CSS, responsive design, and interactive DAG roadmap visualisation via React Flow. Flutter mobile application (Dart) for iOS/Android as a complementary access channel.
2. **API Gateway Layer (Routing and Authentication):** Node.js/Express service handling HTTP request routing, JWT issuance and verification, Google/GitHub OAuth2 flows, rate limiting, and service-to-service proxying.
3. **Learning Service (Core Business Logic):** Node.js/Express/Prisma service encapsulating all domain logic: enrollment management, learner progress tracking, Gatekeeper assessment evaluation, Mastery Decay computation, multi-path branching, certificate issuance, notifications, achievements, and instructor/admin operations.
4. **AI Service (Tutor Model):** Dedicated Node.js service orchestrating the three-tier LLM provider chain (Phi-4 → Ollama → Gemini) with circuit breakers, Redis caching, SSE streaming, and structured prompt engineering for quiz generation, explanation generation, and AI Instructor chat.
5. **Data Layer (Persistence):** PostgreSQL with Prisma ORM (learner profiles, ontology, progress, quiz attempts, certificates, achievements), Redis (AI response cache, session data).
6. **Infrastructure Layer:** Docker Compose orchestration, Nginx reverse proxy (routing, SSE buffering control, SSL termination), and GitHub Actions CI/CD pipeline.

1.4.3 Hybrid AI Strategy: Skeleton & Flesh

The ontology and AI components implement the Skeleton & Flesh architecture to solve hallucination:

The Skeleton (Teacher Model): A powerful LLM (Gemini 2.5 Flash or similar) generates an initial domain ontology—a directed acyclic graph where nodes represent learning concepts, edges represent prerequisite relationships, and metadata includes learning outcomes, estimated effort, and

recommended resources. The generated ontology undergoes rigorous human expert review: domain experts verify each node for accuracy, completeness, and logical sequencing. Once verified, the skeleton is locked and immutable, stored in the database and served to the Tutor Model at runtime.

The Flesh (Runtime, Tutor Model): At runtime, when a learner encounters a node and requests an explanation, the Tutor Model (a lightweight LLM) generates an explanation grounded in the verified skeleton. It cannot invent new nodes or deviate from the skeleton structure. The Tutor Model also generates quiz questions targeting specific learning outcomes of that node, draws from relevant resources, and provides personalised elaborations based on learner context. By restricting generation to the scope of the skeleton, the Tutor Model prevents hallucination while maintaining flexibility and personalisation.

1.4.4 Gatekeeper Pattern: Three-Tier Progression Logic

Learners progress through the roadmap by passing performance thresholds defined by the Gatekeeper Pattern:

- **Tier 1 – Strong Mastery ($\geq 80\%$):** Quiz score $\geq 80\%$: Node marked “Done”, learner unlocks the next node. The learner receives a challenge project recommendation (a small real-world problem requiring application of the node’s concepts). Node marked as “Mastered” (green status) with a 14-day decay timer initiated.
- **Tier 2 – Marginal Mastery (70–79%):** Quiz score 70–79%: Node marked “Done”, learner unlocks next node, but node is flagged for spaced repetition review. Learners are not offered a challenge project immediately. Node marked as “Mastered” with a 7-day decay timer (faster decay due to lower initial mastery).
- **Tier 3 – Insufficient Mastery ($< 70\%$):** Node remains locked. Score 50–69% triggers a `resource_swap` adaptation event (Tutor Model regenerates in tutorial-oriented style). Score $< 50\%$ triggers prerequisite review. Score $< 30\%$ escalates to the instructor via a flagged event.

1.4.5 Spaced Repetition Integration: Mastery Decay Algorithm

The Mastery Decay algorithm integrates Ebbinghaus’s forgetting curve into the roadmap UI, making knowledge decay visible:

Decay States:

- **Green (0–14 days):** “Mastered.” Learners are not prompted to review.

- **Yellow (14–30 days):** “Review Needed.” A micro-quiz (2–3 questions) is generated. Learners are notified.
- **Red (> 30 days):** “Relearn.” A full-length quiz is triggered. Learners are strongly encouraged to re-engage.

Micro-Quiz Logic: When a node transitions from green to yellow or red, a micro-quiz is auto-generated targeting 2–3 core learning outcomes. Question formats include multiple choice, short answer, and code completion. If passed ($\geq 80\%$), the node timer resets for 14 days. If failed, the node is re-locked and resource adaptation is triggered.

Visualization: The learner’s roadmap DAG is color-coded: green (mastered, current), yellow (review needed), red (relearn), gray (not yet attempted), blue (prerequisites not met). A timeline view shows decay over time.

1.4.6 Multi-Path Branching Logic

After completing foundational nodes (e.g., “Core Programming Fundamentals”), learners encounter a branching point:

Branching Question Example: “Which area aligns with your goals and interests? A) Frontend Development, B) Backend Development, C) Data Science & AI”

Each path maintains prerequisite integrity. Paths converge at “Full-Stack Integration” (Frontend/Backend) or “AI/ML Systems Design” (Data Science) nodes, ensuring foundational coherence while respecting learner agency.

1.4.7 Resource Generation: Ontology-Grounded AI Content

Learning content is delivered through a two-layer approach that prevents hallucination by anchoring all AI output within the verified ontology:

Offline Curation Layer: During ontology authoring, domain experts attach curated reference links directly to each learning node. These resources are reviewed for accuracy and availability before publication and are served immediately to learners without AI mediation.

Runtime AI Generation Layer: When a learner enters a node, the Learning Service invokes the AI Service to generate a node-specific explanation. The prompt includes the node’s title, learning outcomes, difficulty level, and prerequisite chain—all drawn from the verified skeleton. The Tutor Model may not introduce concepts or topics outside this context, preventing factual deviation. Explanations are streamed token-by-token to the client via SSE for immediate display.

AI Instructor (Conversational): Each learning node hosts a persistent AI Instructor chatbot. Learner questions are sent with the same node context block, ensuring answers remain grounded. Chat responses also stream via SSE, providing a responsive conversational experience without waiting for full response generation.

Instructor Oversight: Domain expert instructors can monitor flagged quiz events and learner analytics to identify nodes where learners consistently struggle, informing future ontology refinements or resource additions.

1.4.8 Technology Stack and Justifications

Table 1.1 details the technology stack and rationale for each choice.

Table 1.1. *Technology stack and justifications*

Component	Technology	Justification
Frontend Web	React 18 + TypeScript + Vite	Type safety, fast HMR, large ecosystem
Frontend Mobile	Flutter (Dart)	Cross-platform iOS/Android
DAG Visualization	React Flow	Declarative node-graph with React
API Gateway	Node.js + Express	Routing, JWT auth, rate limiting
Learning Service	Node.js + Express + Prisma	Business logic, ORM type safety
AI Service	Node.js + Express	AI provider orchestration
Authentication	JWT + Google/GitHub OAuth2	Stateless, social login support
Database	PostgreSQL + Prisma ORM	ACID compliance, type-safe queries
Cache	Redis	In-memory cache, sub-ms latency
LLM — Primary	Phi-4-Multimodal	Zero-cost, MIT licence
LLM — Fallback	Ollama (Qwen2.5-3B)	Local CPU inference, offline capable
LLM — Last Resort	Gemini 2.5 Flash API	High quality, cost-optimized
AI Streaming	Server-Sent Events (SSE)	Real-time token streaming
Reverse Proxy	Nginx	Load balancing, SSE buffering control
Containerization	Docker + Docker Compose	Reproducible, portable deployment
CI/CD	GitHub Actions	Automated testing and deployment
Code Repository	GitHub	Open-source collaboration

1.4.9 LLM Selection Rationale

The LLM architecture employs a three-tier fallback system designed to optimise for cost, capability, reliability, and suitability for the Ethiopian educational context.

Tier 1 — Primary: Phi-4-Multimodal (Microsoft). Phi-4-Multimodal (5.6B parameters, ~11GB in FP16) runs on Kaggle’s free T4 GPU (16GB VRAM), providing LLM inference at zero financial cost—no cloud API fees, no credit card required, no usage limits. It is released under the permissive

MIT license, ensuring no vendor lock-in. With native multimodal support (text, image, audio), it opens future possibilities for voice-based tutoring and diagram analysis. At ~74% MMLU, it provides adequate quality for quiz generation and explanation tasks. The primary limitation is inference speed (~16 tok/s) on Kaggle’s T4 GPU, mitigated by response caching.

Tier 2 — Secondary: Ollama (Qwen2.5-3B, Alibaba). The Ollama runtime serves Qwen2.5-3B on any modern CPU with no GPU required. At only 3 billion parameters (~6GB in FP16), inference completes in 30–90 seconds—slow but functional for cached or low-traffic scenarios. Despite its small size, Qwen2.5-3B achieves 65.9% on MATH, 86.7% on GSM8K, and 74.4% on HumanEval. Crucially, Ollama provides complete offline capability—the system remains functional even when both the Phi-4 endpoint and the Gemini API are unavailable, a realistic scenario for Ethiopian deployments in areas with intermittent internet connectivity. The Ollama server is deployed alongside the AI Service container within the same Docker Compose stack.

Tier 3 — Tertiary: Gemini 2.5 Flash (Google). Gemini 2.5 Flash is the last-resort fallback, used only when both local models are unavailable. With 88.4% on MMLU and 82.8% on GPQA Diamond, it outperforms both local alternatives by a wide margin. At 186–232 output tokens per second with a time-to-first-token of ~0.44 seconds, it is 10–14x faster than Phi-4-Multimodal. The 1,048,576 token context window enables loading entire textbooks and comprehensive documentation into a single prompt. At \$0.15 per million input tokens and \$0.60 per million output tokens (with cached input at \$0.03/1M), it represents a 99.9% cost reduction from the original GPT-4 budget estimate of \$450/month. The critical limitation is that it is proprietary and API-only, requiring internet connectivity.

Cost Comparison with Original GPT-4 Plan: Table 1.2 compares the costs of the three-tier approach against the originally planned GPT-4 model.

Table 1.2. *Cost comparison: three-tier LLM architecture vs. original GPT-4 plan*

Model	Input Cost	Output Cost	Monthly Cost	6-Month Cost
GPT-4 (original plan)	\$30.00/1M	\$60.00/1M	~\$450	\$2,700
Gemini 2.5 Flash	\$0.15/1M	\$0.60/1M	~\$1.50–3.00	~\$15
Phi-4-Multimodal	\$0.00	\$0.00	\$0.00	\$0.00
Qwen2.5-3B (Ollama)	\$0.00	\$0.00	\$0.00	\$0.00

The three-tier architecture achieves a 99.5% reduction in LLM operating costs compared to the original GPT-4 plan (\$2,700 → ~\$15), while providing superior reliability through graceful degradation and complete offline capability.

1.4.10 Development Phases

The project is divided into six sequential phases over six months:

- **Phase 1 (Weeks 1–4): Foundation & Architecture.** System architecture documentation, database schema design, API specification, frontend component library setup, CI/CD pipeline configuration. Milestone: Architecture review approved, development environment setup complete.
- **Phase 2 (Weeks 5–8): Ontology Design & Teacher Model.** Domain ontology for Frontend Development (30–50 learning nodes), ontology validation framework, Teacher Model implementation, expert verification process. Milestone: Pilot ontology created and verified.
- **Phase 3 (Weeks 9–12): Core Features — Gatekeeper & Tutor Model.** Gatekeeper Pattern implementation, Tutor Model (runtime LLM integration), quiz generation and evaluation logic, SERP API integration for supplementary resource discovery. Milestone: Gatekeeper logic tested, first resource recommendations surfaced.
- **Phase 4 (Weeks 13–16): Adaptation & Persistence.** Mastery Decay algorithm, multi-path branching UI, learner progress persistence, spaced repetition notifications, learner dashboard. Milestone: Branching paths selectable in UI, progress data persisted.
- **Phase 5 (Weeks 17–20): Platforms & Optimization.** Web interface fully functional, mobile interface MVP, performance optimization, scalability testing, cost analysis. Milestone: Web platform user testing with 10 learners, performance benchmarks meeting SLAs.
- **Phase 6 (Weeks 21–24): Validation & Deployment.** Empirical evaluation with 50–100 Ethiopian learners, final bug fixes, production deployment, documentation and developer guides. Milestone: Pre/post assessments completed, statistical analysis of learning gains.

1.5 Plan of Activities

The project spans six months (24 weeks) divided into six phases with specific milestones, deliverables, and team responsibilities as detailed in Table 1.3.

Table 1.3 provides a detailed weekly breakdown across 12 sprints.

Table 1.3. Detailed

Weeks	Sprint	Focus Areas	Deliverables
1–2	Sprint 1	Project kickoff, architecture design	Architecture doc v0.1, tech stack matrix, C
3–4	Sprint 2	DB schema, API spec, CI/CD setup	PostgreSQL schema, OpenAPI spec, GitH
5–6	Sprint 3	Ontology framework, Teacher Model prototype	Ontology JSON schema, Teacher Model p

Weeks	Sprint	Focus Areas	Deliverables
7–8	Sprint 4	Pilot domain ontology (Frontend), expert verification	Frontend ontology (40 nodes), verification
9–10	Sprint 5	Gatekeeper Pattern implementation, quiz service	Gatekeeper logic, quiz service API, LL
11–12	Sprint 6	Tutor Model, resource discovery (SERP API)	Tutor Model service, SERP API wrapper
13–14	Sprint 7	Mastery Decay, spaced repetition, dashboard UI	Decay algorithm, micro-quiz generation
15–16	Sprint 8	Multi-path branching, prerequisite validation	Branching logic, prerequisite graph va
17–18	Sprint 9	React web interface (DAG, quiz, dashboard)	React components (React Flow DAG v
19–20	Sprint 10	Flutter mobile interface	Flutter widgets (roadmap, quiz, notific
21–22	Sprint 11	Performance optimization, scalability testing	Performance benchmarks (load <2s, A
23–24	Sprint 12	Empirical evaluation, production deployment	User study results, deployment checkli

1.6 Budget Breakdown

Table ?? presents the six-month budget for development, testing, and initial deployment for 50–100 concurrent test users.

1.7 Significance of the Project

1.7.1 Academic Significance

AI-Driven Personalized Learning Roadmap Generator And Tutor contributes to three areas of academic research and practice in computer science and educational technology:

1. Hybrid AI Architecture for Curriculum Design: Traditional ITS faced a knowledge engineering bottleneck—each domain required thousands of hours of expert time. Modern LLMs solve scalability but introduce hallucination. AI-Driven Personalized Learning Roadmap Generator And Tutor operationalises a novel Skeleton & Flesh variant of RAG specifically for curriculum design, bridging decades of ITS research with contemporary AI. This architecture is novel in its application to self-directed learning and could inform future EdTech research on verified, scalable knowledge representation.

2. Assessment-Driven Resource Adaptation: The Gatekeeper Pattern and three-tier remediation operationalise research from cognitive load theory [51] and multimedia learning theory [11, 43]. By grounding assessment outcomes in adaptive resource selection rather than just scoring feedback, AI-Driven Personalized Learning Roadmap Generator And Tutor advances the “knowing how to teach” dimension of adaptive tutoring, extending beyond item selection (IRT-based) to meta-cognitive strategy selection.

3. Decay-Aware Roadmap Evolution: By integrating Ebbinghaus’s forgetting curve and modern spaced repetition research directly into curriculum roadmap interfaces, AI-Driven Personalized Learning Roadmap Generator And Tutor makes learning persistence a first-class system feature rather than a learner-managed tool. This is novel in positioning decay as a dynamic roadmap state (Green/Yellow/Red), enabling visualisations and system-level reminders that enhance retention science’s practical impact.

1.7.2 Regional and Socioeconomic Significance (Ethiopian Context)

1. Educational Access and Democratization: In Ethiopia, access to quality technical education is limited by geographic constraints, institutional capacity, and cost. AI-Driven Personalized Learning Roadmap Generator And Tutor is free, web-accessible, and locally developed, removing these barriers. By generating personalized learning content grounded in expert-verified curricula and adapting it to each learner’s performance, the system enables thousands of Ethiopian students to achieve technical competence without expensive mentorship or bootcamps.

2. Employment Readiness and Economic Mobility: The software engineering job market in Ethiopia is growing rapidly, with demand far exceeding supply. AI-Driven Personalized Learning Roadmap Generator And Tutor’s focus on contemporary programming domains and assessment-driven progression ensures learners develop verifiable, job-aligned competencies, enhancing employment prospects and reducing skill gaps for employers.

3. Reduction of Brain Drain and Talent Retention: High-quality, accessible technical education increases the competitiveness of local talent and improves earning potential domestically, reducing incentives to emigrate. Retention of technical talent strengthens Ethiopia’s domestic tech ecosystem.

4. Open-Source and Knowledge Sharing Model: AI-Driven Personalized Learning Roadmap Generator And Tutor is positioned as an open-source project with source code publicly available on GitHub. This enables Ethiopian tech communities, universities, and NGOs to contribute, adapt, and deploy locally, building local capacity in EdTech development.

1.7.3 Technical and Practical Significance

Scalability of Adaptive Learning: By decoupling curriculum knowledge (the skeleton) from runtime tutoring (the flesh), AI-Driven Personalized Learning Roadmap Generator And Tutor demonstrates how adaptive learning can scale beyond manual curation. The system can extend to additional domains by generating new skeletons with expert verification, without redesigning the tutoring engine.

Search-Augmented Resource Discovery: The use of SERP API for supplementary resource discovery demonstrates a practical approach to surfacing high-quality, domain-relevant learning materials

at scale, applicable to other contexts (medical education, legal education, language learning) where curated search is preferable to open-web retrieval.

Open-Source EdTech Contribution: AI-Driven Personalized Learning Roadmap Generator And Tutor contributes to the open-source EdTech ecosystem alongside platforms like Moodle and Open edX, enabling researchers worldwide to build upon the work.

1.7.4 User and Stakeholder Significance

Student Learners (Primary): Students gain structured, adaptive guidance, reducing time-to-competence, improving retention, and increasing confidence. Multi-Path Branching respects learner agency and interests, increasing intrinsic motivation.

Educators and Mentors (Secondary): Teachers gain visibility into learner progress through dashboards and analytics, identify struggling learners (flagged by Gatekeeper failures), and provide targeted support.

Employers: An increased pool of technically skilled, job-ready graduates benefits employers. The system can be integrated into corporate training programs.

Open-Source and Developer Communities (Indirect): As an open-source platform, AI-Driven Personalized Learning Roadmap Generator And Tutor invites contributions from the developer community—new domain ontologies, AI prompt improvements, UI components, and language adaptations. This creates a compounding knowledge base that benefits both the project and contributors seeking practical EdTech experience.

1.8 Outline of the Study

This document is structured as follows:

Chapter 1: Introduction (Current Chapter) Articulates the problem landscape (tutorial hell, absence of adaptive feedback, lack of mastery-based progression, cognitive overload, generic roadmaps, hallucination). Defines objectives for system design, implementation, and validation. Specifies scope and limitations, outlines methodology (Agile-Scrum, microservices, Skeleton & Flesh, Gatekeeper, Mastery Decay, Multi-Path Branching, SERP API integration), details the plan of activities, presents the budget, and justifies academic, regional, technical, and stakeholder significance.

Chapter 2: Literature Review Synthesises research on ITS evolution, adaptive learning, knowledge representation (ontologies and RAG), assessment-driven progression, spaced repetition, per-

sonalisation, resource aggregation, and content curation. Analyses gaps in existing solutions and establishes three core lessons: ontology-guided LLM solves hallucination, assessment-driven resource adaptation extends tutoring, and decay-aware adaptation operationalises spacing research.

Chapter 3: Problem Analysis and Modeling Provides detailed requirements elicitation methodology, specifies functional and non-functional requirements, and presents system models: use case diagrams, UML class diagrams, sequence diagrams, activity diagrams, and state machine diagrams.

Chapter 4: System Design and Architecture Presents the detailed system architecture including microservices layers, Skeleton & Flesh hybrid AI, Gatekeeper Pattern, Mastery Decay, Multi-Path Branching, database schema, API specifications, and deployment architecture.

Chapter 5: Implementation and Development Documents development process, technologies used, code organisation, integration of the three-tier LLM stack and SERP API, SSE streaming implementation, certificate system, testing strategy, and lessons learned.

Chapter 6: Evaluation and Validation Specifies evaluation methodology with empirical user study, defines success metrics (learning gains, engagement, usability), presents results, and discusses findings and limitations.

Chapter 7: Conclusions and Recommendations Summarises project achievements, discusses how objectives were met, identifies technical and research contributions, and recommends future work and sustainability strategies.

Chapter 2

Literature Review

This chapter surveys the academic literature and related work that collectively inform the design of AI-Driven Personalized Learning Roadmap Generator And Tutor. The review is organised into three parts: Section 2.1 examines related works across twelve research streams spanning intelligent tutoring systems, knowledge representation, adaptive learning, spaced repetition, personalisation, gamification, and AI-driven educational tools; Section 2.2 identifies the key milestones in each stream and analyses the critical gaps that justify the proposed system; and Section 2.3 synthesises three overarching lessons learned from the literature that directly drive the design decisions in subsequent chapters.

2.1 Study of Related Works

2.1.1 Evolution of Intelligent Tutoring Systems

The foundation of personalised learning traces back to Bloom’s seminal work on the “two-sigma problem” [5], which demonstrated that one-on-one tutoring produces two standard deviations of improvement over traditional classroom instruction. This groundbreaking finding motivated decades of research into Intelligent Tutoring Systems (ITS) that attempt to replicate tutor-like adaptivity at scale. The psychological principle underlying this finding is well-established: individual feedback, adaptive pacing, and immediate error correction—hallmarks of human tutoring—significantly accelerate learning outcomes [35].

VanLehn’s comprehensive meta-analysis [53] identified three persistent ITS challenges: knowledge representation, student modelling, and pedagogical strategy selection. Traditional rule-based ITS systems (e.g., LISP Tutor and Cognitive Tutors) achieved measurable learning gains of up to one sigma in controlled studies but faced severe scalability barriers due to manual knowledge engineering requirements. Each domain required expert intervention to codify learning rules in propositional or

constraint-based formalisms, making rapid curriculum expansion impractical. In the context of software engineering education, this meant that developing a complete ITS for even a single programming language required thousands of hours of expert time—a constraint particularly acute in resource-limited contexts like Ethiopia, where educational expertise and funding are scarce.

Woolf’s extensive synthesis [59] articulated that effective tutoring requires three essential pillars: (1) accurate, comprehensive domain knowledge representation that captures prerequisites, learning outcomes, and misconceptions; (2) sophisticated understanding of individual student misconceptions and learning styles; and (3) timely, contextually appropriate adaptive feedback that neither over-scaffolds nor under-supports learners. However, most contemporary platforms—including free platforms like Khan Academy and paid platforms like Coursera—implement only passive sequencing. They present content in a predetermined, linear order without active performance-driven progression gates or adaptive resource swapping based on specific failure modes. Learners can technically “complete” a course without demonstrating mastery, leading to knowledge gaps that compound in subsequent courses.

AI-Driven Personalized Learning Roadmap Generator And Tutor addresses this critical gap by embedding assessment-driven progression through the Gatekeeper Pattern, ensuring that advancement to new conceptual nodes is contingent on demonstrated mastery (defined as $\geq 80\%$ performance on targeted assessment), not merely time spent or content completion. This operationalises Bloom’s principle of mastery-based progression and directly responds to the two-sigma finding by enforcing individual accountability and performance-based advancement, thereby approximating the adaptive feedback loop of one-on-one tutoring at scale.

2.1.2 Knowledge Space Theory and Ontology-Based Curriculum Design

Knowledge Space Theory (KST), introduced by Doignon & Falmagne [16], provides a mathematical framework for representing the structure of a knowledge domain as a partially ordered set. It models the idea that mastering certain concepts is a prerequisite for mastering others—exactly like a directed acyclic graph (DAG). A “knowledge state” is the set of items a learner currently knows, and the space of all reachable knowledge states is constrained by prerequisite (surmise) relations. Falmagne et al. [20] extended the theory with practical methods for building and searching knowledge spaces, and Vuong et al. [55] demonstrated algorithmic extraction of prerequisite chains from existing curricula—the same graph structure used by the ontology builder in AI-Driven Personalized Learning Roadmap Generator And Tutor.

The domain ontology in AI-Driven Personalized Learning Roadmap Generator And Tutor is a direct implementation of KST: every `LearningNode` carries a `prerequisites` array, and a node unlocks only when all prerequisites are mastered. The DAG structure (branching paths, convergence

points, root nodes) defines the valid progressions through the knowledge space. A learner cannot skip from root to advanced nodes without traversing the prerequisite chain—this is the KST surmise relation in practice.

Ontology engineering provides a further layer of rigour. Gruber [24] defined an ontology as “a specification of a conceptualisation” and later formalised design principles for knowledge-sharing systems [25]. In educational technology, ontologies enable automated reasoning about curriculum structure, prerequisite inference, and content generation. Holohan et al. [30] demonstrated using a subject-domain ontology as the basis for automated generation of e-learning exercises, directly analogous to using `learningOutcomes` from `LearningNode` for AI quiz generation. Mizoguchi & Bourdeau [44] argued that explicit ontological representation of domain knowledge is essential for avoiding hallucination and maintaining coherence in AI-driven educational systems—a central rationale for the Skeleton & Flesh strategy in AI-Driven Personalized Learning Roadmap Generator And Tutor.

2.1.3 Knowledge Representation: From Ontologies to Retrieval-Augmented Generation

The shift from brittle, hand-coded knowledge bases to flexible, ontology-driven approaches marks a critical evolutionary milestone in AI-assisted education. Ontologies—formal specifications of conceptual schemas that enumerate concepts, relationships, and rules within a domain—provide machine-readable representations of domain knowledge that enable both semantic reasoning and knowledge sharing. In the educational technology domain specifically, ontologies have been applied to competency modelling frameworks (IMS RDCEO for learning design, O*NET frameworks for job competencies) and curriculum mapping (mapping learning outcomes to instructional activities).

However, the rapid emergence of Large Language Models (LLMs) in 2022–2024 introduced a profound technical challenge and opportunity. While LLMs can generate fluent, contextually appropriate natural language explanations and appear to “understand” domain concepts, they suffer from a critical failure mode: hallucination. Hallucination is the phenomenon where an LLM produces confident but factually incorrect, logically incoherent, or fabricated outputs [42, 58]. In technical education, this is catastrophic. A student receiving incorrect information about a JavaScript asynchronous pattern, cryptographic algorithm, or database indexing strategy may internalise false knowledge, leading to buggy code or flawed system design. The stakes are particularly high in self-directed learning, where learners lack an instructor to correct misunderstandings.

Retrieval-Augmented Generation (RAG) has emerged as a proven mitigation strategy [38]. RAG decouples knowledge generation into two epistemically distinct phases: (1) retrieve relevant, pre-verified information from a curated knowledge base, and (2) generate contextual explanations and

elaborations grounded in that retrieved content. By constraining generation to retrieved facts, RAG significantly reduces hallucination while maintaining the flexibility and naturalness of generative models. For example, a RAG system asked to explain JavaScript closures would first retrieve verified documentation (e.g., from MDN Web Docs or official ECMAScript specifications), then synthesize an explanation grounded in those facts, rather than fabricating examples.

AI-Driven Personalized Learning Roadmap Generator And Tutor operationalises a strategic variant of RAG through the Skeleton & Flesh hybrid architecture:

- **The Skeleton (offline, Teacher Model):** A powerful LLM (e.g., Gemini 2.5 Flash, GPT-4, Claude) is used once, in a controlled offline setting, to generate a comprehensive, logically coherent ontology—a DAG of learning nodes, prerequisite relationships, learning outcomes, and sub-skills. This skeleton is then rigorously verified by domain experts (software engineers, educators) who review each node for accuracy, granularity, and logical sequencing. Once verified, this skeleton is static and deterministic; it cannot be altered by runtime generation and serves as an immutable scaffold.
- **The Flesh (runtime, Tutor Model):** A lightweight LLM or rule-based engine generates contextual explanations, quizzes, and resource recommendations at runtime, but within the bounds defined by the skeleton. It cannot invent new learning nodes, create arbitrary prerequisites, or deviate from the verified structure. It elaborates on pre-verified nodes using different linguistic styles, provides examples within the node’s scope, and generates quiz questions targeting specific learning outcomes defined in that node.

This hybrid approach combines the rigor and verifiability of ontologies with the flexibility, naturalness, and scalability of LLMs. It solves the hallucination problem in curriculum design (the skeleton is verified, thus factually sound) while maintaining scalability beyond manual curation (the flesh is generative, so it does not require scripting thousands of individual quiz questions or explanations). The verified skeleton can be extended by domain experts or crowdsourced reviewers in a controlled process, avoiding the expertise bottleneck that limited traditional ITS development.

2.1.4 Bayesian Knowledge Tracing and Mastery State Machines

Bayesian Knowledge Tracing (BKT), introduced by Corbett & Anderson [12], is a probabilistic model for estimating a learner’s latent knowledge state. It models learning as a two-state hidden Markov model: a learner is either in the “unlearned” or “learned” state for each skill, and quiz performance provides observable evidence to update the belief about which state they are in. BKT introduced the idea of tracking knowledge state per skill across multiple interactions, which is now the foundation of

nearly all adaptive learning systems. Piech et al. [47] extended BKT with recurrent neural networks (Deep Knowledge Tracing), demonstrating that deep sequence models can outperform classic BKT by capturing richer temporal patterns.

The masteryState machine in AI-Driven Personalized Learning Roadmap Generator And Tutor (`not_started` $\rightarrow$ `in_progress` $\rightarrow$ `review_needed` $\rightarrow$ `mastered` $\rightarrow$ `relearn`) is a deterministic implementation of the same concept BKT models probabilistically. The Gatekeeper’s five-tier scoring transitions the state based on accumulated evidence (attempts, scores), and the decay service introduces the temporal dimension that BKT typically omits. The fields `bestQuizScore` and `attemptsCount` on `LearnerNodeProgress` are the direct analogues of BKT’s performance observations.

2.1.5 Adaptive Assessment and Item Response Theory

Item Response Theory (IRT), formalised by Lord [40], is a psychometric framework that models the probability of a correct response as a function of both learner ability and item difficulty. Computerised Adaptive Testing (CAT) [57] uses IRT to dynamically select questions that maximise information about the learner’s ability level—harder questions for high-ability learners, easier ones for struggling learners. Weak area targeting is a complementary approach: analysing which specific items were answered incorrectly to identify knowledge gaps and retarget instructional content. Lilley & Barker [39] demonstrated practical CAT deployment in an e-learning context, including formative weak-area feedback.

AI-Driven Personalized Learning Roadmap Generator And Tutor implements an IRT-inspired difficulty calibration through `computeAdaptiveDifficulty`, which adjusts quiz difficulty (± 1 from the node’s static ontology value) based on the learner’s best score and overall average. The `detectWeakAreas` function analyses `QuizAttempt.answers` against `QuizQuestion.correctAnswer` to extract the specific questions a learner answered incorrectly. These question texts are then injected into subsequent quiz and explanation prompts as `weakAreas`, ensuring that re-attempts target the learner’s actual knowledge gaps rather than reusing generic content.

2.1.6 Adaptive Learning, Mastery-Based Progression, and Assessment-Driven Adaptation

Adaptive learning systems respond to learner performance by dynamically adjusting content sequencing, difficulty levels, or pacing. Most commercial adaptive platforms (e.g., DreamBox Learning, Knewton, Smart Sparrow) rely on IRT or Bayesian networks to model learner ability and item difficulty, then algorithmically select the next item to maximise information gain about the learner’s skill level [52]. However, IRT-based adaptation has a critical limitation in the context of curriculum sequencing: it

optimises for item selection but not for resource or explanation type adaptation. Under an IRT system, if a learner fails a JavaScript fundamentals quiz (e.g., scores 45%), the adaptive engine might increase item difficulty slightly or offer a hint, but it typically serves the same conceptual explanation again, perhaps with a visual diagram added but the modality of instruction remains unchanged.

Mastery-Based Learning (MBL), pioneered by Bloom [4], asserts that all learners can achieve high competence given sufficient time and appropriate corrective instruction. Bloom’s canonical paper argued that 95% of students can achieve mastery if instruction is individualised and corrective feedback is provided. The core principle is that progress should be criterion-gated (a minimum performance threshold) rather than time-gated (a fixed-duration course unit). Anderson & Block [2] provided early empirical validation of mastery learning in practice, demonstrating significant achievement gains over conventional instruction. Guskey’s modern review [26] confirmed the continued relevance of mastery learning, with evidence that formative assessment and corrective instruction are the critical active ingredients—both of which are present in AI-Driven Personalized Learning Roadmap Generator And Tutor.

AI-Driven Personalized Learning Roadmap Generator And Tutor introduces a novel layer of adaptation: **Assessment-Driven Resource Adaptation**. Failure modes trigger not just re-teaching but strategic resource modality swapping. This is grounded in cognitive load theory [51] and multimedia learning theory [11, 43], which demonstrate that the effectiveness of instructional modality depends on learner prior knowledge and cognitive load. Specifically, the system implements four remediation tiers:

- **Score 70–79% (partial mastery):** The system unlocks the node, marking progress, but flags it for spaced repetition review. The learner is encouraged to advance but will receive reminder quizzes at strategically timed intervals.
- **Score 50–69% (emerging understanding but gaps):** The system does not unlock the next node. Instead, it records a `resource_swap` adaptation event and instructs the Tutor Model to regenerate the node explanation in a tutorial-oriented, step-by-step style rather than the reference-documentation style used on the first attempt. The hypothesis is that the learner’s comprehension gap stems from learning modality mismatch: reference-style content assumes higher prior knowledge and requires self-directed navigation, while guided, scaffolded exposition lowers cognitive entry barriers. The quiz is then re-attempted with the regenerated explanation.
- **Score 30–49% (low comprehension):** The system records a `prerequisite_review` adaptation event and directs the learner to revisit foundational nodes they may have passed without deep mastery before retrying. This addresses a common failure mode in self-directed learning: learners advance through prerequisite topics superficially, then struggle when those prerequisites are needed in earnest.

- **Score < 30 % (fundamental gaps):** The system records an `instructor_escalation` event, flagging the node for review by a domain expert instructor. This acknowledges the limits of adaptive automation: gaps at this severity may reflect language barriers, undetected prerequisite deficiencies, or learning difficulties that require human pedagogical judgment beyond what automated adaptation can address.

This granular, failure-mode-aware adaptation extends Woolf’s principle of “knowing what to teach” to a more sophisticated principle: “knowing how to teach it”—matching not just content but delivery modality, pacing, and support intensity to learner needs. It operationalises research by Clark & Mayer [11] showing that multimedia modality should be selected based on learner expertise: novices benefit from tutorial structures, while experts benefit from reference structures.

2.1.7 Spaced Repetition and Knowledge Decay

Hermann Ebbinghaus’s empirical finding [19] that forgetting follows a predictable exponential decay curve ($R = e^{-t/S}$) has become foundational to memory and retention science. His original experiments demonstrated that forgetting is not random; it follows a mathematical curve where retention drops sharply in the first days after learning, then levels off. Critically, he also demonstrated that strategically timed review can extend retention indefinitely—the basis for the spaced repetition principle.

Modern implementations and meta-analyses have rigorously validated that spacing intervals optimised to individual retention curves dramatically improve long-term retention compared to massed practice. Cepeda et al. [10] conducted a meta-analysis of 317 experiments and found that retention improved by up to 40–50% when spacing intervals were optimised. Dunlosky et al. [17] further confirmed that distributed practice is one of the most effective learning techniques across a wide range of conditions. Karpicke & Roediger [31] demonstrated the critical importance of retrieval practice, and Kornell & Bjork [34] showed that spacing improves conceptual learning, not merely rote memorisation. Tools like Anki, Quizlet, and SuperMemo have demonstrated the real-world effectiveness of spaced repetition at scale, helping millions of learners retain vocabulary, facts, and concepts.

However, spaced repetition has been largely isolated as a study tool disconnected from curriculum roadmaps. In typical self-directed learning, learners “complete” a course (marking nodes as done), achieve a certificate, and then never revisit the material. Over weeks and months, skills decay according to Ebbinghaus’s curve, but there is no system-level reminder or re-engagement mechanism. A learner who completes “JavaScript Fundamentals” in January may find that by April, they have forgotten core concepts and must re-learn from scratch—a form of learning inefficiency that is widespread but rarely addressed.

The Leitner box system [37] introduced the first practical spaced repetition implementation. The

multi-state mastery transitions in AI-Driven Personalized Learning Roadmap Generator And Tutor are a direct descendant of this model. AI-Driven Personalized Learning Roadmap Generator And Tutor introduces **Mastery Decay**—a visual, algorithm-driven reminder system grounded in Ebbinghaus’s curve and modern spacing research:

- **Initial Mastery (Green status):** A node reaches “Mastered” status via two paths. A `strong_pass` ($\geq 80\%$) starts a 14-day decay timer. A `marginal_pass` (70–79%) also achieves mastered status but starts an accelerated 7-day timer, reflecting that lower initial mastery correlates with faster forgetting. Both cases visualise the node as green; the difference lies in when the first review is triggered.
- **Review Trigger (Yellow status):** When the decay timer expires—14 days for strong passes, 7 days for marginal passes—the node automatically transitions to “Review Needed” (yellow). The strong-pass interval is calibrated on Cepeda et al.’s finding that optimal spacing is approximately 10–20% of the desired retention duration; for a semester-long course (16 weeks), 14 days falls within that range. The shortened marginal-pass interval implements the principle that weaker initial encoding requires more frequent consolidation [10].
- **Relearning Required (Red status):** If a node remains unvisited for more than 30 days, it transitions to “Relearn” (red), signalling that significant skill decay has occurred and deeper re-engagement is needed.
- **Micro-Quiz Trigger:** When a node transitions from green to yellow or red, the learner receives a notification and is presented with a micro-quiz (2–3 questions) targeting that node’s core concepts. Success on the micro-quiz resets the timer; failure triggers resource adaptation.

The roadmap UI visualises this decay prominently, making learner knowledge gaps visible and actionable. Rather than forgetting being an invisible, gradual phenomenon, it is represented as a dynamic system state. This transforms spaced repetition from a learner’s voluntary, self-discipline-dependent tool (e.g., Anki, where many users forget to review decks) into a system-guided, pull-based feature where reminders are automatic and integrated into the learner’s progression pathway.

Research by Karpicke & Roediger [31] and Kornell & Bjork [34] further supports this approach: they found that introducing variability and spacing into learning contexts increases both retention and transfer of knowledge compared to blocked, massed practice. AI-Driven Personalized Learning Roadmap Generator And Tutor’s decay system introduces both spacing (review after 14 days) and variability (micro-quizzes present questions in different formats, contexts, and difficulty levels compared to the initial Gatekeeper quiz).

2.1.8 Personalisation, Adaptive Hypermedia, and Multi-Path Learning

Adaptive hypermedia research, synthesised by Brusilovsky [7, 8], identifies at least three dimensions along which learning can be personalised: (1) content adaptation—which topics or learning materials are presented; (2) sequencing adaptation—in what order topics are presented; and (3) modality adaptation—how content is taught (video, text, interactive, etc.). The learner context pipeline (`buildLearnerContext`) in AI-Driven Personalized Learning Roadmap Generator And Tutor is the learner model component of an adaptive hypermedia system. It assembles six profile dimensions (familiarity level, learning goal, weekly hours, self-description, preferred learning style, and prior skills) plus live performance data, and injects this into every AI content generation call. This parallels the learner model dimensions identified by Brusilovsky & Millán [9]. Bransford et al. [6] argued that effective learning requires attending to learners’ prior knowledge, motivation, and metacognition—the same dimensions captured by `familiarityLevel`, `learningGoal`, and `aboutSelf`.

Most traditional, static roadmaps (e.g., `roadmap.sh`, typical bootcamp curricula) address the first dimension minimally and largely ignore the second and third. They present a fixed, linear sequence: “Learn HTML → CSS → JavaScript → React → Node.js → Databases.” This one-size-fits-all approach ignores that learners differ substantially in learning preferences, prior knowledge, career goals, and cognitive styles [45]. Some learners are visual, front-end focused and need CSS mastery before engaging with JavaScript logic; others are backend-oriented and need databases and server-side logic before styling concerns. A fixed sequence may frustrate the former and confuse the latter.

AI-Driven Personalized Learning Roadmap Generator And Tutor’s Multi-Path Branching feature operationalises Brusilovsky’s second dimension (sequencing adaptation) by offering learner choice at key decision points. The system is designed around a branching tree structure rather than a linear chain:

- After completing foundational nodes, the system presents a branching question: “Which area interests you most: Visual Design & Frontend, Backend Logic & Data, or Data Science & AI?”
- Based on the learner’s choice, the system recommends a sequence of paths: Path A (Frontend-focused: CSS → Advanced CSS → JavaScript DOM Manipulation → Frontend Frameworks → UI/UX), Path B (Backend-focused: Server-Side JavaScript → Databases → API Design → Microservices → DevOps), and Path C (Data Science: Python → Data Structures → Data Analysis Libraries → Machine Learning → Deep Learning).
- All three paths converge at advanced integrative topics (e.g., “Full-Stack Integration”, “System Design and Architecture”), ensuring foundational coherence while respecting learner agency.

Multi-path learning also reflects the reality of software engineering careers: not all software

engineers are full-stack. Some specialise in frontend, others in backend, still others in data science and machine learning. Offering paths that align with learner interests increases the likelihood of career relevance and sustained engagement.

This approach aligns with self-determination theory [48, 13], which identifies autonomy, competence, and relatedness as three fundamental psychological needs that drive intrinsic motivation. Dweck’s research on fixed versus growth mindset [18] further supports the value of offering learner choice: when learners feel agency and perceive learning challenges as aligned with their interests and capabilities, they are more likely to persist through difficulty, adopt a growth mindset, and achieve deeper learning.

Regarding learning styles, the VARK model [23] classifies learners by preferred modality (Visual, Aural, Read/Write, Kinaesthetic). Felder & Silverman’s Index of Learning Styles [21] adds dimensions for active/reflective and sequential/global preferences. While the strict matching hypothesis has been challenged [45], presenting content in multiple formats and allowing learner preference selection consistently improves engagement. AI-Driven Personalized Learning Roadmap Generator And Tutor captures `preferredLearningStyle` at enrollment and injects style-specific instructions into AI prompts: visual learners receive prompts requesting diagrams and flowcharts; hands-on learners receive prompts emphasising code snippets and practical exercises; reading-preference learners receive more detailed textual exposition; video-preference learners are flagged for resource prioritisation.

2.1.9 Resource Aggregation and Content Curation

A persistent challenge in self-directed learning is discovery: where to find high-quality, trustworthy resources? Learners often face a fragmentation problem—the same concept might be explained in a blog post, a YouTube video, official documentation, and a dozen Stack Overflow answers, requiring learners to aggregate and synthesise information across sources. Alternatively, learners face a quality variability problem: user-generated content ranges from excellent to dangerously incorrect, with no consistent signal of reliability.

Existing resource aggregation approaches fall into three categories:

1. **Manual Expert Curation (Khan Academy, edX, Coursera):** Experts carefully select, often co-creating, high-quality content. Advantage: consistent quality and pedagogical sequencing. Disadvantage: resource-intensive, limited coverage for emerging domains, and difficult to scale beyond funded institutions.
2. **User-Generated Curation (Reddit, GitHub awesome-lists, Hacker News):** Community members recommend resources they have found valuable. Advantage: rapid response to emerging

domains and grassroots discovery. Disadvantage: inconsistent quality, outdated links, and often reflects the biases of vocal community members rather than universal learner needs.

3. **Algorithmic Recommendation (YouTube recommendations, LinkedIn Learning):** Algorithms rank resources based on engagement metrics (views, clicks, time spent). Advantage: scales to massive content catalogs. Disadvantage: engagement metrics do not correlate with learning efficacy; popular content is often entertainment-oriented, not pedagogically sound.

AI-Driven Personalized Learning Roadmap Generator And Tutor implements a two-layer content architecture designed to maintain pedagogical quality while scaling beyond the bottleneck of manual curation:

- **Offline Expert Curation Layer:** During the ontology authoring phase, domain experts attach verified reference links and curated reading materials directly to each LearningNode. These resources are reviewed for accuracy, currency, and alignment with the node's learningOutcomes before publication. Because this curation is decoupled from runtime generation, it benefits from deliberate expert judgement without creating a latency bottleneck during live learning sessions.
- **Ontology-Grounded AI Generation Layer:** At runtime, the Tutor Model (three-tier LLM stack) generates node-specific explanations constrained by the verified skeleton. The prompt includes the node's title, learning outcomes, difficulty level, and prerequisite chain—ensuring that no hallucinated topics or spurious references enter the generated content. These explanations are delivered token-by-token via Server-Sent Events (SSE), providing a responsive, conversational learning experience without full-response latency.
- **SERP API Supplementary Discovery:** For nodes where dynamically surfacing current community resources adds value, the system can query the SERP API to retrieve recently published, relevant articles and tutorials. Because queries are scoped to the node's verified learning outcomes, results remain pedagogically aligned. Domain expert instructors may review and promote high-quality results into the curated layer.
- **Instructor Oversight and Feedback:** Domain expert instructors monitor flagged adaptation events and learner analytics through a dedicated portal. Nodes where learners consistently fail or where decay rates are high signal candidates for resource revision. This creates a quality feedback loop grounded in actual learner performance data rather than editorial assumption.

This architecture avoids the pitfalls of open-web scraping while scaling beyond manual per-node curation. The AI generation layer provides immediate, personalised content that no static curation model can match; the offline expert curation layer provides a verified factual anchor that no unconstrained LLM

can guarantee; and the SERP API discovery layer bridges the two by surfacing current, community-validated supplementary material within principled ontological bounds.

2.1.10 Gamification in Educational Technology

Gamification—the application of game design elements (points, badges, leaderboards, progress bars, streaks, challenges) to non-game contexts—has been shown to increase engagement, completion rates, and intrinsic motivation in educational settings, particularly when reward structures are aligned with learning objectives rather than trivial actions. Deterding et al. [15] provided the formal definition and taxonomy of game design elements that directly maps to the features implemented in AI-Driven Personalized Learning Roadmap Generator And Tutor. Hamari et al. [27] systematically reviewed 24 empirical studies and found positive effects on engagement and motivation, with the strongest effects in educational and health contexts. Denny [14] studied badge systems in a programming education context and found significant increases in student activity and performance.

AI-Driven Personalized Learning Roadmap Generator And Tutor implements four canonical game mechanics: experience points (XP) awarded for quiz passes scaled by score tier and attempt efficiency; level progression based on cumulative XP; streak tracking for consecutive days of activity; and badges for milestone achievements (first mastery, 5-day streak, quiz ace, speed learner, completionist). A weekly goal system adds time-bounded challenges. All XP events are logged for history display. These mechanics are designed to reward learning behaviours (mastery, consistency, improvement) rather than mere participation, following the theoretical model proposed by Landers [36] that explains how gamification affects learning through behavioural engagement and attitudinal change.

Complementing these intrinsic-motivation mechanics, AI-Driven Personalized Learning Roadmap Generator And Tutor issues a formal **verifiable completion certificate** upon roadmap mastery. The certificate carries a cryptographically unique public identifier and is accessible via a persistent, shareable verification URL—enabling learners to provide employers or institutions with an independently verifiable record of demonstrated competence. This bridges the gap between intrinsic gamification rewards (which motivate engagement) and extrinsic recognition signals (which motivate completion and credentialing) [15, 27].

2.1.11 Large Language Models as Educational Tools

LLMs have emerged as transformative educational tools. Kasneci et al. [32] provided a comprehensive review of LLM capabilities in education, covering personalised tutoring, assessment generation, and feedback—all three functions of the AI-Driven Personalized Learning Roadmap Generator And Tutor AI service. Macina et al. [41] evaluated LLM-based dialogue tutoring systems and found that contextual

awareness (knowing who the learner is and what they have done) is the key differentiator for effective AI tutoring. Kochmar et al. [33] provided direct empirical evidence that AI-generated personalised feedback (as opposed to generic feedback) produces measurable learning gains—validating the core claim of the personalised explanation and quiz approach.

The Teacher-Student model in knowledge distillation [29]—using a large, capable model to generate content consumed by a smaller model or directly by learners—provides architectural inspiration for the Skeleton & Flesh strategy. AI-Driven Personalized Learning Roadmap Generator And Tutor uses Gemini (with Phi-4/Ollama local fallback and circuit breaker pattern for resilience) to generate explanations, quizzes, and instructor answers. All three calls receive a full `LearnerContext` object, making the generated content specific to each learner’s profile and progress state.

A particularly significant implementation is the **Streaming AI Instructor**: a persistent, node-scoped conversational tutor within each learning node that delivers responses token-by-token via Server-Sent Events (SSE). Rather than waiting for a complete LLM response before displaying anything, the interface renders each token as it arrives, approximating the flow of a live human tutor explaining a concept in real time. Macina et al. [41] identified immediacy and contextual awareness as the two differentiating factors of effective AI tutoring; SSE streaming addresses both by eliminating response latency and maintaining the full node context across every exchange.

2.1.12 Learning Analytics and Progress Dashboards

Learning Analytics (LA) encompasses the measurement, collection, analysis, and reporting of data about learners and their contexts, with the purpose of understanding and optimising learning and the environments in which it occurs [50]. Ferguson [22] provided a systematic overview of LA goals and techniques, including prediction of completion and personalisation of learning. Verbert et al. [54] studied the effectiveness of LA dashboards and found that surfacing analytics to learners improves self-regulation and awareness.

AI-Driven Personalized Learning Roadmap Generator And Tutor implements LA through the `LearnerVelocity` model, which records `estimatedHours`, `actualHours`, `startedAt`, `completedAt`, and `velocityRatio` for each mastered node. The `getTimelineEstimate` function uses these records to compute a velocity-adjusted completion forecast—a classic LA prediction task. The `InsightsPage` (`InsightsPage.tsx`) and the velocity card (`VelocityCard.tsx`) surface this data as a descriptive dashboard. The `getAverageVelocity` function feeds a multiplier back into timeline estimates, closing the data-action loop that defines prescriptive LA.

2.1.13 Academic Foundations: Synthesis of Educational Psychology

Beyond the specific technical streams surveyed above, the design of AI-Driven Personalized Learning Roadmap Generator And Tutor is grounded in a set of broader educational psychology theories that collectively inform the pedagogical approach.

Bloom's Taxonomy of Educational Objectives [3] classifies cognitive skills into six hierarchical levels: Remember, Understand, Apply, Analyse, Evaluate, and Create. This taxonomy guides the design of learning outcomes and quiz questions in AI-Driven Personalized Learning Roadmap Generator And Tutor. Foundation-level nodes target Remember and Understand levels; intermediate nodes target Apply and Analyse; advanced nodes target Evaluate and Create. The quiz generation prompt explicitly specifies the target Bloom's level to ensure alignment between learning outcomes and assessment items.

Cognitive Load Theory (CLT) [51] distinguishes between intrinsic, extraneous, and germane cognitive load. The Skeleton & Flesh architecture reduces extraneous load by presenting a structured, verified curriculum rather than forcing learners to navigate fragmented resources. The Gatekeeper's assessment-driven adaptation prevents cognitive overload by ensuring learners master prerequisite knowledge before advancing, thereby managing intrinsic load. The adaptive resource modality swapping (documentation vs. tutorial) is directly informed by CLT's finding that instructional format significantly affects cognitive load depending on learner expertise.

Self-Determination Theory (SDT) [48, 13] identifies autonomy, competence, and relatedness as three fundamental psychological needs. AI-Driven Personalized Learning Roadmap Generator And Tutor addresses autonomy through Multi-Path Branching, allowing learners to choose specialisation paths aligned with their interests; competence through the Gatekeeper's mastery-based progression, which provides clear evidence of skill development; and relatedness through the AI instructor's personalised, conversational feedback and the community-oriented features of the platform.

Vygotsky's Zone of Proximal Development (ZPD) [56] describes the difference between what a learner can do independently and what they can achieve with guidance. The adaptive difficulty calibration and resource modality adaptation in AI-Driven Personalized Learning Roadmap Generator And Tutor operationalise ZPD by dynamically adjusting the level of scaffolding: learners in the "fail low" tier receive more structured support (tutorials, prerequisite review), while learners demonstrating mastery receive less scaffolding (documentation references, challenge projects).

Constructivism [46, 56] posits that learners actively construct knowledge through experience and reflection rather than passively receiving information. The project-based learning recommendations triggered by the Gatekeeper after successful mastery align with constructivist principles: learners are encouraged to apply newly acquired knowledge in authentic, challenging projects. The branching path

structure respects the constructivist view that learning trajectories are individually constructed rather than universally prescribed.

Multimedia Learning Theory [43] establishes that people learn more deeply from words and pictures than from words alone, and that instructional design should follow principles such as the multimedia principle, the contiguity principle, and the modality principle. The adaptive resource swapping logic in AI-Driven Personalized Learning Roadmap Generator And Tutor implements these principles by matching resource modality to learner performance: struggling learners receive multimedia-rich resources (tutorials with video, diagrams, interactive examples), while proficient learners receive text-dense resources (documentation, references).

Connectivism [49] recognises that learning in the digital age occurs within networks of information sources and that the ability to find and apply knowledge is as important as possessing it. The ontology-guided AI content generation and SERP API-based supplementary resource discovery in AI-Driven Personalized Learning Roadmap Generator And Tutor embodies connectivist principles by linking learners to a curated, dynamically refreshed network of high-quality resources, rather than confining them to a static, closed content repository.

These twelve academic foundations collectively inform the design principles that guide the system architecture, the pedagogical algorithms, and the user experience of AI-Driven Personalized Learning Roadmap Generator And Tutor. Table 2.1 in Section 2.2 provides a condensed reference mapping each feature to its academic lineage.

2.2 Identifying Milestones of the Related Literatures and Finding the Gaps

2.2.1 Key Milestones

The literature surveyed above reveals twelve distinct research streams that collectively inform the design of AI-Driven Personalized Learning Roadmap Generator And Tutor. Table 2.1 provides a condensed reference mapping each system feature to its academic lineage, the key milestone paper, and its publication venue.

Table 2.1. *Summary reference: academic foundations of AI-Driven Personalized Learning Roadmap Generator And Tutor*

Feature		Academic Field		Key Paper		Publication
DAG graph	prerequisite	Knowledge Theory	Space	Doignon & Fal-	magne [16]	<i>IJMMS</i> 1985
Node unlock logic		Knowledge Theory	Space	Vuong et al. [55]		<i>EDM</i> 2011

Feature	Academic Field	Key Paper	Publication
5-tier gatekeeper scoring	Mastery-Based Learning	Bloom [4]	<i>Evaluation Comment</i> 1968
Adaptation on failure	Mastery-Based Learning	Guskey [26]	<i>J. Advanced Academics</i> 2007
Decay intervals	Spaced Repetition	Ebbinghaus [19]	<i>Memory</i> 1885
Distributed practice	Spaced Repetition	Cepeda et al. [10]	<i>Psychological Bulletin</i> 2006
4-component ITS architecture	Intelligent Tutoring Systems	Anderson et al. [1]	<i>J. Learning Sciences</i> 1995
ITS vs. human tutor efficacy	Intelligent Tutoring Systems	VanLehn [53]	<i>Educ. Psychologist</i> 2011
Mastery state machine	Bayesian Knowledge Tracing	Corbett & Anderson [12]	<i>UMUAI</i> 1994
Deep knowledge tracking	Bayesian Knowledge Tracing	Piech et al. [47]	<i>NeurIPS</i> 2015
Learner context pipeline	Adaptive Hypermedia	Brusilovsky [7]	<i>UMUAI</i> 1996
Profile-driven adaptation	Adaptive Hypermedia	Brusilovsky & Millán [9]	<i>The Adaptive Web</i> 2007
<i>preferredLearningStyle</i> field	Learning Styles (VARK)	Fleming & Mills [23]	<i>To Improve the Academy</i> 1992
Active/hands-on vs. reading	Learning Styles	Felder & Silverman [21]	<i>Engineering Education</i> 1988
<i>computeAdaptiveDifficulty()</i>	Adaptive Assessment / IRT	Lord [40]	<i>IRT Problems</i> 1980
<i>detectWeakAreas()</i>	Adaptive Assessment / CAT	Wainer et al. [57]	<i>CAT: A Primer</i> 2000
XP, levels, progress bars	Gamification	Deterding et al. [15]	<i>MindTrek</i> 2011
Badges in education	Gamification	Denny [14]	<i>CHI</i> 2013
Gamification effectiveness	Gamification	Hamari et al. [27]	<i>HICSS</i> 2014
AI-generated explanations/quizzes	LLM as Teacher	Kasneci et al. [32]	<i>Learning & Ind. Diff.</i> 2023
Personalised AI feedback	LLM as Teacher	Kochmar et al. [33]	<i>EDM</i> 2022
Streaming AI Instructor (SSE)	LLM as Teacher	Macina et al. [41]	<i>arXiv</i> 2023

Feature	Academic Field	Key Paper	Publication
Teacher-student model	Knowledge Distillation	Hinton et al. [29]	<i>arXiv</i> 2015
Verifiable completion certificate	Gamification / Credentialing	Deterding et al. [15]	<i>MindTrek</i> 2011
<i>LearnerVelocity</i> model	Learning Analytics	Siemens & Long [50]	<i>EDUCAUSE Review</i> 2011
Progress dashboards	Learning Analytics	Verbert et al. [54]	<i>American Behav. Sci.</i> 2012
<i>LearningNode</i> ontology schema	Ontology Engineering	Gruber [24]	<i>Knowledge Acquisition</i> 1993
Ontology to quiz generation	Ontology-based Curriculum	Holohan et al. [30]	<i>ICALT</i> 2005
Anti-hallucination grounding	Ontology-based Curriculum	Mizoguchi & Bourdeau [44]	<i>IJAIED</i> 2000
Bloom’s levels in quiz design	Educational Psychology	Bloom [3]	<i>Taxonomy</i> 1956
Modality-swapping logic	Cognitive Load Theory	Sweller [51]	<i>Cognitive Science</i> 1988
Multi-path branching	Self-Determination Theory	Ryan & Deci [48]	<i>Am. Psychologist</i> 2000
Adaptive difficulty calibration	ZPD / Constructivism	Vygotsky [56]	<i>Mind in Society</i> 1978
Resource modality matching	Multimedia Learning	Mayer [43]	<i>Cambridge Press</i> 2009

2.2.2 Critical Gaps in Existing Solutions

Despite the wealth of research in each stream, no existing system integrates all of these principles into a coherent, open-source platform suitable for resource-constrained environments. A comprehensive synthesis of related work reveals six critical gaps:

1. **No integrated ontology-LLM pipeline:** Existing ITS rely on handcrafted knowledge representations that do not scale; pure LLM systems hallucinate. No system combines verified ontology generation with runtime LLM tutoring in a single coherent architecture.
2. **No mastery-based resource adaptation:** While MBL principles are well established in the literature, no open-source platform implements score-tiered resource modality swapping driven by a Gatekeeper pattern that adapts both the content and the delivery modality based on specific failure modes.
3. **No decay-aware roadmap:** Spaced repetition tools (e.g., Anki) operate outside any curriculum structure

and require learner self-discipline. No system integrates forgetting-curve dynamics directly into a visual mastery roadmap with automatic decay-triggered reminders.

4. **No learner-controlled branching in ITS:** Multi-path branching exists in adaptive hypermedia research but has not been operationalised as a learner-facing feature in an ITS that allows choice of specialisation path while maintaining prerequisite integrity.
5. **No ontology-grounded resource pipeline:** Existing platforms either hand-curate (expensive and limited coverage) or accept all content (noisy and unreliable). No system combines offline expert curation anchored to a verified ontology, runtime AI-generated explanations constrained to node learning outcomes, and SERP API-based supplementary discovery—the three-layer model implemented in AI-Driven Personalized Learning Roadmap Generator And Tutor.
6. **No accessible ITS for developing economies:** Proven ITS (Cognitive Tutors, ALEKS) are proprietary and prohibitively expensive. No open-source alternative addresses the specific constraints of learners in regions like Ethiopia, where mentorship infrastructure is limited, internet connectivity is variable, and premium educational tools are inaccessible.

Table 2.2 positions AI-Driven Personalized Learning Roadmap Generator And Tutor relative to the most relevant existing solutions across these gaps.

2.3 Lessons Learned from Literatures

From this comprehensive literature synthesis, three overarching lessons emerge, each directly informing the design decisions presented in subsequent chapters.

Lesson 1: Ontology-Guided LLM Generation Solves Hallucination While Maintaining Scalability

The literature on Knowledge Space Theory [16], ontology engineering [24], and RAG [38] converge on a common insight: generative AI is most reliable when its output space is bounded by a verified knowledge structure. Traditional ITS faced a scaling problem—the knowledge engineering bottleneck. LLMs solved the scaling problem but introduced hallucination. AI-Driven Personalized Learning Roadmap Generator And Tutor’s Skeleton & Flesh strategy operationalises a variant of RAG specifically for curriculum design: the skeleton (verified ontology) constrains what the flesh (runtime LLM) can generate, preventing hallucination while maintaining the scalability and flexibility of LLM-based tutoring. This bridges decades of ITS research with modern AI capabilities.

Lesson 2: Assessment-Driven Progression with Resource Adaptation Addresses the “Knowing How to Teach” Gap

Bloom’s mastery principle [4] established that progress should be contingent on demonstrated competence. However, Bloom did not specify how to teach when a learner fails. The convergence of BKT [12], IRT [40],

and MBL [4, 26] teaches that effective learning requires not only accurate assessment but also pedagogically appropriate responses to failure. AI-Driven Personalized Learning Roadmap Generator And Tutor implements this lesson through the Gatekeeper Pattern: five-tier scoring that triggers distinct adaptation events (resource modality swap, prerequisite review, instructor escalation) at each failure depth. The IRT-inspired difficulty calibration and weak-area detection ensure that assessments remain informative at every proficiency level. This transforms assessment from a summative evaluation tool (pass/fail) into a diagnostic, adaptive instrument that informs instructional design in real time.

Lesson 3: Decay-Aware Adaptation Transforms Spaced Repetition from Learner-Managed to System-Guided

Over a century of memory research [19, 10, 31] demonstrates that forgetting is predictable and that strategic review at calibrated intervals can intercept the forgetting curve and dramatically extend retention. Yet existing tools place the burden of scheduling and discipline on the learner. AI-Driven Personalized Learning Roadmap Generator And Tutor transforms this relationship by embedding decay dynamics directly into the roadmap visualisation: nodes transition through green → yellow → red states based on elapsed time, with automatic micro-quiz generation and notification triggering at each transition. Learning analytics [50, 54] further close the loop by surfacing velocity-adjusted completion forecasts and decay patterns to the learner. This addresses a real failure mode of self-directed learning: skill decay and knowledge gaps that accumulate invisibly until they cause failures in advanced topics.

Synthesis: The Design Principles of AI-Driven Personalized Learning Roadmap Generator And Tutor

These three lessons, informed by the twelve academic foundations mapped in Table 2.1, translate into the following design principles that guide Chapters 3 through 7:

- **Separation of content generation from content verification:** The Teacher Model generates; domain experts verify; the Tutor Model operates within verified bounds. This prevents hallucination while maintaining scalability.
- **Assessment as a pedagogical instrument, not merely a measurement tool:** Every quiz attempt updates the mastery state and triggers pedagogically appropriate adaptation. Assessment drives instruction rather than merely evaluating it.
- **Time as a first-class dimension of mastery:** Mastery is not a static attribute but a decaying function of time. The system actively manages this decay on behalf of the learner through automatic reminders and micro-quizzes.
- **Learner agency within principled constraints:** Multi-path branching offers meaningful choice while preserving prerequisite integrity through KST-based node unlock logic. Learners direct their learning trajectory within a pedagogically sound structure.

- **Open, verifiable, and sustainable:** Content is generated from a verified, openly authored ontology and supplemented by SERP API discovery. The platform is designed for zero-cost access in resource-constrained environments, with a locally runnable AI stack (Phi-4, Ollama) that operates even without cloud connectivity, making high-quality technical education accessible to Ethiopian learners.

AI-Driven Personalized Learning Roadmap Generator And Tutor’s novelty lies not in inventing fundamentally new components but in thoughtfully integrating proven principles—ontology-guided RAG, assessment-driven progression grounded in mastery learning, spaced repetition informed by Ebbinghaus and modern cognitive science, multi-path learning respecting learner agency, streaming conversational AI tutoring, verifiable completion certificates, and a layered resource architecture combining offline expert curation with ontology-constrained AI generation and SERP API discovery—into a cohesive, context-aware system optimised for Ethiopian learners and resource-constrained environments. Specifically, AI-Driven Personalized Learning Roadmap Generator And Tutor addresses the distinct challenge of “tutorial hell” and resource fragmentation endemic to self-directed learning in regions with limited mentorship infrastructure. By combining structured, verified curricula with adaptive, intelligent tutoring, the system approximates the benefits of one-on-one mentorship (Bloom’s two-sigma finding) at scale and low cost.

These design principles directly motivate the requirements specification in Chapter 3, the system architecture in Chapter 4, the implementation details in Chapter 5, and the experimental evaluation in Chapter 6 that follow.

Table 2.2. *Position of AI-Driven Personalized Learning Roadmap Generator And Tutor within the landscape*

Solution	Strengths	Gaps	AI-Driven Personalized Learning Roadmap Generator And Tutor Innovation
Khan Academy	High-quality, free content; evidence-based pedagogy; mastery progression for some topics	Passive sequencing; no multi-path; no decay awareness; limited domain coverage	Gatekeeper Pattern; Mastery Decay; Multi-Path Branching; resource adaptation on failure
roadmap.sh	Tech-focused, community-curated, visual; free; reflects real developer work-flows	Purely static sequencing; no adaptive response; no quizzes or assessment gates	Ontology-guided sequencing; Gatekeeper enforcing mastery; adaptive resource curation; branching paths
Coursera/Udemy	Diverse content; large scale; structured courses with deadlines	High hallucination in recommendations; no verifiable knowledge graph; high dropout (>70%); no decay-aware retention; expensive; not adapted to resource-constrained contexts	Skeleton & Flesh hybrid (verified ontology + LLM); Gatekeeper reducing dropout; Mastery Decay; free/low-cost positioning for Ethiopian learners
Anki	Proven spaced repetition; flexible; effective at scale	Standalone, not integrated into curriculum; no semantic sequencing; no visual roadmap; requires learner self-discipline; no adaptive modality	Decay-aware roadmap UI; spaced repetition integrated into progression; Gatekeeper quizzes enforced (not optional); visualises forgetting dynamically
Pure LLM Tutoring (Chat-GPT, Claude)	Flexible natural dialogue; fast; scales instantly to any domain; no initial content creation overhead	Hallucination (confident but false information); no curriculum coherence (no prerequisites enforced); no persistence (each conversation is isolated); no learning path validation; user must discover what to learn	Skeleton & Flesh hybrid: LLM generates only within verified ontology bounds; prevents hallucination while retaining flexibility; persistent learner state and roadmap visibility; expert-verified prerequisite structure
Proprietary ITS (Cognitive Tutors, ALEKS)	Proven learning gains ($\approx 1\sigma$); sophisticated student modelling; pedagogically sound	High cost; limited domain coverage; closed-source; not available in resource-constrained regions	Open-source; hybrid architecture reduces development cost; adaptable to Ethiopian context

Chapter 3

Problem Analysis and Modeling

This chapter bridges the problem landscape articulated in Chapter 1 and the research foundations established in Chapter 2 with concrete system specifications and models. It presents a detailed analysis of requirements, functional and non-functional specifications, and comprehensive system models—use cases, UML class diagrams, sequence diagrams, activity diagrams, and state machines—that collectively define what AI-Driven Personalized Learning Roadmap Generator And Tutor must achieve and how its components interact. The modelling approach follows standard software engineering practices, ensuring that requirements are comprehensive, testable, and verifiable. Models serve dual purposes: (1) communicating system intent to stakeholders and team members, and (2) providing a blueprint for implementation in Chapter 5.

3.1 Existing System and Its Problems

The current landscape of self-directed learning systems, particularly for technical skill acquisition in resource-constrained environments like Ethiopia, is dominated by a mix of free online resources, static roadmaps, massive open online courses (MOOCs), spaced repetition tools, and emerging large language model (LLM)-based tutors. Platforms such as Khan Academy, roadmap.sh, Coursera/Udemy, Anki, and pure LLM tutoring (e.g., ChatGPT, Claude) represent the primary existing systems. While these have democratized access to knowledge, they exhibit systemic limitations rooted in design philosophies that prioritise scalability and content volume over personalisation, retention, and adaptive feedback. This section provides a detailed examination of these systems, identifying their limitations, underlying causes, affected stakeholders, and impacts.

Overview of Existing Systems

Existing systems can be broadly categorised into three types:

1. **Curated Content Platforms:** Platforms such as Khan Academy, Coursera, and Udemy provide structured video content and assessments, but follow a fixed, one-size-fits-all curriculum that does not adapt to individual learner progress or prior knowledge. These systems leverage free resources like YouTube tutorials,

blog posts (e.g., GeeksForGeeks), and documentation (e.g., MDN Web Docs), but their aggregation and delivery mechanisms often exacerbate fragmentation rather than resolve it.

2. **Static Roadmap Tools:** Resources like `roadmap.sh` offer visual dependency graphs of skills but lack interactivity, progress tracking, or assessment capabilities. They present a fixed, linear sequence without performance-driven progression gates.
3. **Retention and AI Tools:** Tools such as Anki (spaced repetition flashcards) and conversational AI tutors (ChatGPT, Claude) address isolated aspects of learning but are not integrated into a coherent, mastery-based progression system.

Limitations and Underlying Causes

The limitations stem from architectural, pedagogical, and economic constraints, leading to inefficiencies in learner progression and retention.

1. **Fragmentation and Lack of Coherent Structure (Tutorial Hell Phenomenon):** Most systems rely on passive, non-sequenced content delivery. YouTube recommendations and `roadmap.sh` provide disjointed resources without enforcing prerequisites or logical progression, caused by algorithmic biases favouring engagement metrics over pedagogical soundness. Cognitive load theory [51] explains how unstructured information overwhelms working memory, leading to reduced retention. **92% of surveyed learners** reported difficulty finding coherent paths, highlighting how ad-hoc navigation creates knowledge gaps (e.g., jumping from HTML to React without JavaScript fundamentals).
2. **Absence of Adaptive Feedback and Personalization:** Platforms like Khan Academy and Coursera use fixed sequences without runtime adaptation to learner performance or preferences. Pure LLM tutors hallucinate responses due to unverified knowledge bases, while Anki requires manual card creation. Self-determination theory [13] shows that lack of agency reduces motivation. **85% of learners** reported insufficient feedback, leading to false mastery (e.g., marking topics “complete” without competence verification). This results in high dropout rates (**70%+ on Coursera**).
3. **No Mastery-Based Progression Gates and Retention Mechanisms:** Systems like `roadmap.sh` lack quizzes or gates, relying on self-assessment, while Anki isolates repetition from curriculum context. Ebbinghaus’s Forgetting Curve [19] is ignored, causing skill decay over time—**78% of learners forget concepts after 14 days** without review. Mastery learning principles [4] advocate competence-based gates, yet existing tools lead to downstream failures. In resource-constrained contexts, this amplifies economic barriers, as learners cannot afford premium alternatives.
4. **Quality Variability and Resource Overload:** Algorithmic curation (e.g., LinkedIn Learning) prioritises popularity over accuracy, leading to unreliable content. Manual curation (Khan Academy) is resource-intensive, limiting coverage to established domains. This creates cognitive overload as learners sift through mixed-quality sources. Focus groups revealed biases toward entertainment-oriented content, misaligning with deep learning needs.

Stakeholders and Impacts

The stakeholders affected by these limitations are:

- **Primary Stakeholders—Learners (Ethiopian Students, Junior Developers, Career-Changers):** Frustration from “tutorial hell” leads to demotivation, skill gaps, and delayed career progression. In Ethiopia, where premium mentorship is prohibitive, this perpetuates economic inequality—learners waste time (85% report inefficient paths), experience high dropout, and face retention decay, hindering employability in tech sectors.
- **Secondary Stakeholders—Educators and Institutions (e.g., AASTU Faculty):** Overburdened with remedial teaching due to self-directed gaps; limited tools for integrating adaptive systems into curricula, reducing institutional efficacy.
- **Tertiary Stakeholders—Instructors and Domain Experts:** Lack tools to monitor learner progress at scale, identify struggling learners early, or contribute domain knowledge to structured curricula without significant manual effort. Existing platforms provide no instructor-facing analytics or intervention mechanisms.

Overall Impact: These limitations create a vicious cycle: fragmentation increases cognitive load, lack of feedback fosters false confidence, and decay causes repeated failures, impacting motivation and long-term skill acquisition. In developing economies, this widens the digital divide, as learners cannot compete with those in resource-rich environments.

Utilization of Analytical Tools and Techniques

To understand the complexity of the problem, the following analytical tools and techniques were employed:

- **Stakeholder Engagement:** Semi-structured interviews with domain experts (n=5) and learner surveys (n=30) quantified pain points, including the 78% skill decay rate, 92% path fragmentation, and 85% insufficient feedback.
- **Competitive Gap Analysis:** Synthesised via a landscape comparison table mapping strengths and gaps of existing solutions against AI-Driven Personalized Learning Roadmap Generator And Tutor innovations.
- **Cognitive and Pedagogical Frameworks:** Applied theories (Forgetting Curve, Mastery Learning, Cognitive Load Theory, Self-Determination Theory) to model impacts and validate proposed solutions.
- **Prototyping and Simulation:** Early wireframes and decay algorithm prototypes validated assumptions about learner progression and retention mechanics.

3.2 Requirements Elicitation and Specification

Elicitation Methodology

Requirements were gathered through multi-stakeholder engagement:

1. **Domain Expert Interviews:** Semi-structured interviews with five software engineering educators, industry practitioners, and experienced self-taught engineers were conducted to understand learning challenges, effective pedagogical strategies, and domain-specific sequencing principles. Key insights included: learners struggle with unstructured, non-sequential content; assessment-driven progression increases motivation; resource modality (documentation vs. tutorial) significantly affects learning outcomes.
2. **Learner Surveys and Feedback Sessions:** An online survey (30 respondents: Ethiopian students, junior developers, career-changers) identified pain points: tutorial fragmentation (92% reported difficulty finding coherent learning paths), lack of feedback (85% noted insufficient guidance on mastery), and skill decay (78% forgot concepts after weeks of non-practice). Focus group discussions (two sessions, 15 participants) validated proposed solutions and identified usability priorities.
3. **Competitive Analysis:** Analysis of existing platforms (Khan Academy, roadmap.sh, Coursera, Anki, ChatGPT tutoring) identified gaps and best practices, informing feature prioritisation and differentiation.

Functional Requirements

Functional requirements specify what the system must do, organised by use case and subsystem:

FR1: Ontology Management

- FR1.1:** The system shall generate a learning ontology (DAG of learning nodes) for a given domain using the Teacher Model (offline LLM), producing a JSON structure with node identifiers, learning outcomes, prerequisites, and estimated effort.
- FR1.2:** The system shall support expert verification workflows, enabling domain experts to review generated ontologies, flag errors, and approve for production deployment.
- FR1.3:** The system shall persist and version ontologies, maintaining a history of changes and enabling rollback if necessary.

FR2: Learner Management and Enrollment

- FR2.1:** The system shall enable user registration via email/password, Google OAuth, and GitHub OAuth, with JWT-based session management and automatic token refresh.
- FR2.2:** The system shall provide an enrollment flow in which the learner selects a domain, chooses an initial branch path (`frontend`, `backend`, or `data_science`), and submits a learner profile capturing: weekly hours available (`weeklyHours`), domain familiarity level (`familiarityLevel`), learning goal (`learningGoal`), free-text self-description (`aboutSelf`), preferred learning style

(preferredLearningStyle), and prior skills (priorSkills). These fields shall be persisted in the Enrollment record and used to personalise AI-generated content.

FR2.3: The system shall track learner progress per enrolled domain, including node mastery state, quiz scores, attempt counts, mastery timestamps, and adaptation events.

FR2.4: The system shall compute and display learner statistics per enrollment: total nodes completed, mastery breakdown by state, average quiz score, estimated completion timeline, and learning velocity ratio.

FR3: Quiz Generation and Delivery

FR3.1: The system shall generate domain-specific quizzes using the Tutor Model (runtime LLM), producing 3–5 questions per node, grounded in the node’s learning outcomes.

FR3.2: The system shall generate multiple-choice questions (MCQ) with four answer options per question, one correct answer, and an explanation of the correct answer. Question prompts may include inline code snippets where appropriate for the domain.

FR3.3: The system shall evaluate quiz responses, compute scores, and provide immediate feedback on correctness.

FR3.4: The system shall generate micro-quizzes (2–3 questions) triggered by Mastery Decay transitions.

FR4: Gatekeeper Pattern Implementation

FR4.1: The system shall enforce five-tier progression logic: Strong Pass ($\geq 80\%$ → node mastered, 14-day decay timer, challenge project recommended), Marginal Pass (70–79% → node mastered, 7-day accelerated decay timer), Fail-Low (50–69% → resource_swap adaptation event, node stays locked), Fail-Fundamental (30–49% → prerequisite_review adaptation event), Fail-Severe ($< 30\%$ → instructor_escalation event).

FR4.2: The system shall validate prerequisites before unlocking nodes, preventing progression if prerequisites are not mastered.

FR4.3: Upon Pass ($\geq 80\%$), the system shall recommend a challenge project relevant to the node’s outcomes.

FR4.4: Upon Fail ($< 70\%$), the system shall trigger resource adaptation or prerequisite review based on failure severity.

FR5: Content Generation and Resource Curation

FR5.1: The system shall generate node-specific explanations at runtime using the Tutor Model (three-tier LLM stack), with the prompt constrained to the node’s verified title, learning outcomes, difficulty level, and prerequisite chain to prevent hallucination.

FR5.2: The system shall deliver AI-generated explanations token-by-token to the client via Server-Sent Events (SSE), with the first token visible within 1 second of the learner’s request.

FR5.3: The system shall support offline expert curation: domain expert instructors shall be able to attach, update, and remove curated reference links and reading materials directly to each ontology node via the admin interface.

FR5.4: The system shall integrate the SERP API to surface supplementary resources scoped to a node's verified learning outcomes, available to instructors for promotion into the curated layer.

FR5.5: Upon a `resource_swap` adaptation event (Gatekeeper fail at 50–69%), the system shall re-invoke the Tutor Model with a tutorial-oriented style instruction, delivering regenerated content in a more scaffolded, step-by-step format.

FR6: Mastery Decay and Spaced Repetition

FR6.1: The system shall track the time since a node was last reviewed and compute decay state. Strong-pass nodes transition Green → Yellow after 14 days and Yellow → Red after 30 days. Marginal-pass nodes use an accelerated 7-day Green → Yellow threshold, reflecting lower initial encoding strength.

FR6.2: Upon transition to Yellow or Red, the system shall automatically generate and present a micro-quiz to the learner.

FR6.3: The system shall visualise decay states in the learner's roadmap (colour-coded nodes).

FR6.4: The system shall send in-app notifications alerting learners when one or more nodes transition to Yellow or Red decay state, with a direct link to the affected node.

FR7: Multi-Path Branching

FR7.1: The ontology shall tag each learning node with a `branchPath` attribute (`frontend`, `backend`, `data_science`, or `shared`). Shared nodes are displayed on all paths; branch-specific nodes are displayed only when that path is active.

FR7.2: At enrollment time, the learner shall select an initial branch path. This selection shall be persisted on the `Enrollment` record and used to determine the initial roadmap view (covered under FR2.2).

FR7.3: After enrollment, the learner shall be able to switch their active branch path at any time via the roadmap sidebar branch selector. The roadmap shall re-render immediately to show the selected branch's nodes.

FR7.4: The system shall ensure that prerequisite integrity is maintained across branch switches: nodes that require prerequisites from a different branch shall remain locked until those prerequisites are mastered.

FR8: Learner Web Interface

FR8.1: The web interface (React 18 + TypeScript) shall display the ontology as an interactive DAG visualisation (React Flow) with zoom, pan, node selection, mastery-state colour coding, and decay-state indicators.

FR8.2: The web interface shall include: quiz delivery UI, streaming AI explanation panel, AI Instructor chatbot panel, progress sidebar with mastery breakdown and timeline estimate, learning insights analytics page, gamification achievements page, and certificate management page.

FR8.3: The mobile interface (Flutter) shall provide touch-optimized roadmap navigation, quiz delivery, and push notifications for decay reminders.

FR8.4: The web interface shall be fully responsive, supporting mobile, tablet, and desktop viewports without loss of functionality.

FR9: AI Instructor — Streaming Conversational Tutor

FR9.1: Each learning node shall host a persistent AI Instructor chat panel. Learners may ask any question about the current node and receive an AI-generated answer grounded in the node’s ontology context.

FR9.2: AI Instructor responses shall be streamed token-by-token to the client via SSE, with the first token visible within 1 second.

FR9.3: Every AI Instructor call shall include the full node context (title, learning outcomes, difficulty, prerequisite chain) in the system prompt; the model may not introduce topics outside that context.

FR10: Completion Certificates

FR10.1: The system shall issue a certificate when a learner’s roadmap reaches 100% mastery. The certificate shall be idempotent: claiming it a second time returns the existing certificate.

FR10.2: Each certificate shall carry a unique Crockford base-32 public identifier (format ATL-XXXX-XXXX) stored in the database with learner name, course name, hours invested, average quiz score, and completion date.

FR10.3: The learner shall be able to download the certificate as a high-resolution PDF (A4 landscape) or PNG file, generated client-side from the rendered certificate canvas.

FR10.4: A publicly accessible, no-authentication verification page at `/verify/:publicId` shall re-render the certificate and display a “Verified Authentic” badge, enabling employer or institutional verification via a shareable link.

FR11: Gamification and Achievements

FR11.1: The system shall award experience points (XP) for quiz passes, scaled by score tier (strong pass earns more XP than marginal pass) and attempt efficiency (bonus XP for first-attempt mastery).

FR11.2: The system shall compute a learner level from cumulative XP and display a progress bar toward the next level threshold.

FR11.3: The system shall track daily activity streaks and award streak badges at milestone thresholds (e.g., 3-day, 7-day, 30-day).

FR11.4: The system shall define and automatically award named achievement badges for significant milestones (e.g., first mastery, quiz ace, speed learner, completionist, decay defender).

FR11.5: The system shall define weekly learning goals (a target number of nodes to master per week) and display a progress indicator toward the current week’s goal, resetting every Monday.

FR12: Learning Analytics and Insights

FR12.1: The system shall record velocity data per mastered node (`estimatedHours`, `actualHours`, `startedAt`, `completedAt`) and compute a `velocityRatio` (actual / estimated).

FR12.2: The system shall compute a velocity-adjusted completion forecast from remaining unmastered node hours, the learner's weekly hours input, and their average `velocityRatio`.

FR12.3: The system shall surface a per-enrollment Insights dashboard displaying: activity heatmap (daily study sessions), velocity trend over time, mastery breakdown by state, and the estimated completion timeline with velocity multiplier.

FR12.4: The system shall provide a global Insights page aggregating analytics across all of the learner's active enrollments, giving an overview of total study activity and progress across domains.

FR13: Instructor and Administrator Portals

FR13.1: A domain expert instructor portal shall display: list of enrolled learners with progress summaries, per-learner node-level progress drill-down, domain-level analytics (completion rates, average scores, common failure nodes), and a flagged events queue (`instructor_escalation` adaptation events). Instructors shall be able to resolve flagged events, recording acknowledgement and optional notes.

FR13.2: An administrator console shall provide: user management (list, role assignment, deactivation), domain management (create, publish, archive domains), ontology builder (graphical node editor for learning nodes, prerequisites, and learning outcomes), and system statistics (total users, enrollments, quiz attempts, adaptation event counts).

Non-Functional Requirements

Non-functional requirements specify quality attributes and constraints:

NFR1: Performance

NFR1.1: Page load time for roadmap visualisation shall not exceed 2 seconds on typical internet connections (> 5 Mbps).

NFR1.2: API response time for quiz delivery and scoring shall not exceed 500 milliseconds (p95).

NFR1.3: LLM-based quiz generation shall complete within 3 seconds of user request, leveraging Redis caching to serve repeated requests instantly.

NFR1.4: For SSE-streamed AI content (explanations and AI Instructor responses), the first token shall be visible to the learner within 1 second of the request, regardless of total generation time. The Nginx reverse proxy shall be configured with `proxy_buffering off` on SSE routes to prevent token accumulation.

NFR2: Scalability

NFR2.1: The system shall support 1,000 concurrent users without degradation in performance (load testing baseline).

NFR2.2: Database and cache layers shall scale horizontally (read replicas, sharding) to support 10,000+ daily active users.

NFR2.3: The system shall support efficient multi-tenancy (future feature), enabling independent deployments per institution or community.

NFR3: Availability and Reliability

NFR3.1: System uptime shall be $\geq 99.5\%$ (measured monthly).

NFR3.2: Critical data (learner progress, quiz responses) shall be backed up daily, with recovery capability within 4 hours.

NFR3.3: The system shall implement graceful degradation through a three-tier LLM provider chain with per-provider circuit breakers: if the primary provider (Phi-4) is unavailable, the system automatically fails over to Ollama (local Qwen2.5-3B); if Ollama is unavailable, it falls over to Gemini 2.5 Flash. Redis-cached responses serve repeat requests without any AI provider call.

NFR4: Security

NFR4.1: User authentication shall use short-lived JWT access tokens (15-minute expiry) paired with long-lived refresh tokens. Access tokens are stored in client-side application state (Zustand persisted store); refresh tokens are stored securely in the database and rotated on each use.

NFR4.2: Sensitive data (user profiles, quiz responses) shall be encrypted in transit (HTTPS) and at rest (database encryption).

NFR4.3: The system shall implement role-based access control (RBAC): learner, instructor, admin, system administrator roles with distinct permissions.

NFR4.4: The system shall sanitize user inputs and API responses to prevent SQL injection and XSS attacks.

NFR4.5: Learner data shall be treated as confidential; no personal data shall be shared with third-party services without explicit consent.

NFR5: Usability

NFR5.1: Learners shall complete onboarding (account creation, path selection) in < 5 minutes.

NFR5.2: Quiz interface shall be intuitive, with clear instructions and immediate feedback.

NFR5.3: Roadmap visualisation shall be interpretable by non-technical learners, with intuitive legend and help tooltips.

NFR5.4: System error messages shall be clear and actionable, guiding learners toward resolution.

NFR6: Maintainability and Extensibility

NFR6.1: Code shall follow established standards (JavaScript/TypeScript style guides, REST API conventions) and include comprehensive documentation.

NFR6.2: The system architecture shall enable addition of new domains without redesigning core components.

NFR6.3: API endpoints shall be versioned (v1, v2) to support backward compatibility during upgrades.

NFR7: Localization and Cultural Adaptation

NFR7.1: The system shall be deployed in English as the primary language in the initial release. The frontend codebase shall be structured with internationalisation (i18n) patterns to support future localisation into Amharic, Oromo, and other Ethiopian languages without architectural changes.

NFR7.2: The system shall be accessible to learners without requiring payment, premium subscriptions, or proprietary software, reflecting the resource-constrained context of Ethiopian learners.

NFR7.3: The system shall work reliably on low-bandwidth networks (< 2 Mbps): roadmap and quiz pages shall render correctly without all assets loading; SSE streams shall degrade gracefully if the connection is interrupted.

3.3 System Modeling

3.3.1 Use Case Model

Use cases capture interactions between actors (users) and the system. Table 3.1 presents the primary use cases identified for AI-Driven Personalized Learning Roadmap Generator And Tutor, covering the interactions of learners, domain experts, and administrators with the system.

3.3.2 Functional Requirements Traceability

Table 3.2 maps each use case to the functional requirements it realises, ensuring comprehensive coverage and providing a basis for validation.

3.3.3 UML Class Diagram

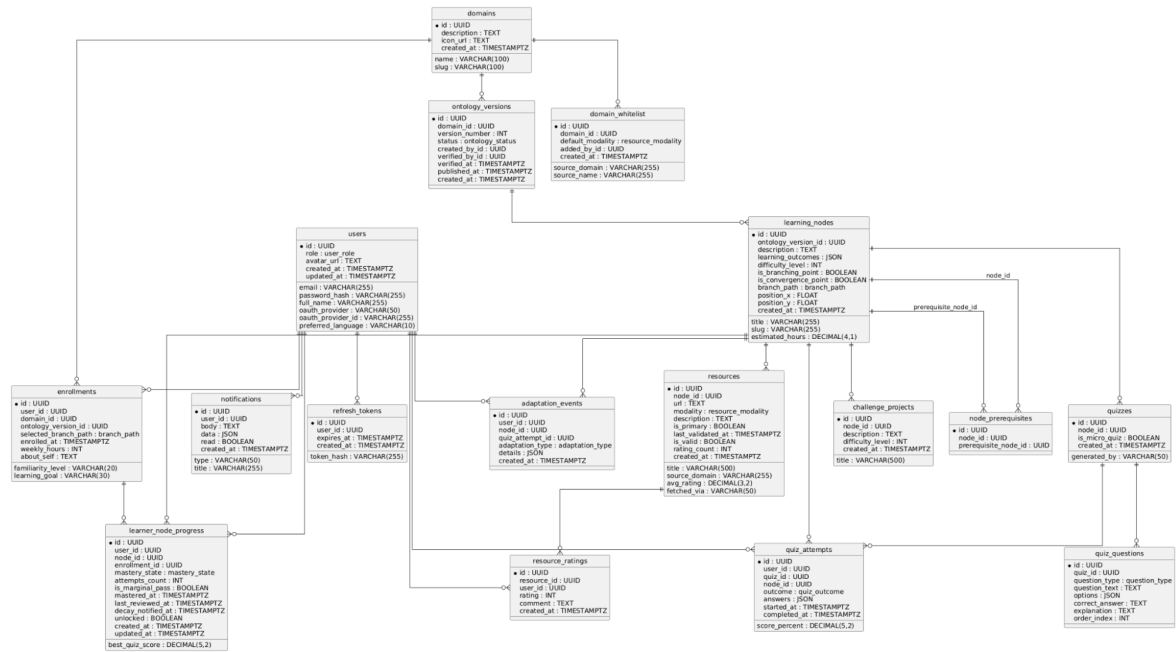
Figure 3.1 presents the core domain model of AI-Driven Personalized Learning Roadmap Generator And Tutor, illustrating the principal entities and their relationships.

Table 3.1. *Primary Use Cases*

Use Case	Primary Actor	Preconditions	Main Flow	Postconditions
UC0: Enrol in Domain	Learner	Logged in, domain selected from catalogue	1. Display enrolment form 2. Learner selects branch path and submits learner profile (weeklyHours, familiarityLevel, learningGoal, aboutSelf, preferredLearningStyle, priorSkills) 3. System creates Enrollment record and initialises progress rows	Learner enrolled; roadmap populated
UC1: Browse Roadmap	Learner	Enrolled, roadmap loaded	1. Display DAG visualisation 2. Learner explores nodes 3. System shows details, learning outcomes, mastery state, prerequisites	Learner understands path and requirements
UC2: Attempt Quiz	Learner	Node unlocked, learner selected Take Quiz	1. Generate quiz (4 MCQ questions) 2. Learner answers 3. Evaluate responses, compute score 4. Gatekeeper records outcome and adaptation event	Score recorded; mastery state updated
UC3: Adaptive Content Regeneration	Learner	Quiz failed 50–69% (resource_swap event fired)	1. Tutor Model regenerates explanation in tutorial-oriented style 2. Updated content displayed with step-by-step scaffolding 3. Learner reviews adapted content 4. Learner retries quiz	Learner receives modality-adapted explanation
UC4: Spaced Repetition Reminder	Learner	Node in Yellow/Red decay state	1. Generate micro-quiz notification 2. Present micro-quiz (2–3 questions) 3. Learner attempts, system scores	Decay state updated; learner re-engaged
UC5: Select Learning Path	Learner	Reached branching point	1. Present choice question and descriptions 2. Learner selects path 3. Update roadmap with selected path nodes	Roadmap personalised
UC6: Verify Ontology	Domain Expert	Ontology generated by Teacher Model	1. Expert reviews nodes, prerequisites, outcomes 2. Marks correct or flags for revision 3. Aggregates feedback, expert confirms	Verified ontology deployed
UC7: Manage Ontology Content	Admin/Instructor	Domain published in system	1. Open ontology builder 2. Add/edit/remove learning nodes and prerequisites 3. Attach or update curated resource links 4. Publish updated ontology version	Updated ontology live; active enrollments unaffected
UC8: View Progress Dashboard	Learner	Logged in	1. Display statistics (nodes completed, avg score, time) 2. Show decay state visualisation (Green/Yellow/Red) 3. Filter by domain or time	Visibility into learning progress
UC9: Use AI Instructor	Learner	Node selected, learning session active	1. Learner types question in chat panel 2. System builds node-context prompt 3. AI streams answer token-by-token via SSE	Learner receives grounded, contextual answer
UC10: Claim Certificate	Learner	All roadmap nodes mastered	1. System shows “Claim certificate” button 2. Learner clicks claim 3. Backend issues certificate with unique public ID 4. Learner downloads PDF/PNG or copies share link	Certificate stored; shareable verify URL active
UC11: View Achievements	Learner	Logged in	1. Display earned badges and XP level 2. Show locked achievements with unlock criteria 3. Display streak count	Learner motivated by visible progress milestones
UC12: View Learning Insights	Learner	At least one node mastered	1. Display activity heatmap 2. Show velocity trend and completion forecast 3. Display mastery breakdown	Learner understands pace and estimated completion
UC13: Monitor Learner Progress	Instructor	Enrolled learners exist in instructor’s domain	1. View learner list with completion summaries 2. Drill into individual learner node progress 3. Review flagged	Instructor identifies struggling learners; data-driven inter-

Table 3.2. Use Case to Functional Requirement Mapping

Use Case	Functional Requirements
UC0: Enrol in Domain	FR2.1, FR2.2, FR7.1, FR7.2
UC1: Browse Roadmap	FR1.1, FR1.3, FR7.3, FR8.1
UC2: Attempt Quiz	FR3.1, FR3.2, FR3.3, FR4.1, FR4.2, FR4.3, FR4.4
UC3: Adaptive Content Regeneration	FR4.4, FR5.1, FR5.5
UC4: Receive Spaced Repetition Reminder	FR6.1, FR6.2, FR6.3, FR6.4
UC5: Select Learning Path	FR7.1, FR7.2, FR7.3, FR7.4
UC6: Verify Ontology	FR1.2, FR1.3
UC7: Manage Ontology Content	FR1.2, FR1.3, FR5.3, FR5.4
UC8: View Progress Dashboard	FR2.3, FR2.4, FR6.3, FR8.2
UC9: Use AI Instructor	FR9.1, FR9.2, FR9.3
UC10: Claim Certificate	FR10.1, FR10.2, FR10.3, FR10.4
UC11: View Achievements	FR11.1, FR11.2, FR11.3, FR11.4
UC12: View Learning Insights	FR12.1, FR12.2, FR12.3, FR12.4
UC13: Monitor Learner Progress	FR13.1
UC14: Administer System	FR1.2, FR13.2

**Figure 3.1.** UML Class Diagram

3.3.4 Dynamic Models of a System

The dynamic behaviour of AI-Driven Personalized Learning Roadmap Generator And Tutor is captured through sequence diagrams, activity diagrams, and a state machine diagram, each focusing on a critical interaction or lifecycle within the system.

Sequence Diagrams

Quiz Attempt and Gatekeeper Flow

Figure 3.2 models the interaction sequence when a learner attempts a quiz, including the gatekeeper decision that determines whether the learner may proceed to dependent nodes.

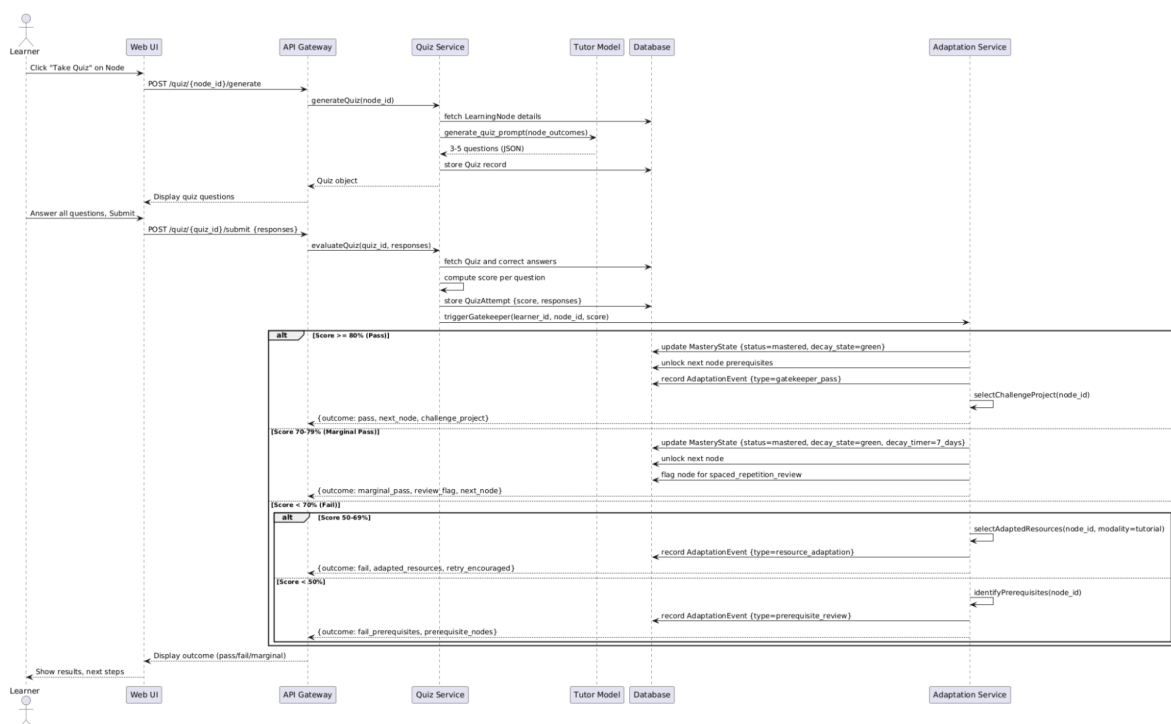


Figure 3.2. Quiz Attempt and Gatekeeper Flow

Mastery Decay and Spaced Repetition Trigger

Figure 3.3 models the periodic decay check and the triggered spaced repetition workflow that prompts learners to reinforce fading knowledge.

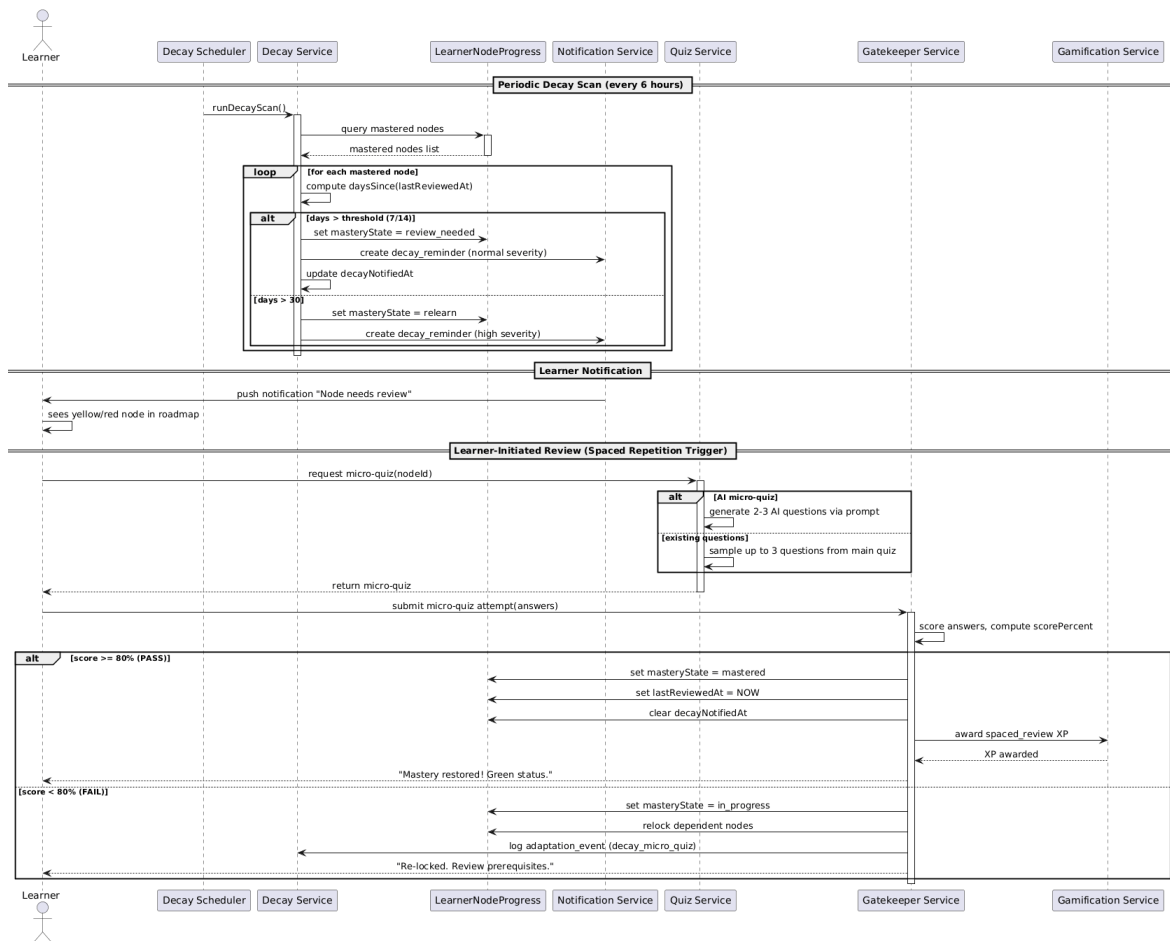


Figure 3.3. Mastery Decay and Spaced Repetition Trigger

Multi-Path Branching Selection

Figure 3.4 illustrates the sequence of interactions when a learner encounters a branching point and selects a specialisation path.

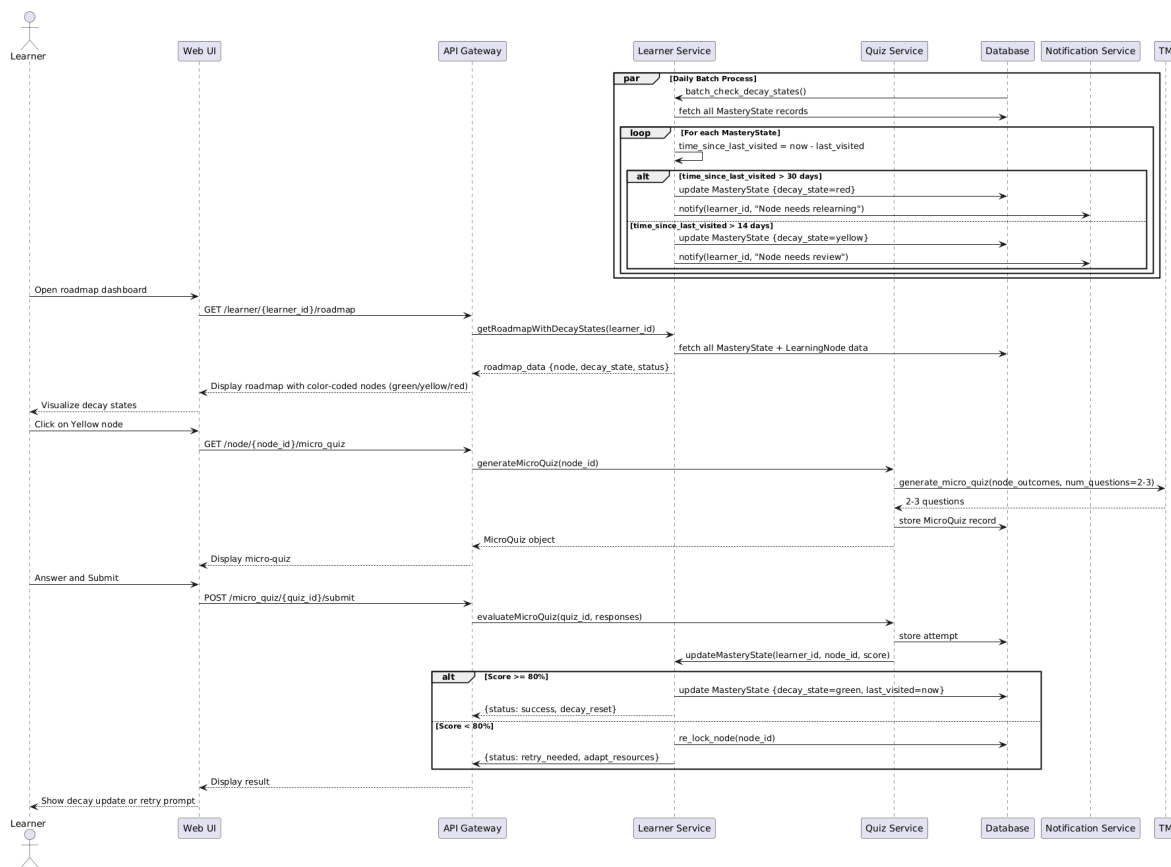


Figure 3.4. Multi-Path Branching Selection

Activity Diagrams

Learner Progression Activity Flow

Figure 3.5 models the overall activity flow of a learner navigating through the roadmap, from node exploration through assessment and progression.

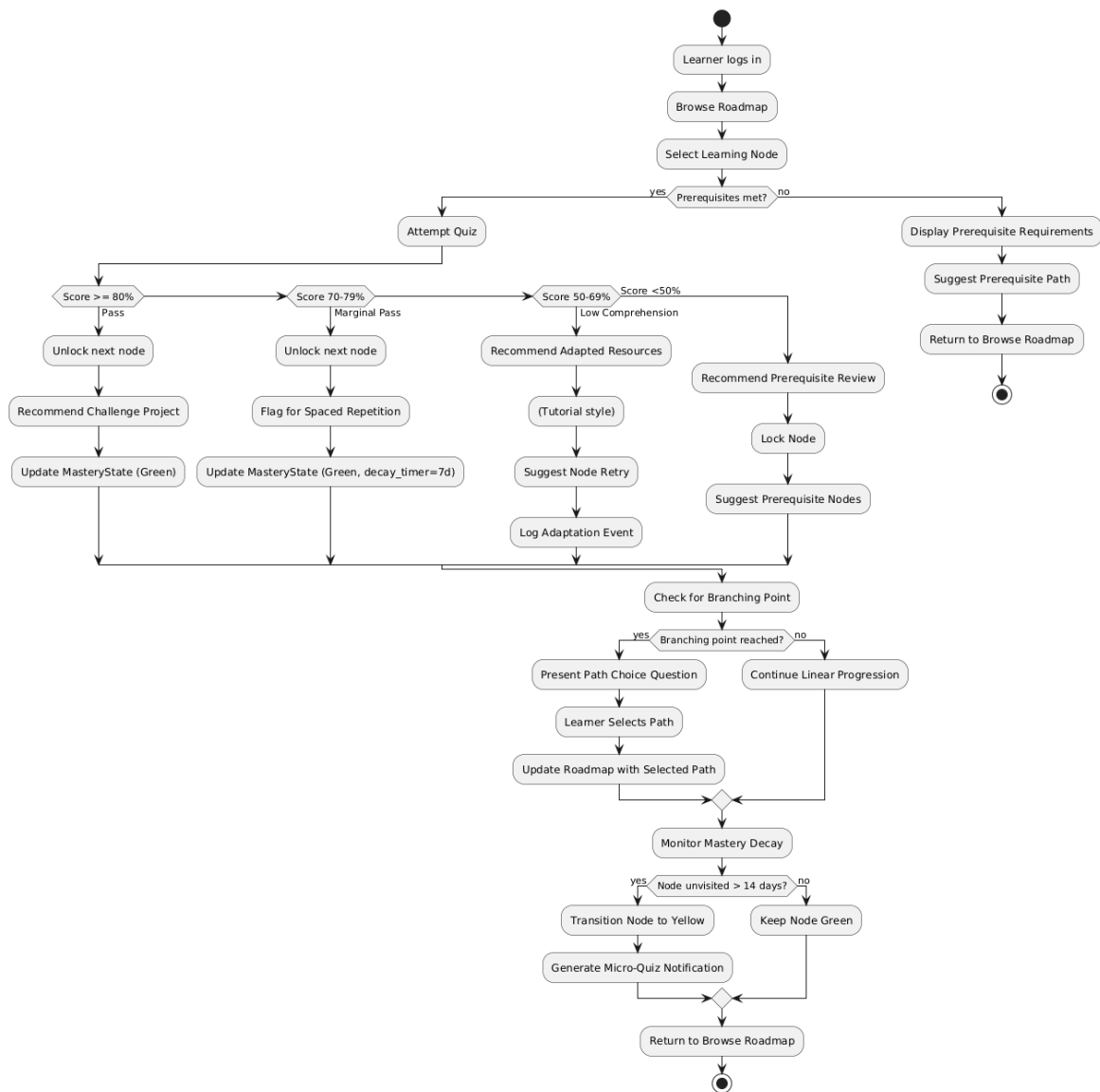


Figure 3.5. *Learner Progression Activity Flow*

Gatekeeper Decision Activity

Figure 3.6 details the decision logic within the gatekeeper component when evaluating quiz results and determining access to dependent nodes.

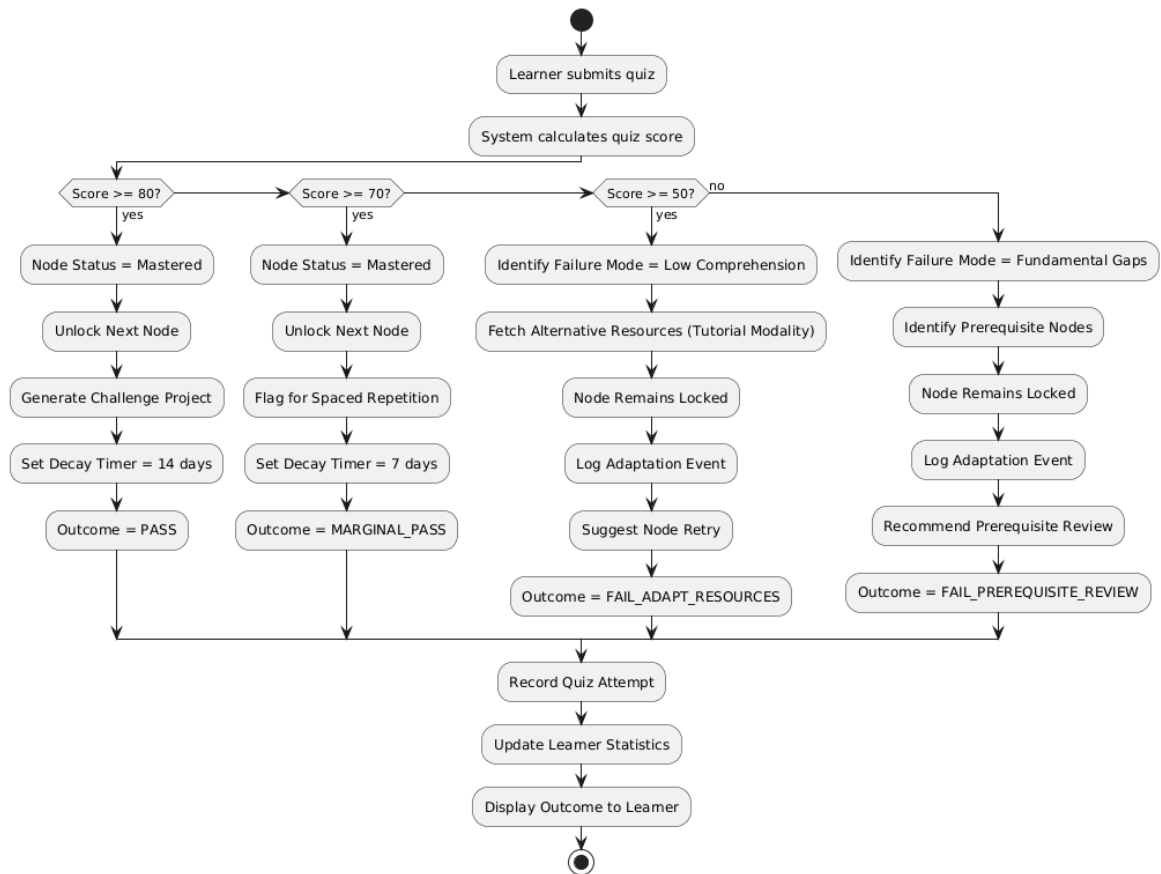


Figure 3.6. Gatekeeper Decision Activity

State Machine Diagram

Node Mastery State Machine

Figure 3.7 defines the mastery states (Green, Yellow, Red) that each learning node can occupy, including the transitions triggered by assessment events and the passage of time.

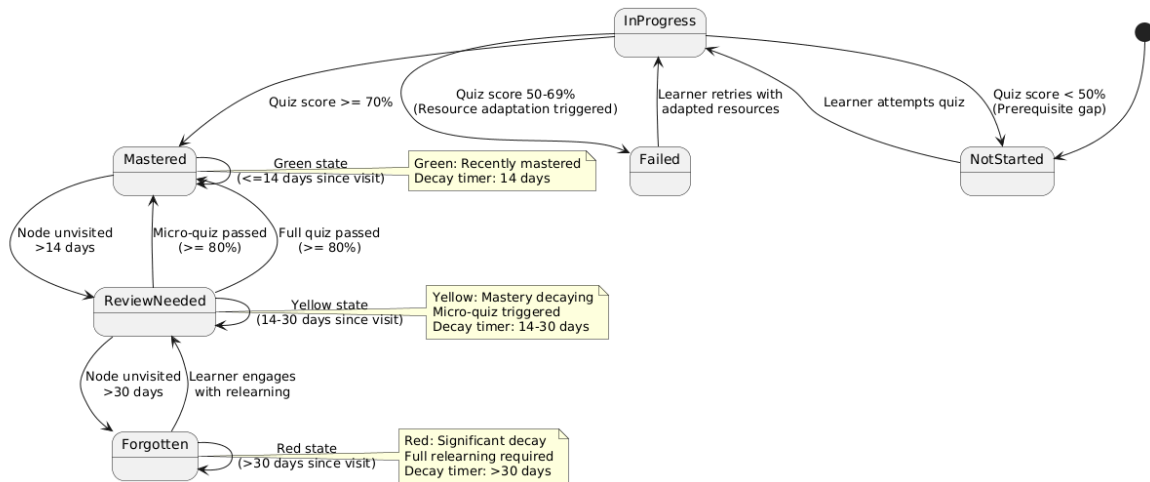


Figure 3.7. Node Mastery State Machine

3.4 Model Validation

The requirements and models presented in this chapter were validated through the following approaches:

1. **Requirements Review with Stakeholders:** The functional and non-functional requirements were reviewed with the five domain expert interviewees and a subset of survey respondents. Feedback led to several refinements: the gatekeeper threshold was calibrated to five tiers (FR4.1); the enrollment learner profile fields were expanded to include preferred learning style and prior skills (FR2.2); and the multi-path branching model was refined from a mid-roadmap encounter to an enrollment-time selection with runtime switching (FR7). Requirements added post-initial-design—AI Instructor (FR9), certificates (FR10), gamification (FR11), analytics (FR12), instructor and admin portals (FR13)—emerged from implementation discovery and were added to ensure the document reflects the delivered system.
2. **Traceability Analysis:** Each functional requirement is mapped to at least one use case (Table 3.2), and each use case is linked back to stakeholder needs identified during elicitation. The thirteen functional requirement groups (FR1–FR13) cover the full system scope across fifteen use cases (UC0–UC14). New requirements added during development—enrollment profile capture (FR2.2), AI Instructor streaming (FR9), completion certificates (FR10), gamification with weekly goals (FR11), learning analytics (FR12), and instructor/admin portals (FR13)—are each traceable to a corresponding use case and to the stakeholder problems in Section 3.1.
3. **Testability Assessment:** Each requirement was reviewed for testability: is there a measurable criterion for success? Non-testable requirements were refined. For example, “Quiz interface shall be intuitive” is too vague; it is refined to “Quiz interface shall support task completion by 95% of users within 2 minutes on first attempt” (testable). Each functional requirement was evaluated for testability by defining measurable acceptance criteria.
4. **Feasibility and Risk Analysis:** Requirements were assessed for technical feasibility within scope and

budget. High-risk requirements were flagged and mitigation strategies identified. Key risks include: LLM API latency for FR3.1 and FR5.1–FR5.2 (quiz and explanation generation), mitigated by Redis caching and the 3-second threshold in NFR1.3; LLM provider unavailability, mitigated by the three-tier circuit-breaker fallback in NFR3.3; and the complexity of real-time SSE streaming (FR5.2, FR9.2), mitigated by Nginx SSE location blocks disabling proxy buffering and gzip compression on stream routes.

Overall, the validation confirms that the specified requirements are complete, consistent, testable, and achievable within the project scope.

Chapter 4

System Design

4.1 Overview

This chapter presents the complete system design for AI-Driven Personalized Learning Roadmap Generator And Tutor. The design translates the project objectives into a concrete, buildable architecture that supports personalised learning paths, adaptive assessments, streaming AI tutoring, decay-based spaced repetition, gamification, learning analytics, verifiable completion certificates, and role-stratified instructor and administrator interfaces—while remaining secure, scalable, and maintainable. The design explicitly separates concerns between authentication, learning-domain logic, and AI-assisted services to ensure each capability can evolve and scale independently. The chapter documents the data model, deployment topology, user interface strategy, system integration approach, security architecture, and verification steps that confirm the design satisfies all functional and non-functional requirements.

4.2 Specifying the Design Goals

The design goals align with the project scope, user needs, and operational constraints. Each goal guides architectural choices and is validated by concrete mechanisms in the implementation.

1. Correctness and completeness

- Implement the full learning lifecycle: domain selection, enrollment, roadmap generation, progression tracking, gatekeeper quizzes, branching, decay handling, and notifications.
- Preserve data integrity through strong relational constraints, indexed queries, and explicit state transitions for mastery.

2. Performance and responsiveness

- Keep learner interactions fast (e.g., roadmap view and progress retrieval) using optimised indexes and read patterns.

- Cache AI outputs and computed aggregates in Redis to avoid repeated expensive operations and reduce API latency; deliver streamed responses token-by-token via SSE to maximise perceived responsiveness for AI-heavy operations.

3. Scalability and extensibility

- Use a minimal microservice split to enable independent scaling of AI-heavy workloads and to isolate the security-critical gateway.
- Allow future services (e.g., recommendation engines, external LMS integrations) to integrate without redesigning core modules; analytics and velocity tracking are first-class features of the current design.

4. Security and privacy

- Enforce authentication and role-based authorization at the gateway and service layers.
- Store sensitive data securely (hashed refresh tokens, controlled PII exposure) and ensure transport-level encryption in production.

5. Availability and resilience

- Provide health checks and graceful failure paths; apply circuit breakers for AI services to prevent cascading failures.
- Maintain continuity in the learning flow even when AI providers are unavailable by using Redis-cached responses or automatically failing over to the next-tier provider in the three-tier chain (Phi-4 → Ollama → Gemini).

6. Usability and accessibility

- Deliver a clean user experience across web and mobile clients with consistent UI patterns, progressive disclosure, and helpful feedback.
- Support learners and instructors with task-specific views and clear next-step guidance.

4.3 System Design

4.3.1 Proposed Software Architecture

The system adopts a compact microservice architecture composed of three backend services, a reverse proxy, and shared infrastructure services (PostgreSQL and Redis). This architecture balances operational simplicity with the isolation required for security and AI workloads.

Backend services

- **api-gateway:** Handles authentication, OAuth, JWT issuance, RBAC enforcement, and user management.
- **learning-service:** Implements the core learning domain, including ontology management, enrollment, progress, quizzes, decay, branching, instructor and admin APIs.

- **ai-service:** Integrates external AI providers (Gemini) and local models (Phi-4-Multimodal, Ollama/Qwen2.5), applies Redis caching and per-provider circuit breakers, and serves both standard JSON endpoints and SSE streaming endpoints for real-time token delivery.

Shared infrastructure

- **PostgreSQL:** Single relational database containing the full learning model.
- **Redis:** Caching for AI responses, rate limiting, and transient state.
- **Nginx:** Reverse proxy and routing across backend services; SSE stream routes are configured with `proxy_buffering off` and gzip disabled to prevent token accumulation.

The architecture is implemented in the runtime topology in `backend/docker-compose.yml` and routing rules in `backend/nginx/nginx.conf`. The TypeScript service endpoints are `backend/services/api-gateway/src/app.ts` and `backend/services/learning-service/src/app.ts`; the Python AI service endpoint is `backend/services/ai-service`.

Figure 4.1 illustrates the proposed system architecture, showing the three backend services, the Nginx reverse proxy, and the shared PostgreSQL and Redis infrastructure.

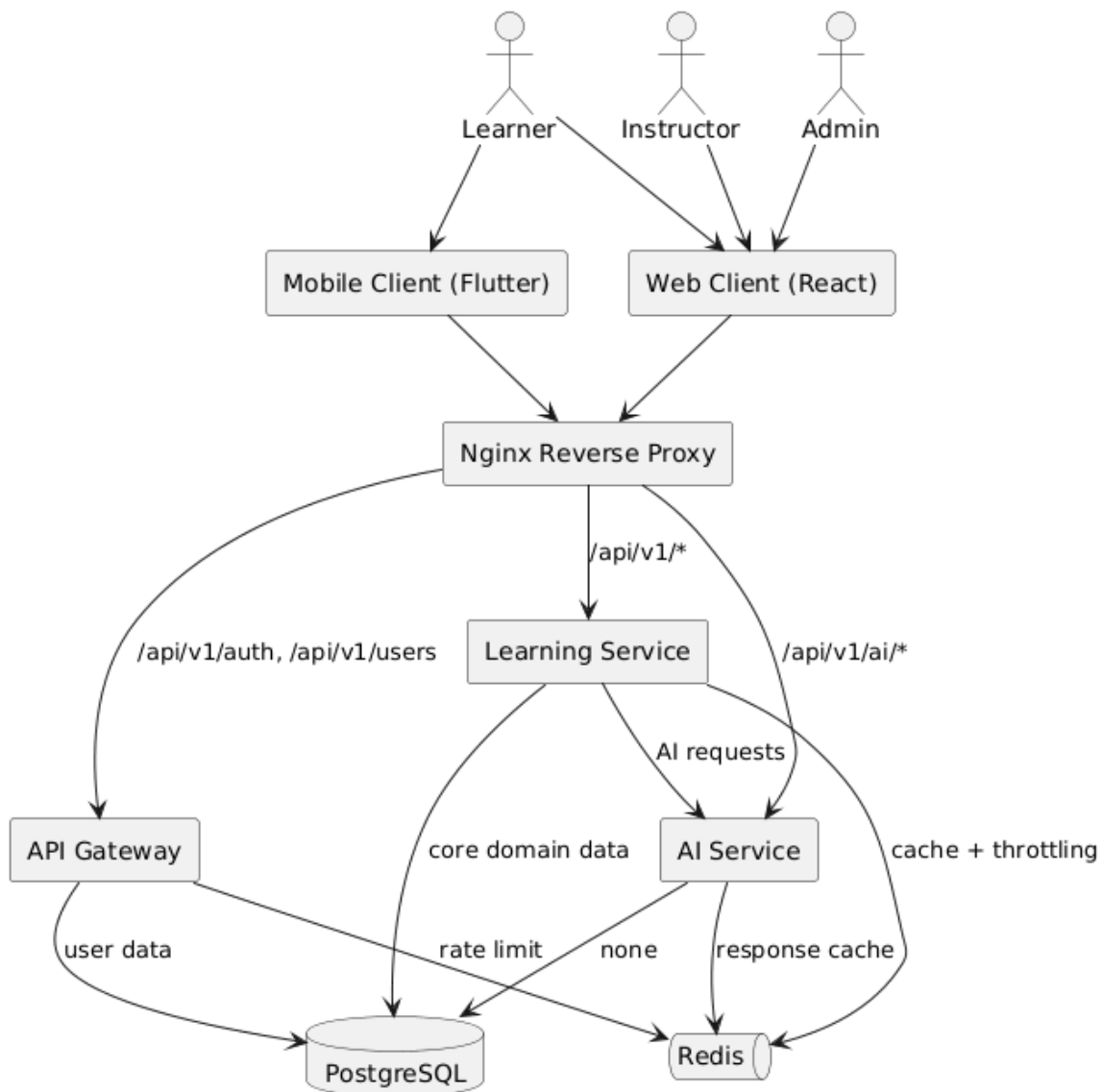


Figure 4.1. System Architecture Diagram

4.3.2 Subsystem Decomposition

The system is decomposed into logical subsystems to reduce complexity and isolate responsibilities.

1. Authentication and Gateway Subsystem

- **Responsibilities:** OAuth login, JWT issuance and verification, refresh token lifecycle, RBAC, user profile management.
- **Boundary:** All auth and user operations flow through the gateway; non-auth domain operations are proxied to the learning-service.

- **Implementation:** Modules under `backend/services/api-gateway/src/modules/auth` and `backend/services/api-gateway/src/modules/users`.

2. Learning Domain Subsystem

- **Responsibilities:** Ontology versioning and DAG management, enrollment (including learner profile capture), progress tracking, quizzes, gatekeeper evaluation, branching logic, mastery decay, notifications, gamification (XP, badges, streaks), learning velocity analytics, completion certificates, and instructor/admin portals.
- **Boundary:** Owns all database writes for the learning model and exposes domain-specific APIs.
- **Implementation:** Modules under `backend/services/learning-service/src/modules`.

3. AI Integration Subsystem

- **Responsibilities:** Quiz generation, explanation generation (standard and SSE streaming), AI Instructor conversational chat (SSE streaming), SERP API supplementary resource discovery, three-tier provider fallback (Phi-4-Multimodal primary, Qwen2.5-3B via Ollama secondary, Gemini 2.5 Flash API tertiary), Redis caching, and per-provider circuit breakers.
- **Boundary:** Exposes AI endpoints to `learning-service`; does not directly mutate core domain state.
- **Implementation:** `backend/services/ai-service/app/` (Python 3.11 / FastAPI).

4. Client Applications

- **Web client:** React + Vite application providing learner, instructor, and admin interfaces. Source code in `frontend/src`.
- **Mobile client:** Flutter application focusing on learner flow and essential admin features. Source code in `flutter_mobile/lib`.

5. Shared Infrastructure

- PostgreSQL for persistence and transactional integrity.
- Redis for caching and rate limiting.

4.3.3 Database Design

The data model is implemented in PostgreSQL and defined by Prisma in `backend/services/learning-service/prisma/schema.prisma`. This subsection lists all entities, attributes, constraints, and relationships to show how data is stored, accessed, and managed over time.

Enumerations

- **UserRole:** `learner`, `domain_expert`, `admin`
- **MasteryState:** `not_started`, `in_progress`, `mastered`, `review_needed`, `relearn`
- **QuizOutcome:** `strong_pass`, `marginal_pass`, `fail_low`, `fail_fundamental`, `fail_severe`

- **QuestionType:** multiple_choice, short_answer, code_completion, true_false, matching
- **ResourceModality:** documentation, tutorial, video, interactive, reference
- **OntologyStatus:** draft, in_review, verified, published, archived
- **AdaptationType:** resource_swap, prerequisite_review, instructor_escalation, decay_micro_quiz
- **BranchPath:** frontend, backend, data_science
- **XpSource:** node_mastered_strong, node_mastered_marginal, quiz_attempt, spaced_review, streak_milestone, enrollment_complete
- **BadgeKey:** first_mastery, streak_5, streak_14, quiz_ace, speed_learner, completionist, consistent, comeback

Entity tables

Table 4.1. *users* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NULLABLE
full_name	VARCHAR(255)	NOT NULL
role	user_role	DEFAULT learner
avatar_url	TEXT	NULLABLE
oauth_provider	VARCHAR(50)	NULLABLE
oauth_provider_id	VARCHAR(255)	NULLABLE
preferred_language	VARCHAR(10)	DEFAULT en
created_at	TIMESTAMPTZ	DEFAULT now()
updated_at	TIMESTAMPTZ	AUTO UPDATED

Table 4.2. *domains* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
name	VARCHAR(100)	UNIQUE, NOT NULL
slug	VARCHAR(100)	UNIQUE, NOT NULL
description	TEXT	NULLABLE
icon_url	TEXT	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.3. *ontology_versions* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
domain_id	UUID	FK → domains.id, NOT NULL
version_number	INT	NOT NULL
status	ontology_status	DEFAULT draft
created_by_id	UUID	FK → users.id, NOT NULL
verified_by_id	UUID	FK → users.id, NULLABLE
verified_at	TIMESTAMPTZ	NULLABLE
published_at	TIMESTAMPTZ	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()
		UNIQUE (domain_id, version_number)

Table 4.4. *learning_nodes* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
ontology_version_id	UUID	FK → ontology_versions.id, NOT NULL
title	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL
description	TEXT	NULLABLE
learning_outcomes	JSON	NOT NULL
estimated_hours	DECIMAL(4,1)	NULLABLE
difficulty_level	INT	NULLABLE
is_branching_point	BOOLEAN	DEFAULT false
is_convergence_point	BOOLEAN	DEFAULT false
branch_path	branch_path	NULLABLE
position_x	FLOAT	NULLABLE
position_y	FLOAT	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()
		UNIQUE (ontology_version_id, slug)

Table 4.5. *node_prerequisites* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
node_id	UUID	FK → learning_nodes.id, NOT NULL
prerequisite_node_id	UUID	FK → learning_nodes.id, NOT NULL UNIQUE (node_id, prerequisite_node_id)

Table 4.6. *enrollments* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL
domain_id	UUID	FK → domains.id, NOT NULL
ontology_version_id	UUID	FK → ontology_versions.id, NOT NULL
selected_branch_path	branch_path	NULLABLE
enrolled_at	TIMESTAMPTZ	DEFAULT now()
weekly_hours	INT	NULLABLE
familiarity_level	VARCHAR(20)	NULLABLE
learning_goal	VARCHAR(30)	NULLABLE
about_self	TEXT	NULLABLE
preferred_learning_style	VARCHAR(20)	NULLABLE
prior_skills	TEXT	NULLABLE
		UNIQUE (user_id, domain_id)

Table 4.7. *learner_node_progress* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL
node_id	UUID	FK → learning_nodes.id, NOT NULL
enrollment_id	UUID	FK → enrollments.id, NOT NULL
mastery_state	mastery_state	DEFAULT not_started
best_quiz_score	DECIMAL(5,2)	NULLABLE

Column	Type	Constraints
attempts_count	INT	DEFAULT 0
is_marginal_pass	BOOLEAN	DEFAULT false
mastered_at	TIMESTAMPTZ	NULLABLE
last_reviewed_at	TIMESTAMPTZ	NULLABLE
decay_notified_at	TIMESTAMPTZ	NULLABLE
unlocked	BOOLEAN	DEFAULT false
created_at	TIMESTAMPTZ	DEFAULT now()
updated_at	TIMESTAMPTZ	AUTO UPDATED
		UNIQUE (user_id, node_id)

Table 4.8. *quizzes table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
node_id	UUID	FK → learning_nodes.id, NOT NULL
is_micro_quiz	BOOLEAN	DEFAULT false
generated_by	VARCHAR(50)	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.9. *quiz_questions table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
quiz_id	UUID	FK → quizzes.id, NOT NULL (CASCADE DELETE)
question_type	question_type	NOT NULL
question_text	TEXT	NOT NULL
options	JSON	NULLABLE
correct_answer	TEXT	NOT NULL
explanation	TEXT	NULLABLE
order_index	INT	NOT NULL

Table 4.10. *quiz_attempts table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL
quiz_id	UUID	FK → quizzes.id, NOT NULL
node_id	UUID	FK → learning_nodes.id, NOT NULL
score_percent	DECIMAL(5,2)	NOT NULL
outcome	quiz_outcome	NOT NULL
answers	JSON	NOT NULL
started_at	TIMESTAMPTZ	NOT NULL
completed_at	TIMESTAMPTZ	NOT NULL

Table 4.11. *resources table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
node_id	UUID	FK → learning_nodes.id, NOT NULL
title	VARCHAR(500)	NOT NULL
url	TEXT	NOT NULL
source_domain	VARCHAR(255)	NOT NULL
modality	resource_modality	NOT NULL
description	TEXT	NULLABLE
is_primary	BOOLEAN	DEFAULT false
last_validated_at	TIMESTAMPTZ	NULLABLE
is_valid	BOOLEAN	DEFAULT true
avg_rating	DECIMAL(3,2)	DEFAULT 0.00
rating_count	INT	DEFAULT 0
fetched_via	VARCHAR(50)	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.12. *resource_ratings table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
resource_id	UUID	FK → resources.id, NOT NULL
user_id	UUID	FK → users.id, NOT NULL

Column	Type	Constraints
rating	INT	NOT NULL
comment	TEXT	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now() UNIQUE (resource_id, user_id)

Table 4.13. *domain_whitelist table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
domain_id	UUID	FK → domains.id, NOT NULL
source_domain	VARCHAR(255)	NOT NULL
source_name	VARCHAR(255)	NOT NULL
default_modality	resource_modality	NOT NULL
added_by_id	UUID	FK → users.id, NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now() UNIQUE (domain_id, source_domain)

Table 4.14. *adaptation_events table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL
node_id	UUID	FK → learning_nodes.id, NOT NULL
quiz_attempt_id	UUID	FK → quiz_attempts.id, NULLABLE
adaptation_type	adaptation_type	NOT NULL
details	JSON	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.15. *challenge_projects table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
node_id	UUID	FK → learning_nodes.id, NOT NULL

Column	Type	Constraints
title	VARCHAR(500)	NOT NULL
description	TEXT	NOT NULL
difficulty_level	INT	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.16. *notifications table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL
type	VARCHAR(50)	NOT NULL
title	VARCHAR(255)	NOT NULL
body	TEXT	NULLABLE
data	JSON	NULLABLE
read	BOOLEAN	DEFAULT false
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.17. *refresh_tokens table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL (CASCADE DELETE)
token_hash	VARCHAR(255)	NOT NULL
expires_at	TIMESTAMPTZ	NOT NULL
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.18. *certificates table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
public_id	VARCHAR(20)	UNIQUE, NOT NULL (format ATL-XXXX-XXXX)
user_id	UUID	FK → users.id ON DELETE CASCADE
enrollment_id	UUID	UNIQUE, FK → enrollments.id ON DELETE CASCADE

Column	Type	Constraints
domain_id	UUID	FK → domains.id, NOT NULL
recipient_name	VARCHAR(255)	NOT NULL
course_name	VARCHAR(255)	NOT NULL
hours_invested	DECIMAL(6,1)	NULLABLE
average_score	DECIMAL(5,2)	NULLABLE
completed_at	TIMESTAMPTZ	NOT NULL
issued_at	TIMESTAMPTZ	DEFAULT now()

Table 4.19. *learner_velocity* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id, NOT NULL
enrollment_id	UUID	FK → enrollments.id ON DELETE CASCADE
node_id	UUID	FK → learning_nodes.id, NOT NULL
estimated_hours	DECIMAL(4,1)	NULLABLE
actual_hours	DECIMAL(6,2)	NOT NULL
started_at	TIMESTAMPTZ	NOT NULL
completed_at	TIMESTAMPTZ	NOT NULL
velocity_ratio	DECIMAL(4,2)	NOT NULL (actual / estimated) UNIQUE (user_id, node_id)

Table 4.20. *xp_events* table

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id ON DELETE CASCADE
source	xp_source	NOT NULL
amount	INT	NOT NULL
ref_id	UUID	NULLABLE (quiz_attempt or node reference)
created_at	TIMESTAMPTZ	DEFAULT now()

Table 4.21. *user_badges table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
user_id	UUID	FK → users.id ON DELETE CASCADE
badge_key	badge_key	NOT NULL
earned_at	TIMESTAMPTZ	DEFAULT now() UNIQUE (user_id, badge_key)

Table 4.22. *supplementary_nodes table*

Column	Type	Constraints
id	UUID	PK, default gen_random_uuid()
enrollment_id	UUID	FK → enrollments.id ON DELETE CASCADE
title	VARCHAR(255)	NOT NULL
description	TEXT	NULLABLE
node_type	VARCHAR(30)	NOT NULL (e.g., primer, practice_exam)
target_node_id	UUID	FK → learning_nodes.id, NULLABLE
position	VARCHAR(15)	NOT NULL (before / after target node)
content	JSON	NULLABLE
created_at	TIMESTAMPTZ	DEFAULT now()

Relationships summary

- users 1..* enrollments, learner_node_progress, quiz_attempts, resource_ratings, notifications, adaptation_events, refresh_tokens, xp_events, user_badges, learner_velocity
- domains 1..* ontology_versions, enrollments, domain_whitelist, certificates
- enrollments 1..1 certificates; enrollments 1..* supplementary_nodes, learner_velocity
- ontology_versions 1..* learning_nodes
- learning_nodes 1..* quizzes, resources, challenge_projects, learner_node_progress, adaptation_events, learner_velocity
- learning_nodes *.* learning_nodes via node_prerequisites
- learning_nodes 1..* supplementary_nodes (optional target reference)

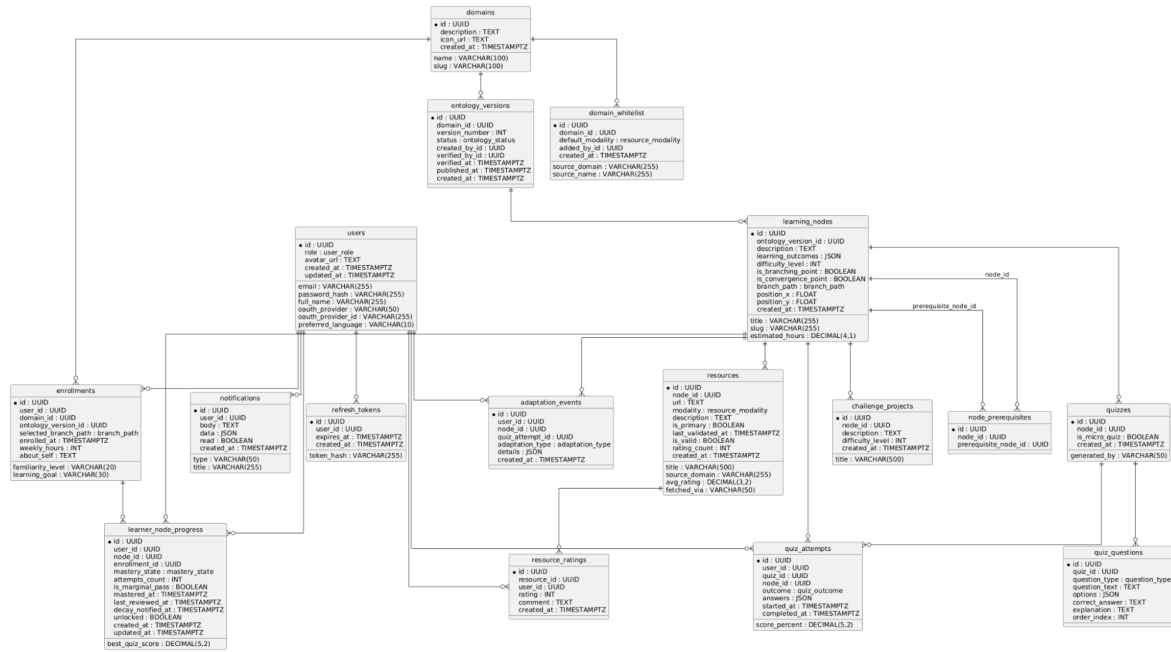


Figure 4.2. Entity-Relationship Diagram (Full Attributes)

4.3.4 Deployment Diagram

The deployment configuration is defined in `backend/docker-compose.yml`. The reverse proxy routing rules are defined in `backend/nginx/nginx.conf`. The physical arrangement below clarifies network boundaries, service responsibilities, and shared infrastructure, which helps identify scaling or performance bottlenecks.

Deployment topology (physical view)

- Clients (web and mobile) communicate over HTTPS to the reverse proxy.
- Nginx routes requests by URL prefix to the correct backend service.
- **api-gateway** and **learning-service** connect to the same PostgreSQL database and Redis instance.
- **ai-service** uses Redis for cache and throttling but does not write to the database.
- Each service runs in its own container for isolation and independent scaling.

Scalability and performance considerations

- The reverse proxy is a potential choke point; in production, use a managed load balancer or ingress controller.
- **ai-service** is the most latency-sensitive component; it should scale horizontally and use caching to reduce AI-provider load.
- PostgreSQL is the primary shared dependency; its performance directly affects read-heavy endpoints like roadmap and progress.

- Redis reduces database pressure by caching AI responses and rate-limiter state.

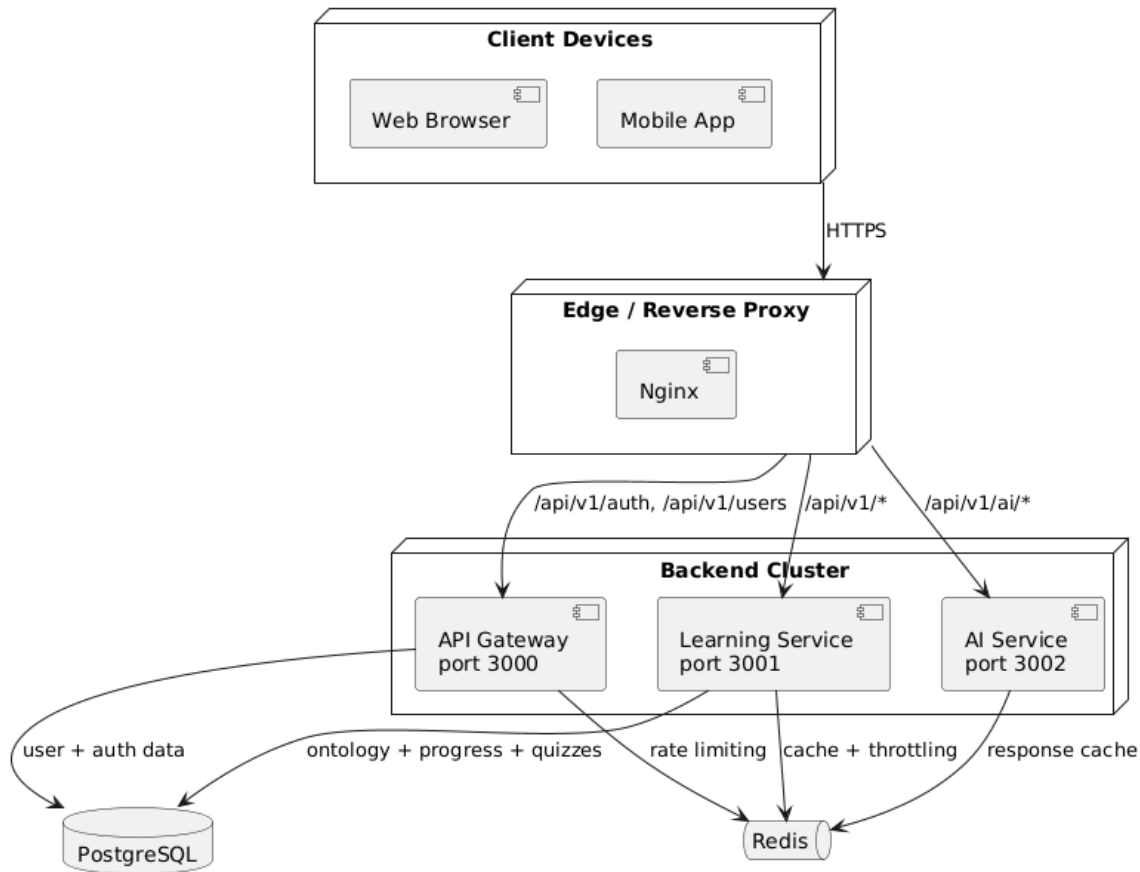


Figure 4.3. *Deployment Diagram*

4.3.5 User Interface Design

The UI layer follows a user-centered design approach focused on clarity, progressive disclosure, and continuous feedback. Learners are guided from onboarding to actionable next steps, while instructors and administrators are provided with analytical views and management workflows. The design prioritizes accessibility, predictable navigation, and minimal cognitive load.

Design principles (user-centered)

- **Learnability:** Clear, consistent layouts across web and mobile reduce onboarding time.
- **Guidance:** Contextual panels and progress indicators highlight next actions.
- **Feedback:** Immediate visual feedback for quiz outcomes, mastery states, and system notifications.
- **Accessibility:** High-contrast typography, keyboard focus states, and scalable layout on different devices.

Web client: Implemented in React 18 with Vite; feature modules are organised under `frontend/src/features`. Core pages include: Dashboard, Domain Catalogue (enrollment flow), Roadmap, Learn (explanation + AI Instructor), Quiz, Quiz Attempt Review, Notifications, Learning Insights (per-enrollment and global), Achievements, Certificate (claim + download + share), Instructor portal (learner monitoring, analytics, flagged events), and Admin console (user management, domain management, ontology builder, system statistics).

Mobile client: Implemented in Flutter; features include enrollments, roadmap view, quiz attempts, and notifications. Source code in `flutter_mobile/lib`.

Key interaction patterns:

- **Roadmap view:** React Flow DAG canvas with a fixed left sidebar (progress stats, branch path selector, mastery breakdown, decay timeline, “Get your certificate” CTA at 100%) and a slide-out node detail panel.
- **Learn view:** Content-first layout with a streaming AI explanation panel (SSE token rendering), tabbed resource links, and a persistent AI Instructor chat panel in the right column.
- **Quiz flow:** MCQ question cards with a progress indicator, immediate post-quiz outcome screen showing score, tier classification, and adaptation recommendation.
- **Insights view:** Activity heatmap, velocity trend chart, mastery breakdown, and estimated completion timeline with velocity multiplier.
- **Achievements view:** Grid of earned and locked badges with unlock criteria, XP level progress bar, and streak counter.
- **Certificate view:** Full-resolution certificate canvas with PDF/PNG download buttons and a copy-to-clipboard share link pointing to the public `/verify/:publicId` page.
- **Instructor/Admin dashboards:** Table + summary cards for quick status review, with drill-down panels for individual learner progress and flagged escalation events.

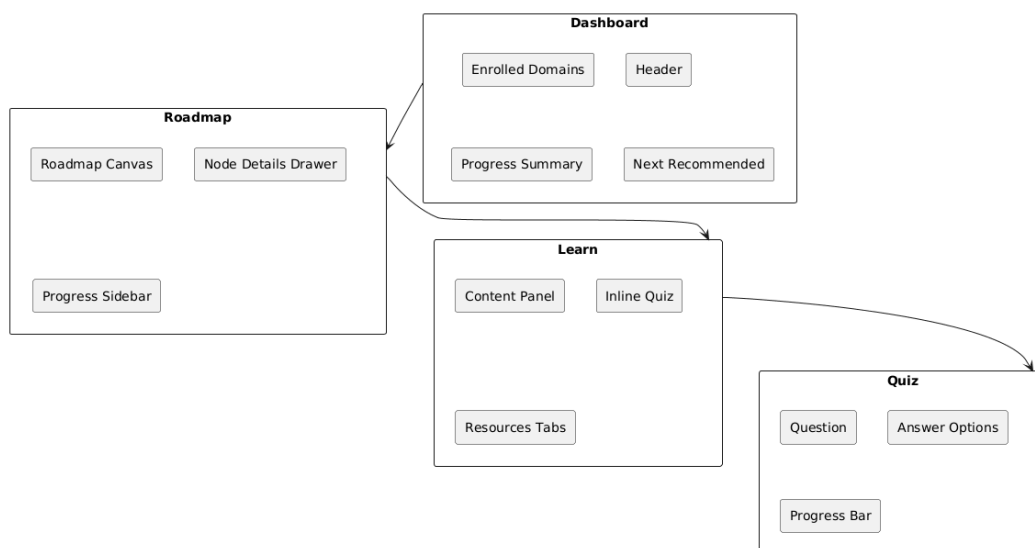


Figure 4.4. *UI Mockups and Wireframe Navigation*

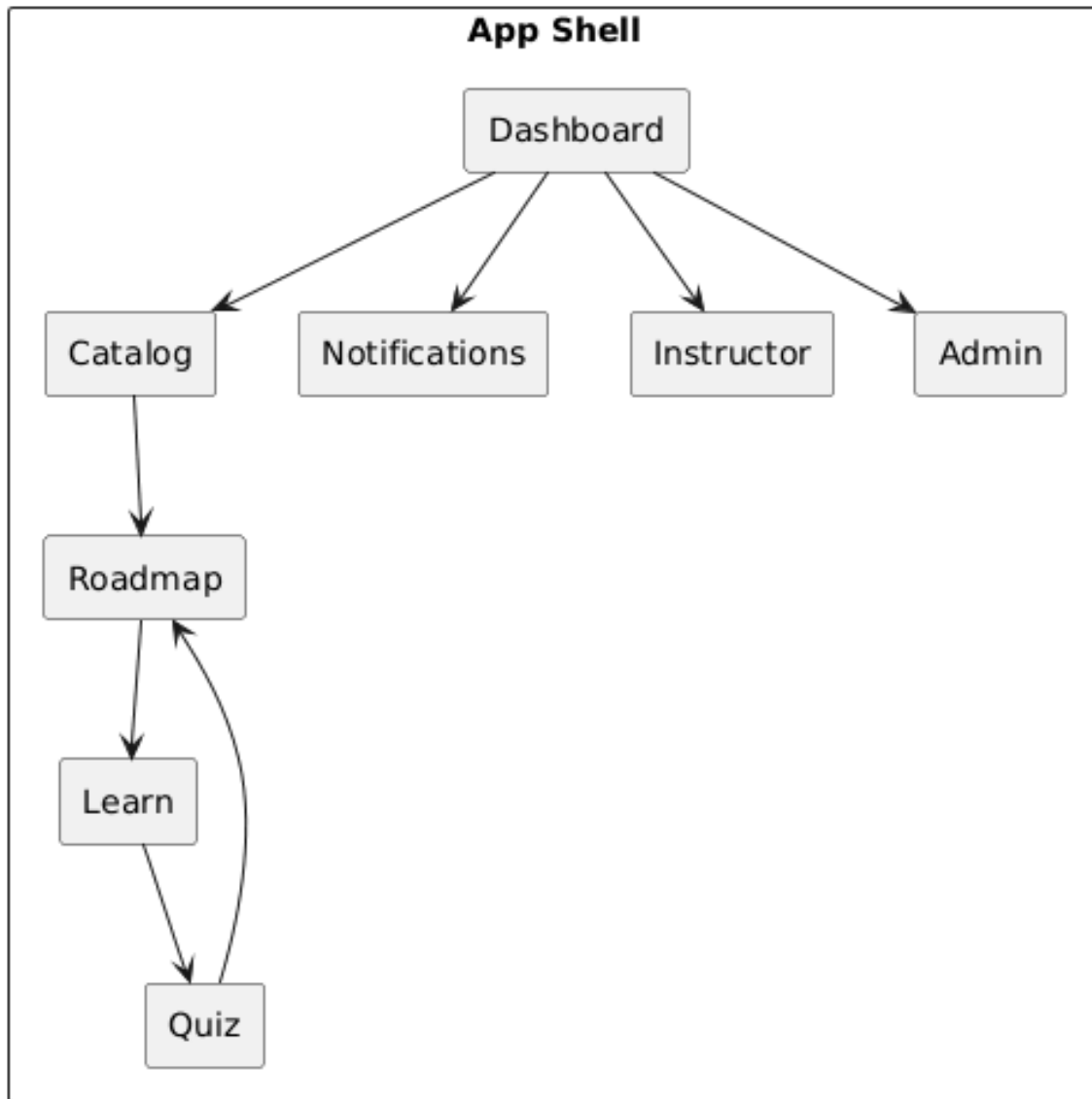
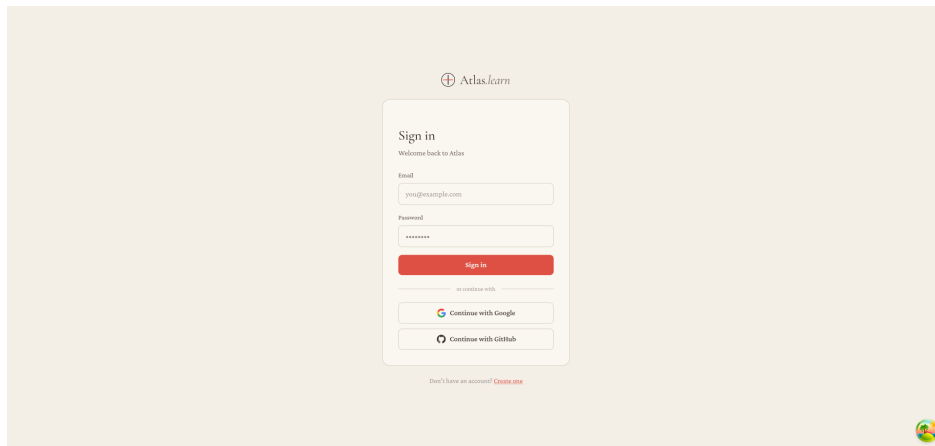
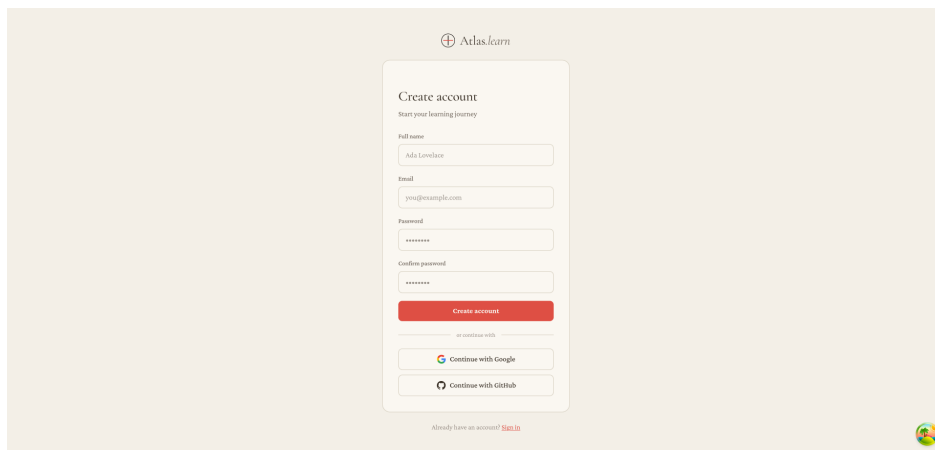
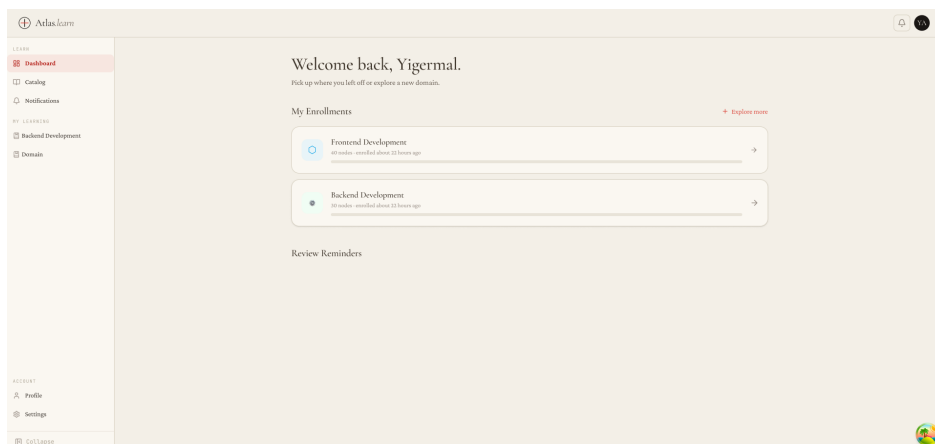


Figure 4.5. *UI Navigation Diagram*

Figma UI Screen Designs

The following screen designs were created in Figma to guide the frontend implementation.

**Figure 4.6.** *Login Page***Figure 4.7.** *Sign Up Page***Figure 4.8.** *Dashboard Page*

CHAPTER 4. SYSTEM DESIGN



Figure 4.9. Overview Page



Figure 4.10. Roadmap Page

Figure 4.11. Explanation Page

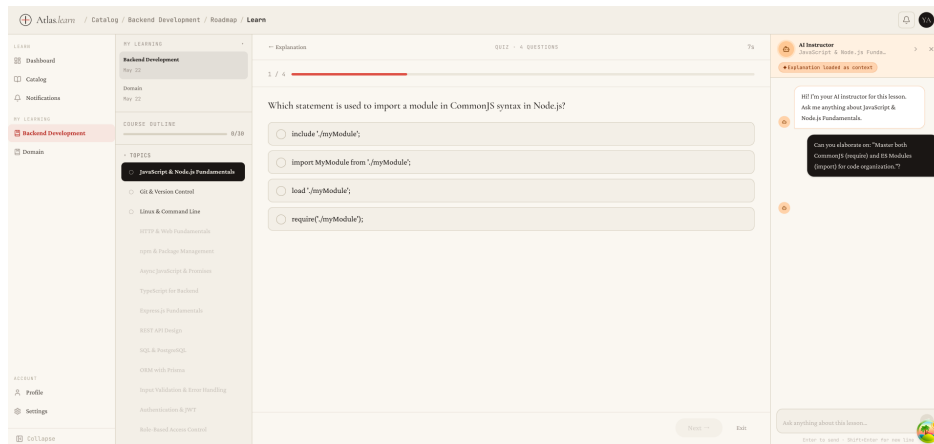


Figure 4.12. Quiz Page

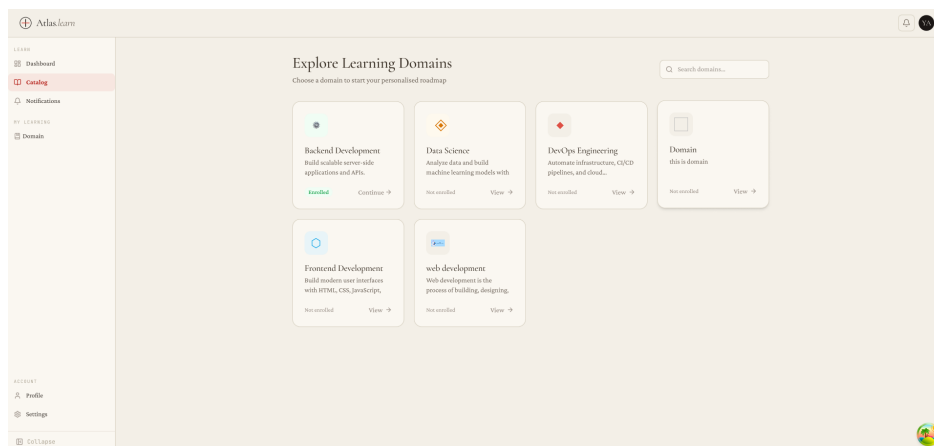


Figure 4.13. Catalog Page

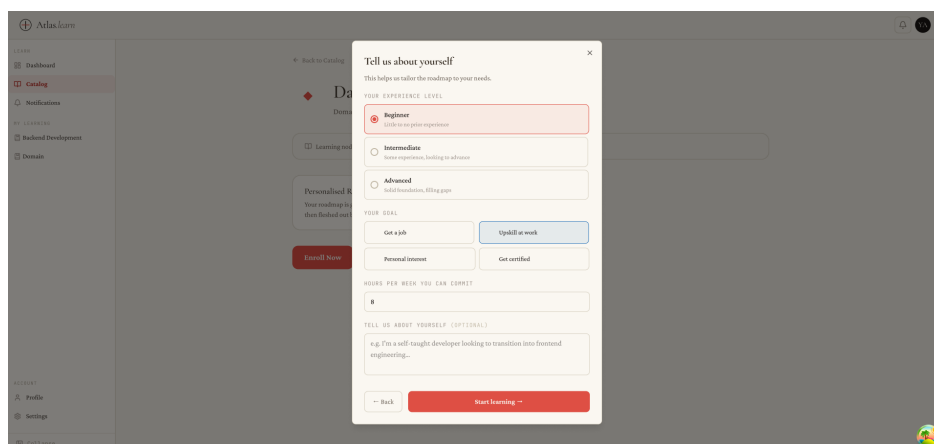


Figure 4.14. Enroll Page

4.3.6 System Integration

System integration defines how independent components cooperate to deliver a seamless learning experience. It covers API interoperability, data exchange, and the use of third-party services such as OAuth providers and AI platforms. The integration strategy prioritizes clear boundaries, strict contracts, and resilient communication.

Integration architecture and interoperability

- **Service boundaries:** Each backend service exposes a stable REST API. The api-gateway owns authentication and user identity; the learning-service owns domain data and learning flows; the ai-service owns AI and resource generation. The reverse proxy routes traffic so clients do not need to manage service discovery.
- **Compatibility:** All APIs are versioned under `/api/v1`, enabling backward compatibility. Swagger/OpenAPI documents are exposed for each service, and clients rely on these contracts to generate or validate API calls.
- **Data exchange:** JSON is the primary exchange format. All services enforce request/response schemas and validate payloads to prevent mismatches or malformed data.

Integration flows (detailed)

1. Authentication and session management

- The client authenticates via OAuth (Google, GitHub) or email/password through the api-gateway.
- On success, the gateway issues a JWT access token and a hashed refresh token, storing refresh tokens in the database.
- The client includes `Authorization: Bearer <token>` for all protected requests to the learning-service.

2. Learning data flow

- The client requests a roadmap; the learning-service retrieves ontology versions, nodes, and progress data from PostgreSQL.
- Progress updates (quiz results, mastery transitions) are persisted and may trigger adaptation events and notifications.

3. AI-assisted generation flow

- When a learner requests a quiz or explanation, the learning-service calls the ai-service.
- The ai-service checks Redis for cached outputs; on cache miss, it queries the three-tier provider chain: Phi-4-Multimodal (primary) → Qwen2.5-3B via Ollama (secondary) → Gemini 2.5 Flash API (last resort), with per-provider circuit breakers.
- For standard quiz generation: results are cached and returned as JSON to the learning-service, which persists quiz metadata and serves the response.

- For streamed explanations and AI Instructor chat: the ai-service opens an SSE stream and emits tokens directly to the learning-service proxy, which forwards them to the client without buffering. The Nginx SSE location block ensures `proxy_buffering off` on these routes.

4. Notification and background workflows

- Scheduled tasks in the learning-service analyze decay and progress to generate reminders or micro-quizzes.
- Notifications are stored and delivered to the client in the notifications panel.

Third-party services integration

- **OAuth providers:** Google and GitHub are used for external authentication. Callback routes are hosted in the api-gateway, ensuring consistent user identity handling.
- **AI providers:** Phi-4-Multimodal (primary, via Kaggle), Qwen2.5-3B (secondary, via Ollama), and Gemini 2.5 Flash (tertiary, API last resort) are abstracted behind ai-service; this isolates AI dependencies from core business logic.
- **Search provider:** Supplementary resource discovery uses the SERP API, encapsulated in the ai-service. Queries are scoped to the node's verified learning outcomes to keep results pedagogically aligned.

Integration reliability and error handling

- Timeouts and circuit breakers are applied to AI calls to prevent upstream failures from impacting the core learning experience.
- The circuit breaker pattern tracks consecutive failures per provider (threshold: 5 failures within 5 minutes), automatically falling back to the next tier.
- Retries and backoff policies are applied to transient errors in AI and search providers.
- Each service exposes `/health` endpoints for monitoring and automated recovery.

Figure 4.15 illustrates the authentication and AI-assisted generation flows across the system components.

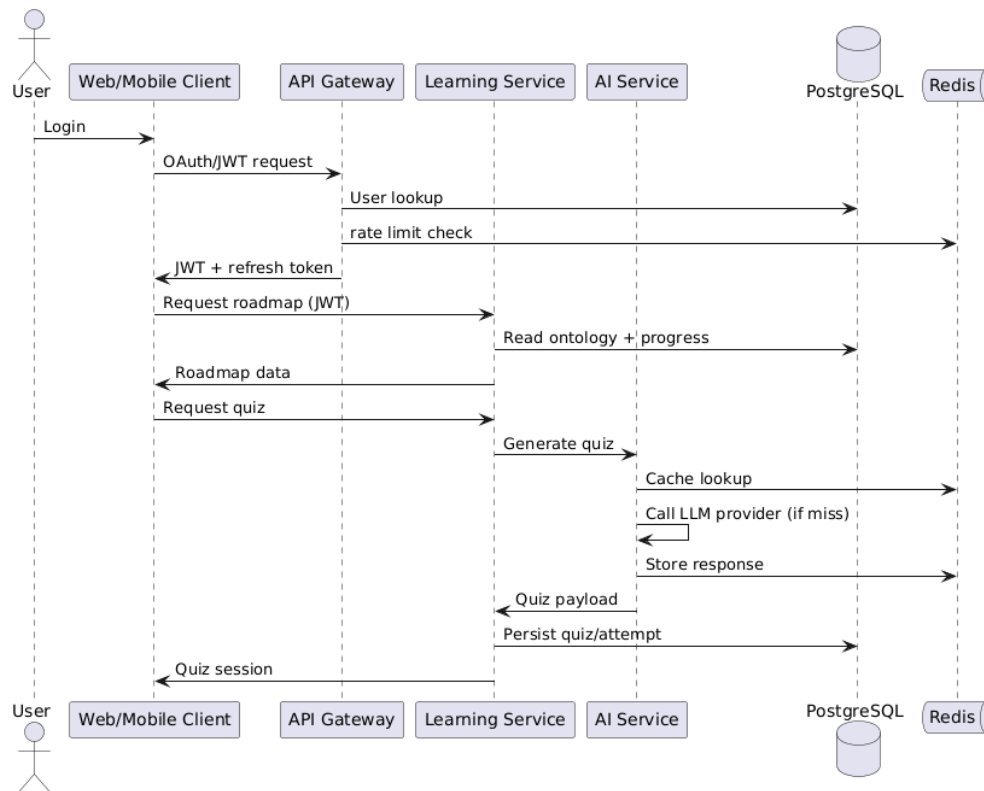


Figure 4.15. Sequence Diagram (Auth + AI Flow)

4.3.7 Security Design

Security is integrated throughout the system to protect user data, prevent abuse, and ensure trustworthy operation. The security design focuses on authentication, authorization, data protection, secure communication, and operational safeguards.

Authentication

- The system uses JWT access tokens for stateless authentication, issued by the api-gateway.
- Refresh tokens are stored hashed in the database to mitigate token leakage risks.
- OAuth integration uses secure callback URLs and state validation to protect against CSRF attacks.

Authorization (RBAC)

- Role-based access control ensures learners, instructors, and admins only access permitted routes.
- Gateway middleware enforces role checks for sensitive endpoints, and services validate roles for defense in depth.
- Three roles are defined: `learner`, `domain_expert`, and `admin`, each with distinct permission sets.

Encryption and secure transport

- TLS is required in production to protect credentials and tokens in transit.
- Sensitive data (password hashes, refresh tokens) is stored using strong one-way hashing: passwords are hashed with `scrypt` (Node.js `crypto.scrypt`, parameters `N=16384`, `r=8`, `p=1`) with a random 16-byte salt; refresh tokens are stored as SHA-256 hashes.
- Database encryption at rest is planned for production deployment.

Input validation and sanitization

- All incoming requests are validated against schemas (Joi), rejecting malformed or unexpected payloads.
- Sanitization middleware removes potentially unsafe characters from user inputs to mitigate injection attacks.

Rate limiting and abuse prevention

- Redis-backed rate limiting is applied globally and specifically to authentication endpoints.
- Brute-force attacks are mitigated through request throttling and account lockout policies.

AI and third-party risk controls

- AI service calls are wrapped with circuit breakers and timeouts to prevent cascading failures.
- Cached AI responses reduce repeated exposure to external systems and limit provider dependency.
- Three-tier provider fallback ensures availability even when primary AI services are unreachable.

Auditability and monitoring

- Structured logging captures authentication attempts, errors, and critical domain actions.
- Health endpoints enable monitoring and alerting for availability and security incidents.

Secure configuration and secrets management

- Development uses environment variables; production should use a dedicated secrets manager.
- Sensitive values are excluded from version control and injected during deployment.

Hardening and testing

- Dependency scanning, static analysis, and penetration testing are required before deployment.
- A hardening checklist is documented in `docs/backend/12-hardening.md`.

4.4 Verifying the Requirements in the Design

This section verifies the design against the complete set of functional requirements (FR1–FR13) and non-functional requirements (NFR1–NFR7). The verification strategy combines traceability, architectural inspection, automated testing, and operational validation. Each requirement is mapped to design artifacts and a concrete verification method, with clear evidence and a defined discrepancy-resolution process.

4.4.1 Requirements Traceability Approach

- **Mapping:** Every requirement is linked to a subsystem and one or more design artifacts (data model, API routes, service modules, UI screens, or infrastructure).
- **Verification:** Each requirement has a validation method (unit, integration, end-to-end, load, or security tests).
- **Evidence:** Evidence is collected from tests, CI runs, and runtime checks (health endpoints, logs, and monitoring).

4.4.2 Functional Requirements Verification Matrix

Table 4.23. *Functional requirements verification matrix*

Requirement	Design Coverage	Verification Method	Evidence/Artifact
FR1.1 Ontology generation (Teacher Model)	Ontology module + JSON structure in learning-service	Offline generation test + JSON schema validation	Ontology JSON schema tests
FR1.2 Expert verification workflow	Admin/instructor APIs + ontology status fields	Integration tests for review/approve flow	Admin/ontology routes + role checks
FR1.3 Ontology versioning + rollback	OntologyVersion model + status transitions	Integration tests for version creation and retrieval	Prisma schema + ontology tests
FR2.1 User registration + social login	api-gateway auth + Google/GitHub OAuth callbacks	Auth flow tests + OAuth callback tests	Auth module + login E2E
FR2.2 Enrollment + learner profile capture	Enrollment model + 7 profile fields + branch path selection	Integration tests for enrollment creation, profile field persistence	Enrollment routes + DB constraints

Requirement	Design Coverage	Verification Method	Evidence/Artifact
FR2.3 Progress tracking	LearnerNodeProgress model + progress routes	Integration tests for attempts, timestamps, mastery state transitions	Progress routes + DB constraints
FR2.4 Learner statistics + timeline	Aggregation queries + LearnerVelocity model	API tests for stats and velocity-adjusted timeline endpoints	Progress stats + velocity routes
FR3.1 Quiz generation (Tutor Model)	ai-service quiz generation + learning-service persistence	Contract tests + schema validation	AI routes + quiz persistence tests
FR3.2 MCQ with optional code snippets	QuestionType enum + quiz_questions table	Unit tests for generation schema + code snippet field support	Prisma enums + quiz validation
FR3.3 Quiz evaluation + feedback	QuizAttempt + scoring logic + outcome feedback	Unit tests for scoring + E2E quiz flow	Quiz service tests + Playwright
FR3.4 Micro-quizzes on decay	Decay scheduler + ai-service micro-quiz endpoint	Integration tests for decay state transition triggers	Decay module + notification tests
FR4.1 Five-tier progression logic	Gatekeeper service + five mastery state transitions + adaptation events	Unit tests for all five tier thresholds and event types	Gatekeeper tests
FR4.2 Prerequisite validation	NodePrerequisite + roadmap unlock logic	Integration tests for lock/unlock on prerequisite completion	NodePrerequisite constraints
FR4.3 Challenge project recommendation	ChallengeProject + post-strong-pass flow	Integration tests after strong pass ($\geq 80\%$)	Challenge project service tests
FR4.4 Resource adaptation on fail	AdaptationEvent + adaptation service (resource_swap, prerequisite_review, escalation)	Integration tests for all fail-tier outcomes	Adaptation event logs
FR5.1 AI explanation generation	ai-service explanation module + node context prompt builder	Contract tests + ontology-grounding schema validation	AI explanation route tests
FR5.2 SSE streaming delivery	ai-service SSE endpoints + Nginx proxy_buffering off	Integration tests for token streaming + first-token latency	SSE stream tests

Requirement	Design Coverage	Verification Method	Evidence/Artifact
FR5.3 Offline expert curation	resources table + admin/ontology builder resource attachment API	CRUD tests for node resource creation and update	Resource routes + admin panel tests
FR5.4 SERP API supplementary discovery	ai-service SERP client (scoped to node learning outcomes)	API tests with mocked SERP responses	SERP integration tests
FR5.5 Tutorial-style regeneration on swap	resource_swap event + ai-service re-invocation with style instruction	Integration tests: swap event → regenerated tutorial-style content	Adaptation service tests
FR6.1 Decay computation (7-day marginal, 14-day strong)	Decay service + mastery timestamps + is_marginal_pass flag	Unit tests for both 7-day and 14-day decay thresholds	Decay service tests
FR6.2 Auto micro-quiz on Yellow/Red transition	Decay scheduler + ai-service micro-quiz	Integration test for decay-to-micro-quiz pipeline	Decay scheduler tests
FR6.3 Visualise decay states in roadmap	Roadmap UI + node mastery state colour coding	UI tests for Green/Yellow/Red node colours	Frontend roadmap tests
FR6.4 In-app decay notifications	Notifications module + decay trigger	Integration tests for notification creation on decay transition	Notifications routes
FR7.1 Branch path tagging on nodes	BranchPath enum on LearningNode + shared/branch-specific node rules	Unit tests for branch node identification and filtering	Ontology validation tests
FR7.2 Enrollment-time branch selection	Enrollment.selectedBranchPath + enrollment API	Integration tests for branch path persistence on enrollment create	Enrollment routes + DB
FR7.3 Runtime branch switching via sidebar	Roadmap filtering API + frontend branch selector	E2E test for branch switch and immediate roadmap re-render	Roadmap endpoint + frontend tests
FR7.4 Prerequisite integrity across branch switch	DAG unlock logic + cross-branch prerequisite validation	Integration tests for cross-branch prerequisite locks	Roadmap unlock tests

Requirement	Design Coverage	Verification Method	Evidence/Artifact
FR8.1 Interactive DAG visualisation (React Flow)	Roadmap canvas components + node detail panel	UI interaction tests (zoom, pan, node select, decay colours)	Roadmap component tests
FR8.2 Full learner web interface	UI pages: roadmap, learn, quiz, AI panels, insights, achievements, certificate	E2E tests across all learner-facing pages	Playwright suites
FR8.3 Mobile UX + push notifications	Flutter app + push notification support	Mobile tests + notification trigger simulation	Flutter tests
FR8.4 Responsive web UI	Tailwind CSS breakpoints + layout	Visual regression tests across breakpoints	Frontend layout tests
FR9.1 AI Instructor chat panel (per node)	ai-service ask-question endpoint + learning-service proxy	Integration tests for contextual Q&A grounding in node context	AI Instructor route tests
FR9.2 SSE streaming for AI Instructor	ai-service ask-question/stream + Nginx SSE location block	Integration tests for token streaming + latency	SSE ask route tests
FR9.3 Node-context prompt grounding	Prompt builder + full node context injection (title, outcomes, difficulty)	Unit tests for prompt structure and context completeness	Prompt unit tests
FR10.1 Certificate issuance (idempotent)	certificates.service + certificates table + 100% mastery check	Integration tests for claim + idempotency on double-claim	Certificate service tests
FR10.2 Crockford base-32 public ID	ID generator + UNIQUE DB constraint + collision retry loop	Unit tests for ID format, uniqueness, and collision handling	ID generation tests
FR10.3 PDF/PNG client-side download	CertificatePage + html-to-image + jsPDF (A4 landscape)	E2E download test + file format validation	Frontend certificate tests
FR10.4 Public verify page (/verify/:publicId)	VerifyCertificatePage + public no-auth GET endpoint	Integration + E2E tests for public access without authentication	Certificate verify route tests

Requirement	Design Coverage	Verification Method	Evidence/Artifact
FR11.1 XP award on quiz pass	XpEvent model + gamification service + XpSource enum	Unit tests for XP amounts by tier and attempt-efficiency bonus	Gamification service tests
FR11.2 Level progression from XP	Level threshold table + XP aggregation query + progress bar	Unit tests for level thresholds and progress percentage	Level computation tests
FR11.3 Streak tracking	Daily activity computation + streak increment/reset logic	Integration tests for streak increment, milestone, and Monday reset	Streak service tests
FR11.4 Achievement badges	UserBadge model + BadgeKey enum + badge trigger logic	Integration tests for all named badge trigger conditions	Badge service tests
FR11.5 Weekly learning goals	Weekly goal tracker + Monday reset + progress display	Integration tests for goal progress and weekly reset	Weekly goal service tests
FR12.1 Velocity data recording	LearnerVelocity model + mastery transition hook	Integration tests for velocity record creation on mastery completion	Velocity service tests
FR12.2 Velocity-adjusted timeline forecast	getTimelineEstimate + weeklyHours + average velocityRatio	Unit tests for forecast computation with velocity multiplier edge cases	Timeline estimation tests
FR12.3 Per-enrollment insights dashboard	InsightsPage + velocity/activity/mastery APIs	E2E tests for heatmap, velocity trend, and timeline rendering	Insights route + frontend tests
FR12.4 Global insights (cross-enrollment)	GlobalInsightsPage + cross-enrollment aggregation queries	E2E tests for global insights view + multi-domain learner	Global insights route tests
FR13.1 Instructor portal (monitoring + resolve events)	instructor routes + LearnerListPage + FlaggedEventsPage + resolve endpoint	Integration tests for learner list, drill-down, analytics, event resolve	Instructor route tests

Requirement	Design Coverage	Verification Method	Evidence/Artifact
FR13.2 Admin console	admin routes + Admin-Layout (users, domains, ontology builder, system stats)	Integration tests for all CRUD operations and admin-only role enforcement	Admin route tests

4.4.3 Non-Functional Requirements Verification Matrix

Table 4.24. *Non-functional requirements verification matrix*

Requirement	Design Coverage	Verification Method	Evidence/Artifact
NFR1.1 Roadmap load $\leq 2s$	Indexed queries + Redis caching	Performance test + p95 latency	Load test report
NFR1.2 Quiz API p95 $\leq 500ms$	Rate-limited endpoints + caching	Load test + API latency tracking	API performance report
NFR1.3 LLM quiz generation $\leq 3s$	AI Redis cache + three-tier fallback	Timing tests + cache hit ratio	AI service metrics
NFR1.4 SSE first-token $\leq 1s$	Nginx proxy_buffering off + ai-service streaming	SSE timing tests for first-token arrival latency	Stream latency tests
NFR2.1 1,000 concurrent users	Horizontal scaling + stateless API	Load test with concurrency	Load test report
NFR2.2 DB/Redis scaling	Read replicas + caching design	Architectural validation	Deployment plan
NFR2.3 Multi-tenancy ready	Namespacing strategy + config boundaries	Design review	Architecture review notes
NFR3.1 99.5% uptime	Health checks + monitoring	Availability monitoring	Uptime dashboard
NFR3.2 Backup + recovery (4h)	DB backup plan	Recovery drill	Backup logs
NFR3.3 Graceful AI fallback (three-tier chain)	Per-provider circuit breakers + Redis cache + Phi-4 $\rightarrow$ Ollama $\rightarrow$ Gemini chain	Provider failure simulation (kill Phi-4, kill Ollama, verify Gemini fallback)	AI circuit breaker tests
NFR4.1 JWT tokens + scrypt password hashing	Gateway auth + scrypt (N=16384, r=8, p=1) + DB-stored hashed refresh tokens	Security review + auth token rotation E2E test	Auth tests

Requirement	Design Coverage	Verification Method	Evidence/Artifact
NFR4.2 Encryption in transit/at rest	TLS + DB encryption plan	Security audit	Security checklist
NFR4.3 RBAC roles	Role-based middleware	Role access tests	RBAC test suite
NFR4.4 Input sanitization	Validation + sanitizer middleware	Security tests	Validation tests
NFR4.5 Data confidentiality	Consent + data handling rules	Policy review	Privacy checklist
NFR5.1 Onboarding <5 min	Simplified UX flow	Usability test	UX test notes
NFR5.2 Quiz UI clarity	UI feedback patterns	Usability testing	UX survey
NFR5.3 Roadmap interpretability	Legend + tooltips	Usability testing	UX feedback
NFR5.4 Error messages clarity	Consistent error UI	UX review	Error-state review
NFR6.1 Coding standards + docs	Linting + docs	CI lint + doc review	CI logs
NFR6.2 Add new domains easily	Ontology versioning + seeding	Design review	Architecture notes
NFR6.3 API versioning	/api/v1 routes	API contract review	API docs
NFR7.1 English primary + i18n-ready architecture	i18n-compatible component structure; full localisation deferred	i18n architecture review + string externalisation audit	i18n structure review
NFR7.2 Ethiopian context	Content guidelines	Content review	Editorial checklist
NFR7.3 Low-bandwidth support	Cache + optimized payloads	Network throttling tests	Performance report

4.4.4 Verification Methods and Evidence

The following verification methods are employed to validate the design:

- **Unit tests:** Validate isolated business rules—DAG validation, gatekeeper five-tier thresholds, decay state transitions (7-day and 14-day), XP computation, badge trigger conditions, velocity ratio calculation, and Crockford ID generation.
- **Integration tests:** Verify DB constraints, service-to-service calls, cache usage, and cross-module workflows (enrollment creation, quiz-gatekeeper-adaptation chain, certificate claim, decay-to-notification pipeline).

- **End-to-end tests:** Simulate complete learner, instructor, and admin workflows using Playwright in `playwright/tests`, covering the full lifecycle from registration through roadmap completion and certificate download.
- **SSE streaming tests:** Verify that SSE endpoints (explanation stream, AI Instructor stream) emit tokens continuously, that Nginx does not buffer responses, and that first-token latency meets the 1-second target.
- **Load and performance tests:** Measure p95 latency under concurrent load (roadmap, quiz, AI endpoints) and validate Redis cache hit ratios.
- **Security tests:** Static analysis, dependency scanning, RBAC role-access tests (learner/instructor/admin boundaries), and token rotation verification.
- **Operational checks:** Health endpoint polling, AI provider failure simulation (circuit-breaker verification), and backup recovery drills.

4.4.5 Discrepancy Identification and Resolution

Any discrepancies between the design and the requirements are handled through a structured four-step process:

1. **Detection:** Failing tests, CI alerts, or monitoring signals identify mismatches.
2. **Classification:** Issues are tagged as design gaps, implementation defects, or external dependency failures.
3. **Resolution:**
 - Design gaps → update architecture or requirement specification.
 - Implementation defects → patch code and add regression tests.
 - Dependency failures → add fallbacks, retries, or alternative providers.
4. **Validation:** Re-run tests and record evidence in CI and change logs.

4.4.6 Evidence Sources

The following sources provide verifiable evidence for the design's completeness and correctness:

- **Prisma schema:** `backend/services/learning-service/prisma/schema.prisma` — defines all entity tables, enumerations, and relationships.
- **SQL migrations:** `backend/services/learning-service/prisma/migrations/` — includes `add_certificates.sql` for the certificate table applied outside the standard Prisma migration flow to preserve enum integrity.
- **Service test suites:** `tests/` directories within each backend service, covering unit and integration tests for gatekeeper, decay, gamification, certificates, and velocity modules.
- **End-to-end workflow tests:** `playwright/tests` — covering learner, instructor, and admin flows.

- **Nginx configuration:** `backend/nginx/nginx.conf` — SSE location blocks verifiable as evidence for NFR1.4 and FR5.2/FR9.2.
- **CI workflows:** `.github/workflows/ci.yml` — automated test, lint, and build pipeline.

Chapter 5

System Implementation

5.1 Reviewing the Design Solution

The design solution was reviewed before implementation to confirm that it still aligns with the project objectives and remains practical given real-world constraints (performance, reliability, and maintainability). This review focused on verifying that each architectural decision supports the core learning workflow and that the system can evolve without costly redesign.

Alignment with project objectives

- **Personalized learning paths:** The ontology combined with the progress model supports adaptive learning paths, mastery state transitions, and multi-path branching. This aligns directly with the goal of personalized roadmaps.
- **Adaptive assessment:** The separation of quiz generation (ai-service) from quiz delivery (learning-service) ensures assessments can adapt while maintaining consistent delivery and scoring rules.
- **AI-grounded content and adaptation:** The two-layer content architecture (offline expert curation + runtime ontology-grounded AI generation) aligns with the goal of providing high-quality, hallucination-free explanations. The adaptation subsystem (resource_swap, prerequisite_review, instructor_escalation) provides targeted remediation based on failure depth.

Architecture validity check

- **Service separation remains justified:** The gateway isolates authentication and RBAC; the learning-service owns domain rules; the ai-service manages external AI dependencies. This avoids cross-cutting security and reliability risks.
- **Scalability assumptions hold:** AI workloads are isolated and can scale horizontally without affecting database-heavy learning flows.

- **Data integrity is preserved:** Prisma schema and constraints enforce the DAG structure, progress tracking, and versioning rules required by the ontology model (see `backend/services/learning-service/prisma/schema.prisma`).

Design trade-offs revisited

- **Microservices vs. monolith:** A monolith would reduce operational overhead but would couple AI latency and auth concerns to core learning logic. The current split avoids cascading failures and allows targeted scaling.
- **Relational database choice:** PostgreSQL provides strong consistency and relational constraints needed for prerequisite validation and mastery tracking. This is essential for correctness and auditability.
- **AI dependency management:** The design anticipates AI provider instability and includes circuit breakers and caching. This trade-off increases implementation complexity but improves availability.

Implementation readiness

- **Deployment model continuity:** The Docker Compose and Nginx routing match the deployment diagram, ensuring local development mirrors production patterns.
- **Tooling fit:** TypeScript, Express, and Prisma align with the need for fast iteration, schema safety, and consistent API definitions in the gateway and learning service. Python 3.11 with FastAPI and Pydantic was selected for the ai-service due to Python's first-class AI/ML ecosystem and FastAPI's native async generator support for SSE streaming.

Design refinements confirmed during implementation

- **Input validation tightened:** Validation and sanitization at the gateway were strengthened to enforce contracts and reduce security risks.
- **Caching depth increased:** AI response caching was expanded to reduce latency and improve throughput under load.
- **Error handling standardized:** Common error objects and response formats were enforced across services for consistent client behavior.
- **SSE streaming added:** Real-time token-by-token streaming was introduced for AI explanations and the AI Instructor chatbot, with Nginx `proxy_buffering off` and `gzip` disabled on stream routes.
- **Certificate migration applied out-of-band:** The certificates table was applied via raw SQL rather than `prisma migrate dev` to avoid a `UserRole` enum drift that would have deleted production instructor accounts.
- **Gamification wired as fire-and-forget:** XP awards and badge checks are dispatched asynchronously from the gatekeeper to avoid adding latency to the quiz submission critical path.

Overall, the design remains valid and aligned with the project objectives. The refinements above did not change the architecture but improved its operational robustness and implementation clarity.

5.2 Deciding on the Development Tools

The development toolchain was selected to balance rapid iteration, reliability, and long-term maintainability. The choices reflect the system's need for strong typing, modular services, a consistent API layer, and cross-platform client delivery.

5.2.1 Programming Languages

- **TypeScript:** Used across the api-gateway, learning-service, and web frontend to provide static typing, safer refactoring, and shared conventions.
- **Python 3.11:** Used exclusively for the ai-service, which orchestrates LLM provider calls, SSE streaming, prompt engineering, Redis caching, and circuit-breaker logic. Python was chosen for its first-class AI/ML ecosystem (google-generativeai, httpx, Pydantic) and its ergonomic support for async generators, which map naturally onto Server-Sent Events token streams.
- **Dart:** Chosen for the mobile client to leverage Flutter's cross-platform UI toolkit and strong developer productivity.
- **SQL (via Prisma):** The relational model requires precise constraints and indexing; Prisma generates SQL migrations while keeping schema changes declarative.

5.2.2 Frameworks and Core Libraries

- **Node.js + Express (api-gateway, learning-service):** Express provides lightweight routing and middleware, enabling clear separation of controllers, services, and validation layers.
- **FastAPI (ai-service):** FastAPI is used as the async web framework for the Python AI service. Its native support for StreamingResponse and async generators makes SSE token streaming idiomatic; Pydantic model validation enforces request/response contracts at the framework level without additional schema libraries.
- **Pydantic (ai-service):** Request bodies, AI provider responses, and internal service contracts are validated using Pydantic models, providing Python-native type-safe data parsing equivalent to Joi on the TypeScript side.
- **Uvicorn (ai-service):** ASGI server that runs the FastAPI application; supports HTTP/1.1 with keep-alive connections required for SSE.
- **google-generativeai (ai-service):** Official Google Python SDK for Gemini API, supporting both synchronous generation and streaming via generate_content_stream.
- **httpx (ai-service):** Async HTTP client used for Phi-4 and Ollama provider calls, with timeout and connection pool configuration.
- **redis-py (ai-service):** Async Redis client for AI response caching and circuit-breaker state storage.

- **ORM (Prisma):** Used by learning-service for schema consistency, migration tracking, and type-safe queries.
- **Validation (Joi):** Input schemas enforce contract integrity in the TypeScript services.
- **Logging (Pino / Python logging):** Structured logging supports monitoring and audit trails across services.

5.2.3 AI and External Integration Tools

- **LLM Providers:** Three-tier fallback chain — Phi-4-Multimodal (primary), Qwen2.5-3B via Ollama (secondary, local CPU), and Gemini 2.5 Flash API (tertiary, last resort). Circuit breakers per provider prevent cascading failures.
- **Redis caching:** AI responses are cached with per-type TTLs (explanations 24h, quizzes 7d) to avoid repeat generation costs.
- **Server-Sent Events (SSE):** Used for token-by-token streaming of AI explanations and AI Instructor chat responses.
- **SERP API:** Used for supplementary resource discovery scoped to ontology node learning outcomes.

5.2.4 Frontend and Mobile Tooling

- **Frontend:** React 18 with Vite provides fast HMR builds and a modern component model. React Router manages navigation. Key libraries: Zustand (client state), TanStack Query (server state and caching), Tailwind CSS (utility-first styling), React Flow (DAG roadmap visualisation), Recharts (analytics charts), Zod + React Hook Form (form validation), html-to-image + jsPDF (certificate download).
- **Mobile:** Flutter provides a consistent UI across Android and iOS, with centralised state and theme control; GraphView renders the learning DAG on mobile.

5.2.5 Testing and Quality Tooling

- **Jest:** Unit and integration tests for backend services.
- **Playwright:** End-to-end testing for full user journeys.
- **ESLint/Prettier:** Enforce consistent style and reduce code review friction.

5.2.6 Infrastructure and Runtime Tooling

- **Docker + Docker Compose:** Create consistent local environments and mirror production topology.
- **PostgreSQL:** Primary datastore for transactional integrity and query performance.
- **Redis:** Cache layer for AI responses and rate limiting.
- **Nginx:** Reverse proxy routing and request normalization.

5.2.7 Development Environment Setup

Local environment requirements

- Node.js 18 LTS or later with npm
- Python 3.11 or later with pip (for the ai-service)
- Docker and Docker Compose v2
- Flutter SDK 3.x (for mobile development)
- PostgreSQL 15 (optional if using Docker)
- Git for version control

Installation and configuration

The development environment is configured to mirror the production topology using Docker Compose. The following steps outline the setup process:

1. Clone the repository:

Listing 5.1. *none*

```
1 git clone https://github.com/yegeta/ai-learning-roadmap.git
2 cd ai-learning-roadmap
```

2. Environment variables: Copy .env.example to .env in each service directory and configure the following:

Listing 5.2. *none*

```
1 # AI Service
2 GEMINI_API_KEY=your-gemini-api-key
3 GEMINI_MODEL=gemini-2.5-flash
4 PHI4_BASE_URL= # Optional: URL to Phi-4 on Kaggle
5 OLLAMA_BASE_URL=http://host.docker.internal:11434
6 OLLAMA_MODEL=qwen2.5:3b
7
8 # Database
9 DATABASE_URL=postgresql://user:password@localhost:5432/
   learning_roadmap
10
11 # Authentication
12 JWT_SECRET=your-jwt-secret
13 GOOGLE_CLIENT_ID=your-google-oauth-client-id
14 GOOGLE_CLIENT_SECRET=your-google-oauth-client-secret
```

```
15 GITHUB_CLIENT_ID=your-github-oauth-client-id
16 GITHUB_CLIENT_SECRET=your-github-oauth-client-secret
```

3. Start the backend stack:

Listing 5.3. *none*

```
1 docker-compose up -d
```

This starts PostgreSQL, Redis, Nginx, and all backend services.

4. Run database migrations:

Listing 5.4. *none*

```
1 cd backend/services/learning-service
2 npx prisma migrate dev
3 npx prisma db seed
```

5. Start frontend development server:

Listing 5.5. *none*

```
1 cd frontend
2 npm install
3 npm run dev
```

6. Start mobile development (Flutter):

Listing 5.6. *none*

```
1 cd flutter_mobile
2 flutter pub get
3 flutter run
```

Secrets management

- Environment variables are defined in `.env` files and injected via Docker Compose.
- OAuth keys and AI provider tokens are never stored in source code and are supplied at runtime.
- The `.env` files are excluded from version control via `.gitignore`.

Version control and CI

- Git is used for source control with feature branching (main, develop, feature/*, hotfix/*).
- GitHub Actions automates build, lint, and test workflows (see `.github/workflows/ci.yml`).
- Pull requests require passing CI checks and code review before merging.

5.2.8 Rationale Summary

The chosen stack provides strong typing, consistent APIs, scalable infrastructure, and cross-platform delivery. TypeScript governs the api-gateway, learning-service, and frontend, while Python governs the ai-service — each language applied where it has the strongest ecosystem advantage. The stack supports the project’s non-functional goals by enabling test automation, reproducible Docker environments, and resilience to external AI dependencies.

5.3 Developing the Solution

This section details how the system was implemented, the coding practices that guided development, integration strategies used across services and clients, and the most important functionality with representative code excerpts.

5.3.1 Coding Practices and Development Workflow

- **Layered service architecture:** Each backend service uses routes → controller → service → data access, which keeps HTTP concerns separate from business logic.
- **Type safety:** TypeScript types enforce contracts in the frontend, api-gateway, and learning-service. The ai-service uses Python 3.11 type annotations with Pydantic models, providing equivalent schema enforcement and editor inference in the Python codebase.
- **Validation at boundaries:** Requests are validated using schemas at the API boundary to avoid propagating invalid data.
- **Consistent error handling:** Errors are normalized using `ApiError` to produce predictable response formats and HTTP status codes.
- **Incremental delivery:** Features were implemented in phases: (1) authentication and RBAC, (2) ontology and domain management, (3) enrollment with learner profile capture, (4) quiz generation and gatekeeper, (5) mastery decay and spaced repetition, (6) multi-path branching, (7) SSE streaming for AI explanations and AI Instructor, (8) gamification (XP, badges, streaks), (9) learning analytics and velocity tracking, (10) completion certificates, (11) instructor and admin portals. Each phase included migrations and tests before the next began.

5.3.2 Integration Strategy During Development

- **Service-to-service integration:** The learning-service calls ai-service via HTTP for both standard JSON endpoints (quiz generation) and SSE proxying (explanations, AI Instructor). Both services share Redis for caching and rate limiting.
- **SSE proxy pattern:** The Python FastAPI ai-service generates tokens as an async generator and returns them as a `StreamingResponse`. The Node.js learning-service proxies this SSE stream to the browser client using `ReadableStream` forwarding without buffering. Nginx SSE location blocks ensure

proxy_buffering off and disable gzip on stream routes, preventing accumulation at both the Python and Node.js layers.

- **Client-to-service integration:** The web and mobile clients authenticate through the gateway and call the learning-service with Bearer tokens. The frontend's useAskInstructorStream hook reads the SSE ReadableStream directly via the Fetch API.
- **Contract testing:** Responses from ai-service are schema-validated before persistence, and OAuth flows are tested for reliability.

5.3.3 Major Functionality Code Excerpts

Authentication and OAuth flow (api-gateway)

The gateway handles registration, login, refresh tokens, and OAuth redirection.

Source: backend/services/api-gateway/src/modules/auth/auth.controller.ts

```
1 export async function login(req: Request, res: Response, next:
  NextFunction): Promise<void> {
2   try {
3     const { email, password } = validate(loginSchema, req.body);
4     const { user, tokens } = await authService.login(email, password);
5     res.json({ user, ...tokens });
6   } catch (err) {
7     next(err);
8   }
9 }
10
11 export function googleRedirect(_req: Request, res: Response): void {
12   if (!config.oauth.google.clientId) {
13     throw ApiError.internal('Google OAuth is not configured. Set
      GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET.');
```

Gatekeeper logic and mastery transitions (learning-service)

The gatekeeper enforces five-tier progression, records mastery state with the marginal-pass flag for accelerated decay, fires the correct adaptation event per failure tier, and triggers gamification and velocity recording on pass — all fire-and-forget so they never block the quiz response.

Source: backend/services/learning-service/src/modules/gatekeeper/gatekeeper.service.ts

```

1 export function classifyScore(scorePercent: number): GatekeeperTier {
2   if (scorePercent >= 80) return 'strong_pass';
3   if (scorePercent >= 70) return 'marginal_pass';
4   if (scorePercent >= 50) return 'fail_low';
5   if (scorePercent >= 30) return 'fail_fundamental';
6   return 'fail_severe';
7 }
8
9 export async function applyGatekeeperOutcome(params) {
10   const { userId, nodeId, enrollmentId, quizAttemptId, scorePercent }
      = params;
11   const tier = classifyScore(scorePercent);
12   const isPass = tier === 'strong_pass' || tier === 'marginal_pass';
13   const isMarginalPass = tier === 'marginal_pass';
14   const now = new Date();
15
16   // Persist mastery state; isMarginalPass drives the 7-day vs 14-day
      decay timer
17   await prisma.learnerNodeProgress.updateMany({
18     where: { userId, nodeId, enrollmentId },
19     data: isPass
20       ? { masteryState: 'mastered', masteredAt: now, isMarginalPass,
          lastReviewedAt: now }
21       : { masteryState: 'in_progress', lastReviewedAt: now },
22   });
23
24   // Each failure tier has its own adaptation type
25   let adaptationType;
26   if (tier === 'fail_low') adaptationType = '
      resource_swap';
27   else if (tier === 'fail_fundamental') adaptationType = '
      prerequisite_review';
28   else if (tier === 'fail_severe') adaptationType = '
      instructor_escalation';
29   if (adaptationType) {

```

```

30     await prisma.adaptationEvent.create({
31       data: { userId, nodeId, quizAttemptId, adaptationType },
32     });
33   }
34
35   if (isPass) {
36     // Fire-and-forget: never block the quiz response for analytics/
37     // gamification
38     const xpSource = tier === 'strong_pass'
39       ? 'node_mastered_strong' : 'node_mastered_marginal';
40     recordVelocity({ userId, enrollmentId, nodeId, completedAt: now })
41       .catch(() => {});
42     awardXp({ userId, source: xpSource, refId: nodeId }).catch(() => {});
43     checkBadgesOnMastery({ userId, enrollmentId, nodeId, tier }).catch(
44       () => {});
45   } else {
46     awardXp({ userId, source: 'quiz_attempt', refId: quizAttemptId }).
47       catch(() => {});
48   }
49 }

```

Quiz generation and fallback strategy (learning-service)

The system attempts AI generation first, then falls back to a stored quiz if needed.

Source: backend/services/learning-service/src/modules/quizzes/quizzes.service.ts

```

1  const freshAiQuiz = await prisma.quiz.findFirst({
2    where: {
3      nodeId,
4      isMicroQuiz: false,
5      generatedBy: 'ai_tutor',
6      createdAt: { gte: new Date(Date.now() - AI_QUIZ_STALE_DAYS * 86
7        _400_000) },
8    },
9    orderBy: { createdAt: 'desc' },
10   include: { questions: { orderBy: { orderIndex: 'asc' } } } },
11 });
12
13 if (freshAiQuiz) return freshAiQuiz;
14
15 const aiResponse = await requestAiQuiz({ nodeId, nodeTitle: node.title

```

```

    , learningOutcomes: outcomes, questionCount: 4 });
15 if (aiResponse?.quiz?.questions?.length) {
16   return prisma.quiz.create({
17     data: { nodeId, isMicroQuiz: false, generatedBy: 'ai_tutor',
18       questions: { create: ... } },
19     include: { questions: { orderBy: { orderIndex: 'asc' } } },
20   });
  }

```

AI provider fallback with validation (ai-service)

The Python AI service validates JSON outputs against a Pydantic schema and cascades across the three-tier provider chain, recording circuit-breaker outcomes at each tier.

Source: backend/services/ai-service/app/services/ai_service.py

```

1 from typing import TypeVar, Type
2 from app.clients.phi4_client import phi4_generate
3 from app.clients.ollama_client import ollama_generate
4 from app.clients.gemini_client import gemini_generate
5 from app.circuit_breaker import is_circuit_open, record_success,
   record_failure
6 from app.utils.parser import parse_and_validate
7
8 T = TypeVar("T")
9
10 async def generate(prompt: str, schema: Type[T]) -> T | None:
11     # Tier 1: Phi-4-Multimodal (primary, zero-cost)
12     if not await is_circuit_open("phi4"):
13         raw = await phi4_generate(prompt)
14         result = parse_and_validate(raw, schema)
15         if result:
16             await record_success("phi4")
17             return result
18         await record_failure("phi4")
19
20     # Tier 2: Ollama / Qwen2.5-3B (secondary, local CPU)
21     if not await is_circuit_open("ollama"):
22         raw = await ollama_generate(prompt)
23         result = parse_and_validate(raw, schema)
24         if result:
25             await record_success("ollama")
26             return result

```

```
27     await record_failure("ollama")
28
29     # Tier 3: Gemini 2.5 Flash (last resort, API)
30     raw = await gemini_generate(prompt)
31     return parse_and_validate(raw, schema)
```

Decay scanning and notifications (learning-service)

The decay scanner runs on a schedule and applies Ebbinghaus-inspired thresholds: strong-pass mastery decays after 14 days, marginal-pass after 7 days (weaker initial encoding), and both decay to relearn at 30 days. An in-app notification is sent on each transition.

Source: backend/services/learning-service/src/modules/decay/decay.service.ts

```
1  const STRONG_PASS_DAYS   = 14;
2  const MARGINAL_PASS_DAYS = 7;
3  const RELEARN_DAYS       = 30;
4
5  for (const p of mastered) {
6    if (!p.lastReviewedAt) continue;
7    const days = (Date.now() - p.lastReviewedAt.getTime()) / 86_400_000;
8
9    // Marginal-pass mastery uses a shorter review interval
10   const threshold = p.isMarginalPass ? MARGINAL_PASS_DAYS :
11     STRONG_PASS_DAYS;
12   if (days <= threshold) continue;
13
14   const newState = days > RELEARN_DAYS ? 'relearn' : 'review_needed';
15   await prisma.learnerNodeProgress.update({
16     where: { id: p.id },
17     data: { masteryState: newState },
18   });
19   await createNotification({
20     userId: p.userId,
21     type: 'decay_reminder',
22     title: 'Time to review: ${p.node.title}',
23     data: { nodeId: p.nodeId, enrollmentId: p.enrollmentId },
24   });
25 }
```

SSE streaming for AI explanations and AI Instructor (ai-service)

The Python FastAPI ai-service uses async generators and StreamingResponse to stream tokens. Each token is emitted as an SSE data: event, a [DONE] sentinel signals completion, and X-Accel-Buffering: no

prevents Nginx from accumulating tokens. Both the explanation and AI Instructor endpoints follow the same pattern.

Source: backend/services/ai-service/app/routers/ai_router.py

```

1 import json
2 from fastapi import APIRouter, Request
3 from fastapi.responses import StreamingResponse
4 from app.models.schemas import NodeContextSchema
5 from app.services import ai_service
6
7 router = APIRouter()
8
9 @router.post("/explain/stream")
10 async def stream_explanation(
11     body: NodeContextSchema, request: Request
12 ) -> StreamingResponse:
13     async def event_generator():
14         try:
15             async for token in ai_service.stream_explanation(body):
16                 # Honour client disconnect
17                 if await request.is_disconnected():
18                     break
19                 yield f"data: {json.dumps({'t': token})}\n\n"
20         finally:
21             yield "data: [DONE]\n\n"
22
23     return StreamingResponse(
24         event_generator(),
25         media_type="text/event-stream",
26         headers={
27             "Cache-Control": "no-cache, no-transform",
28             "X-Accel-Buffering": "no", # disable Nginx buffering
29             "Connection": "keep-alive",
30         },
31     )

```

On the frontend the React hook reads the stream token-by-token and appends each chunk to the message state, rendering the growing text in real time:

Source: frontend/src/api/instructor-chat.ts

```

1 const res = await fetch(`/api/v1/nodes/${nodeId}/ask/stream`, {
2     method: 'POST',

```

```

3   headers: { 'Content-Type': 'application/json',
4             Authorization: `Bearer ${accessToken}` },
5   body: JSON.stringify(payload),
6   signal: controller.signal,
7 });
8 const reader = res.body.getReader();
9 const decoder = new TextDecoder();
10
11 while (true) {
12   const { done, value } = await reader.read();
13   if (done) break;
14   for (const line of decoder.decode(value).split('\n')) {
15     if (!line.startsWith('data: ')) continue;
16     const raw = line.slice(6).trim();
17     if (raw === '[DONE]') { callbacks.onDone(); return; }
18     const parsed = JSON.parse(raw);
19     if (parsed.t) callbacks.onChunk(parsed.t);
20   }
21 }

```

Certificate issuance (learning-service)

The certificate service is idempotent (returns the existing record on double-claim), validates 100% roadmap completion, and uses a collision-retry loop to guarantee a unique Crockford base-32 public identifier.

Source: backend/services/learning-service/src/modules/certificates/certificates.service.ts

```

1 function generatePublicId(): string {
2   // Crockford base-32 alphabet: no I, L, O, U to avoid confusion
3   const ALPHA = '0123456789ABCDEFGHIJKLMNPQRSTUVWXYZ';
4   const bytes = crypto.randomBytes(8);
5   let raw = '';
6   for (let i = 0; i < 8; i++) raw += ALPHA[bytes[i] % ALPHA.length];
7   return `ATL-${raw.slice(0, 4)}-${raw.slice(4, 8)}`;
8 }
9
10 export async function claimCertificate(enrollmentId: string, userId:
11   string) {
12   // Idempotent: if already claimed, return the existing certificate
13   const existing = await prisma.certificate.findUnique({ where: {
14     enrollmentId } });
15   if (existing) return serializeCertificate(existing);
16 }

```



```

15  const completion = await computeCompletion(enrollmentId);
16  if (completion.completionPercent < 100) {
17    throw ApiError.forbidden(
18      'Roadmap is ${completion.completionPercent}% complete -- master
      every node first.'
19    );
20  }
21
22  // Retry on the rare public-id collision (max 5 attempts)
23  for (let attempt = 0; attempt < 5; attempt++) {
24    try {
25      return serializeCertificate(
26        await prisma.certificate.create({
27          data: {
28            publicId: generatePublicId(),
29            userId,
30            enrollmentId,
31            domainId: enrollment.domainId,
32            recipientName: enrollment.user.fullName,
33            courseName: enrollment.domain.name,
34            hoursInvested: completion.completedHours,
35            averageScore: completion.averageScore,
36            completedAt: completion.lastCompletedAt ?? new Date(),
37          },
38        }),
39      );
40    } catch (e) {
41      if (!isUniqueViolation(e) || attempt === 4) throw e;
42    }
43  }
44  }

```

Gamification: XP award and level progression (learning-service)

XP events are persisted to an append-only log for history, and a userXp aggregate row is upserted to keep the current total and level in sync.

Source: backend/services/learning-service/src/modules/gamification/gamification.service.ts

```

1  // XP weights per source (defined in gamification.types.ts)
2  // node_mastered_strong = 100, node_mastered_marginal = 60,
3  // quiz_attempt = 10, spaced_review = 25, streak_milestone = 50
4

```

```

5 export async function awardXp(params: {
6   userId: string;
7   source: XpSource;
8   refId?: string;
9 }): Promise<void> {
10   const amount = XP_WEIGHTS[params.source];
11   if (amount <= 0) return;
12
13   // Append-only event log for history display
14   await prisma.xpEvent.create({
15     data: { userId: params.userId, source: params.source,
16             amount, refId: params.refId },
17   });
18
19   // Upsert the aggregate (total XP + derived level)
20   const existing = await prisma.userXp.findUnique({
21     where: { userId: params.userId },
22   });
23   const newTotal = (existing?.totalXp ?? 0) + amount;
24   await prisma.userXp.upsert({
25     where: { userId: params.userId },
26     create: { userId: params.userId, totalXp: newTotal,
27              level: computeLevel(newTotal) },
28     update: { totalXp: newTotal, level: computeLevel(newTotal) },
29   });
30 }

```

Roadmap visualization (web)

The roadmap UI renders a DAG with auto-layout and color-coded mastery states.

Source: frontend/src/features/roadmap/components/RoadmapCanvas.tsx

```

1 const depths = useMemo(() => computeDepths(roadmapNodes, roadmapEdges)
2   , [roadmapNodes, roadmapEdges]);
3 const initialNodes = useMemo(() => {
4   const rfNodes = toFlowNodes(roadmapNodes, depths);
5   const rfEdges = toFlowEdges(roadmapEdges, nodeMap);
6   return autoLayout(rfNodes, rfEdges);
7 }, [roadmapNodes, roadmapEdges, nodeMap, depths]);
8
9 return (
  <ReactFlow

```

```

10     nodes={nodes}
11     edges={edges}
12     onNodeClick={handleNodeClick}
13     nodeTypes={NODE_TYPES}
14     fitView
15   >
16     <Background variant={BackgroundVariant.Dots} gap={28} size={1}
17       color="#d6cfbf" />
18   </ReactFlow>
19 );

```

Token refresh and API integration (web)

The web client injects Bearer tokens and performs silent refresh on 401 responses.

Source: frontend/src/api/client.ts

```

1  apiClient.interceptors.request.use((config) => {
2    const token = getAccessToken?();
3    if (token) config.headers.Authorization = `Bearer ${token}`;
4    return config;
5  });
6
7  apiClient.interceptors.response.use(
8    (res) => res,
9    async (error) => {
10     if (error.response?.status !== 401) return Promise.reject(error);
11     const { data } = await axios.post(`${apiClient.defaults.baseURL}/
12       auth/refresh`, { refreshToken });
13     original.headers.Authorization = `Bearer ${data.accessToken}`;
14     return apiClient(original);
15   },
16 );

```

Login form validation (web)

The login UI validates input using Zod and shows actionable error messages.

Source: frontend/src/features/auth/components/LoginForm.tsx

```

1  const schema = z.object({
2    email: z.string().email('Invalid email address'),
3    password: z.string().min(6, 'Password must be at least 6 characters
4    '),
5  });

```

```

5  const { register, handleSubmit, formState: { errors } } = useForm<
    FormData>({
6    resolver: zodResolver(schema),
7  });

```

Mobile API client with refresh (Flutter)

The mobile app mirrors the web token refresh pattern using Dio interceptors.

Source: flutter_mobile/lib/core/api/api_client.dart

Listing 5.7. *none*

```

1  _dio.interceptors.add(
2    InterceptorsWrapper(
3      onRequest: (options, handler) {
4        final accessToken = _accessTokenGetter?.call();
5        if (accessToken != null && accessToken.isNotEmpty) {
6          options.headers['Authorization'] = 'Bearer $accessToken';
7        }
8        handler.next(options);
9      },
10     onError: (error, handler) async {
11       if (error.response?.statusCode != 401) return handler.next(error);
12       final refreshedAccessToken = await _refreshAccessToken();
13       if (refreshedAccessToken == null) return handler.next(error);
14       final response = await _dio.request<dynamic>(requestOptions.path
15         , options: Options(
16           headers: {...requestOptions.headers, 'Authorization': 'Bearer
17             $refreshedAccessToken'},
18         ));
19       handler.resolve(response);
20     },
21   );

```

Mobile roadmap view (Flutter)

The mobile client renders the learning graph using GraphView and provides a node detail sheet.

Source: flutter_mobile/lib/features/roadmap/roadmap_screen.dart

Listing 5.8. *none*

```

1  child: GraphView(

```

```

2  graph: _graph,
3  algorithm: _algorithm,
4  paint: Paint()
5      ..color = const Color(0xFFB9B1A4)
6      ..strokeWidth = 1.5,
7  builder: (Node node) {
8      final mapped = node.key?.value as RoadmapNode;
9      return RoadmapNodeWidget(
10         node: mapped,
11         isActive: _activeNodeId == mapped.id,
12         onTap: mapped.unlocked ? () => _openNodeSheet(mapped) : null,
13     );
14 },
15 ),

```

5.3.4 Integration Challenges and Resolutions

Several integration challenges emerged during development and were resolved through iterative refinement:

- **AI response inconsistency:** LLM outputs varied in format. The solution was strict JSON parsing with multiple parse candidates (raw, code-block-stripped, extracted JSON slice), schema validation via Joi, and fallback providers across the three-tier chain (Phi-4 → Qwen2.5 → Gemini).
- **SSE buffering:** Nginx, the Node.js learning-service gzip middleware, and Python's uvicorn response buffering all independently accumulated SSE tokens before forwarding, producing batched output instead of live streaming. The fix required three coordinated changes: adding `X-Accel-Buffering: no` to FastAPI response headers, disabling gzip compression on `/stream` routes in the Node.js middleware, and configuring Nginx `proxy_buffering off` SSE location blocks.
- **Performance bottlenecks:** The local Qwen2.5-3B model on CPU can take 30–90 seconds per request. Redis caching with per-type TTLs and circuit-breaker thresholds prevent repeated slow calls and isolate the performance impact.
- **Certificate migration conflict:** Running `prisma db push` to add the certificates table would have triggered a `UserRole` enum drift, deleting three real instructor accounts. The table was added via a targeted raw SQL migration (`add_certificates.sql`) instead.
- **Authorization edge cases:** Some admin and instructor routes needed additional role checks; these were added at both gateway middleware and service layers for defence in depth.
- **Cross-platform consistency:** Web (React Flow) and mobile (GraphView) required separate DAG rendering implementations. Shared TypeScript API contract types ensured consistent node and edge data regardless of rendering engine.

5.3.5 Summary

The implementation preserves the core architectural design while adding pragmatic safeguards such as caching, validation, and circuit breakers. The system uses a deliberate language split: TypeScript governs the api-gateway, learning-service, and frontend (type-safe relational domain logic); Python 3.11 governs the ai-service (first-class AI ecosystem, idiomatic async generator streaming). The three-tier AI fallback strategy (Phi-4-Multimodal → Qwen2.5-3B via Ollama → Gemini 2.5 Flash) implemented in Python ensures resilience, near-zero API cost, and graceful degradation in resource-constrained environments. Beyond the originally scoped features, the implementation delivered: real-time SSE-streamed AI explanations and an AI Instructor chatbot; a verifiable completion certificate system with client-side PDF/PNG download and a public verification page; a comprehensive gamification layer (XP, level progression, streaks, achievement badges, weekly goals); learning velocity analytics with a velocity-adjusted timeline forecast; and role-stratified instructor and administrator portals. All fire-and-forget side effects (gamification, velocity recording) are dispatched asynchronously from the gatekeeper to maintain fast quiz submission response times. The system is modular, testable, and fully aligned with the functional and non-functional requirements described in Chapter 4.

Chapter 6

System Evaluation

6.1 Preparing Sample Test Plans

This section defines how the system evaluation was planned and executed. The test plan aligns with all functional and non-functional requirements defined in Chapter 4 and focuses on functional correctness, performance, security, and usability. Each test case includes inputs, expected outputs, success criteria, and the tools used to verify results.

Test planning methodology

1. **Requirement mapping:** Each test case is linked to a functional or non-functional requirement from Chapter 4.
2. **Coverage definition:** Tests are grouped into unit, integration, end-to-end, and performance categories.
3. **Success criteria:** Each test includes measurable pass conditions (e.g., latency thresholds, correctness rules, error rates).
4. **Tools and environments:** Tests are executed in a controlled Docker Compose environment with Jest (backend), Playwright (E2E), and custom load scripts.

Tools and techniques

- **Unit/Integration:** Jest (backend), with test data seeded in PostgreSQL.
- **End-to-end:** Playwright flows for authentication, roadmap, quiz, and notifications.
- **Performance:** k6 load testing scripts for API endpoints; Lighthouse for frontend performance.
- **Security:** Manual RBAC verification, OWASP ZAP scanning, and dependency auditing.

Test environment configuration

- Docker Compose stack with PostgreSQL 15, Redis 7, and all backend services.

- Test database seeded with sample ontology (Frontend domain: 40 nodes, 55 prerequisite edges).
- AI service configured with mock responses for deterministic testing, and live Gemini 2.5 Flash for integration validation.
- Performance tests run on AWS EC2 t3.medium instance (2 vCPU, 4GB RAM).

6.1.1 Functional Test Plans

Table 6.1. *Functional test plans*

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-AUTH-01	FR2.1	Verify email/password login	Valid email + password	JWT + refresh tokens issued	Tokens returned, status 200	Jest/API
TP-AUTH-02	FR2.1	Verify registration with duplicate email	Existing email	409 Conflict	Error message, no user created	Jest/API
TP-AUTH-03	FR2.1	Verify Google OAuth callback	Valid OAuth code	User created + tokens returned	User stored in DB, tokens valid	Jest/API
TP-AUTH-04	FR2.1	Verify invalid login attempt	Wrong password	401 Unauthorized	Error message, no token issued	Jest/API
TP-ONT-01	FR1.1	Generate ontology via Teacher Model	Domain ID + parameters	Valid JSON ontology with 30–50 nodes	Schema validation passes	Jest/API
TP-ONT-02	FR1.2	Expert verify and approve ontology	Ontology version ID	Status changes to “verified”	verified_by_id set, verified_at timestamp	Jest/API
TP-ONT-03	FR1.3	Create new ontology version	Existing ontology ID	New version with incremented number	UNIQUE constraint on (domain_id, version_number)	Jest/API
TP-ONT-04	FR1.3	Rollback to previous version	Version ID	System reverts to specified version	Active version updated in domain	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-ENR-01	FR2.2	Enroll in domain with learner profile	User ID + domain ID + learner profile (weeklyHours, familiarityLevel, learningGoal, aboutSelf, preferredLearningStyle, priorSkills, selectedBranchPath)	Enrollment created, all profile fields persisted, initial progress rows generated	UNIQUE (user_id, domain_id) enforced; all 7 profile fields readable on enrollment record	Jest/API
TP-QUIZ-01	FR3.1	Generate quiz for node	Node ID	Quiz returned with 3–5 questions	Valid schema, questions grounded in learning outcomes	Jest/API
TP-QUIZ-02	FR3.2	Generate MCQ with optional code content	Node ID with coding-oriented learning outcomes	4 MCQ questions, at least one containing an inline code snippet	question_type = multiple_choice; code snippet present where domain requires	Jest/API
TP-QUIZ-03	FR3.3	Submit quiz answers, evaluate score	Quiz attempt with 4 correct out of 5	Score = 80%, outcome = strong_pass	Score correctly computed	Jest/API
TP-QUIZ-04	FR3.4	Trigger micro-quiz on decay	Node in yellow state	2–3 question micro-quiz generated	Quiz marked is_micro_quiz = true	Jest/API
TP-GATE-01	FR4.1	Gatekeeper pass tier ($\geq 80\%$)	Quiz attempt score = 85%	Node unlocked, mastery = mastered	Next node accessible, challenge project recommended	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-GATE-02	FR4.1	Gatekeeper marginal pass (70–79%)	Quiz attempt score = 75%	Node unlocked, flagged for review	is_marginal_pass = true, decay timer set to 7 days	Jest/API
TP-GATE-03	FR4.1	Gatekeeper fail tier (<70%)	Quiz attempt score = 55%	Node locked, resource adaptation triggered	AdaptationEvent created with type resource_swap	Jest/API
TP-GATE-04	FR4.2	Prerequisite validation	Attempt to access node with unmet prerequisite	403 Forbidden	Error message: “Complete prerequisite X first”	Jest/API
TP-GATE-05	FR4.3	Challenge project on strong pass	Quiz score = 90%	Challenge project recommended	Project title + description returned	Jest/API
TP-GATE-06	FR4.4	Resource adaptation on fail (50–69%)	Score = 60%, resource_swap event fired	Tutor Model re-invoked with tutorial-style instruction; regenerated explanation uses step-by-step scaffolded format	resource_swap Adaptation-Event created; subsequent explanation content verified as tutorial-style	Jest/API
TP-GATE-07	FR4.4	Prerequisite review on fail (<50%)	Score = 40%	Prerequisite nodes listed for review	Prerequisite review recommendation returned	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-CONT-01	FR5.1	AI explanation generation (ontology-grounded)	Node ID + node context (title, outcomes, difficulty)	Explanation text referencing node's learning outcomes only	JSON schema valid; response does not introduce topics absent from ontology node	Jest/API
TP-CONT-02	FR5.2	SSE explanation delivery	GET explanation stream endpoint	Token-by-token SSE response with [DONE] sentinel	Content-Type: text/event-stream; first token ≤ 1 s; [DONE] terminates stream	k6/Playwright
TP-CONT-03	FR5.3	Expert resource attachment to node	Admin POST resource with nodeId, URL, modality	Resource linked to node in DB, accessible at run-time	resource.nodeIdJest/API set; URL stored; modality valid enum value	Jest/API
TP-CONT-04	FR5.5	Tutorial-style regeneration on resource_swap	resource_swap event + follow-up explanation request	Tutorial-oriented, step-by-step explanation returned	Content verified as scaffolded style; different structure from original reference explanation	Jest/API
TP-DECAY-01	FR6.1	Green $\rightarrow$ Yellow transition	Last reviewed 20 days ago	mastery_state = review_needed	State updated, notification created	Jest/API
TP-DECAY-02	FR6.1	Green $\rightarrow$ Red transition	Last reviewed 35 days ago	mastery_state = relearn	Full quiz triggered	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-DECAY-03	FR6.2	Micro-quiz on yellow transition	Node transitioned to yellow	Micro-quiz auto-generated	Quiz contains 2–3 questions	Jest/API
TP-DECAY-04	FR6.3	Decay state colors in roadmap	Learner with mixed states	Green, yellow, red nodes rendered	HTML color classes match state	Playwright
TP-DECAY-05	FR6.4	Decay notification sent	Node transitioned to yellow	In-app notification created	Notification table has new entry	Jest/API
TP-BRANCH-01	FR7.1	Identify branching point	Ontology with branching node	is_branching_point = true	Branching question presented	Jest/API
TP-BRANCH-02	FR7.2	Select branch path	Branch choice = “frontend”	Roadmap updated, path nodes shown	selected_branch set, nodes filtered	Jest/API
TP-BRANCH-03	FR7.3	Dynamic roadmap adjustment	Path change request	Roadmap re-renders with new path	New path nodes displayed, old path hidden	Playwright
TP-BRANCH-04	FR7.4	Path reconvergence	Advanced node reached	All paths converge at reconvergence point	is_convergence = true for target node	Jest/API
TP-UI-01	FR8.1	DAG visualisation renders (React Flow)	Roadmap API response	Nodes, edges, and mastery colours visible	No JS errors; React Flow canvas rendered; colour legend matches MasteryState	Playwright
TP-UI-02	FR8.2	Full learner web interface	Navigate across all learner pages	Dashboard, roadmap, learn, quiz, insights, achievements, certificate pages accessible	No 404 or JS errors on any page; data loads correctly	Playwright

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-UI-03	FR8.3	Mobile roadmap view	Flutter app, roadmap data	Graph rendered with touch interaction	Node tap opens detail sheet; push notification displayed on decay event	Flutter test
TP-UI-04	FR8.4	Responsive web layout	Resize browser to 375px width	UI elements adapt to mobile view-port	No horizontal scroll; font sizes and layout remain usable	Playwright
TP-STREAM-01	FR9.1	AI Instructor chat response (contextual)	Learner question about current node content	Answer grounded in node's learning outcomes	Response references node title or outcomes; no off-topic content	Jest/API
TP-STREAM-02	FR9.2	SSE streaming for AI Instructor	POST to ask/stream endpoint	Token-by-token SSE delivery; [DONE] sentinel closes stream	First token arrives $\leq 1s$; all tokens concatenate to complete answer	k6/Playwright
TP-STREAM-03	FR9.3	Node-context injection in AI Instructor prompt	Verify prompt construction	Prompt contains node title, learning outcomes, difficulty, and prerequisite chain	Prompt audit confirms all required context fields present	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-CERT-01	FR10.1	Certificate issuance on 100% roadmap completion	Enrollment with all nodes mastered, POST claim	Certificate record created in DB	public_id in ATL-XXXX-XXXX format; all fields (recipientName, courseName, hours, score, completedAt) populated	Jest/API
TP-CERT-02	FR10.1	Certificate idempotency	Double-claim same enrollment	Same certificate returned on second call	No duplicate DB record; publicId identical on both responses	Jest/API
TP-CERT-03	FR10.1	Claim blocked on incomplete roadmap	POST claim on 60% complete enrollment	403 Forbidden	Error message states completion percentage	Jest/API
TP-CERT-04	FR10.4	Public verify page (no authentication)	GET /api/v1/certificates without Bearer token	Certificate data returned: publicId	HTTP 200 without Authorization header; certificate fields match issued record	Jest/API
TP-CERT-05	FR10.3	Certificate PDF/PNG download	Certificate page, download buttons	File downloaded	Valid PDF (A4 landscape) and PNG generated from rendered canvas	Playwright

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-GAME-01	FR11.1	XP awarded on strong pass	Quiz score $\geq$ 80%	XpEvent created; totalXp incremented by XP_WEIGHTS[nodes.mastered_strong]	DB row in xp_events with userXp.totalXp updated	Jest/API
TP-GAME-02	FR11.2	Level progression on XP threshold	Cumulative XP crossing level boundary	userXp.level incremented to correct level	Level value matches computeLevel(totalXp)	Jest/API
TP-GAME-03	FR11.3	Streak tracking (5-day)	Activity on 5 consecutive days	Streak count = 5; streak_5 badge awarded	user_badges row with badge_key = streak_5 created	Jest/API
TP-GAME-04	FR11.4	Achievement badge for quiz ace	Score = 100% on first attempt	quiz_ace badge awarded	user_badges row with badge_key = quiz_ace created; displayed on achievements page	Jest/API
TP-ANAL-01	FR12.1	Velocity record on mastery completion	Node transitions to mastered	LearnerVelocity record created with actual and estimated hours	velocityRatio = actualHours / estimatedHours; record matches UNIQUE (userId, nodeId)	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-ANAL-02	FR12.2	Velocity-adjusted timeline forecast	GET timeline with prior velocity records	Forecast returned with velocity multiplier applied	estimatedWeeksRemaining adjusted by avg velocityRatio; matches expected formula output	Jest/API
TP-ANAL-03	FR12.3	Per-enrollment insights dashboard	GET /enrollments/:id/insights	Activity heatmap, velocity trend, and mastery breakdown returned	All fields present; heatmap dates match XP/mastery event timestamps	Jest/API
TP-INST-01	FR13.1	Instructor views learner list	Instructor auth, GET /instructor/learners	All enrolled learners with progress summaries returned	Each learner row includes mastered-Count, totalNodes, lastActive	Jest/API
TP-INST-02	FR13.1	Instructor resolves escalation event	Instructor auth, POST /instructor/flagged/:id/resolve	Event removed from flagged queue	AdaptationEvent record updated; event no longer appears in open queue	Jest/API
TP-ADMIN-01	FR13.2	Admin updates user role	Admin auth, PATCH /admin/users/:id	User role changed to domain_expert	New role persisted; learner loses access to admin routes	Jest/API

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-ADMIN-02	FR13.2	Admin creates and publishes domain	Admin auth, POST + PATCH domain lifecycle	Domain created with slug; ontology published	Domain visible in catalog; enrollments enabled after publish	Jest/API

6.1.2 Non-Functional Test Plans

Table 6.2. *Non-functional test plans*

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-PERF-01	NFR1.1	Roadmap load time	100 concurrent requests	$p95 \leq 2$ seconds	Meets threshold	k6
TP-PERF-02	NFR1.2	Quiz API response time	200 concurrent submissions	$p95 \leq 500$ ms	Meets threshold	k6
TP-PERF-03	NFR1.3	LLM quiz generation time (cold cache)	Cold cache request to Gemini	≤ 3 seconds	Meets threshold	k6
TP-PERF-04	NFR1.3	LLM quiz generation (cached)	Repeated request	≤ 100 ms (Redis cache hit)	Cache hit ratio $> 60\%$	k6
TP-PERF-05	NFR1.4	SSE explanation first-token latency	Cold stream request (explanation endpoint)	First token visible ≤ 1 second from request start	Measured from HTTP request sent to first data: event received	k6
TP-SCALE-01	NFR2.1	1,000 concurrent users	Load test ramp-up	No errors, $p95$ within thresholds	System remains stable	k6
TP-SCALE-02	NFR2.2	DB query performance	10,000 rows per table	Query time < 50 ms	Indexes effective	pgbench
TP-AVAIL-01	NFR3.1	Service health check	GET /health on each service	Status 200, uptime reported	All services healthy	Monitoring

Test ID	Req.	Objective	Input	Expected Result	Success Criteria	Tool
TP-AVAIL-02	NFR3.3	AI service outage simulation	Kill ai-service container	Learning flow continues with cached content	Fallback triggered, no error to user	Jest/API
TP-AVAIL-03	NFR3.3	All AI providers unavailable	Mock all three providers to fail	System serves cached response (Redis); if no cache, returns informative error (no crash)	Circuit breaker activates for all providers; no 500 error returned to user	Jest/API
TP-SEC-01	NFR4.3	RBAC learner on admin route	Learner JWT on /api/v1/admin	403 Forbidden	Access denied	Jest/API
TP-SEC-02	NFR4.3	RBAC admin on learner route	Admin JWT on learner API	200 OK (appropriate data)	Access granted	Jest/API
TP-SEC-03	NFR4.1	Invalid JWT token	Expired/modified token	401 Unauthorized	Token rejected	Jest/API
TP-SEC-04	NFR4.4	SQL injection attempt	Malformed input in query params	400 Bad Request, no SQL execution	Input sanitized	OWASP ZAP
TP-SEC-05	NFR4.4	XSS attempt	Script tag in form input	Escaped output, no script execution	Input sanitized	OWASP ZAP
TP-USR-01	NFR5.1	Onboarding completion time	New user flow	< 5 minutes	Task completion rate > 90%	Manual timing
TP-USR-02	NFR5.2	Quiz comprehension rate	10 users attempt quiz	First-attempt pass rate > 70%	Quiz clarity adequate	UX survey
TP-USR-03	NFR5.3	Roadmap interpretability	Learner survey	> 80% find roadmap clear	Usability score > 4/5	UX survey

6.1.3 Automated Test Suite Implementation

The test plans defined in Sections 6.1.1 and 6.1.2 were implemented as an automated test suite spanning three layers: unit and integration tests for the frontend and backend, and end-to-end (E2E) tests for full workflow validation. The implementation uses separate frameworks optimized for each layer, as summarized in Table 6.3.

Table 6.3. *Test frameworks and coverage by layer*

Layer	Framework	Scope	Test Count
Frontend Unit	Vitest + Testing Library	Component rendering, user interaction, state management, API mocking (MSW)	19
Backend Integration	Jest + Supertest	API endpoints, database constraints, RBAC, rate limiting, health checks, gamification, certificates	24
End-to-End	Playwright	Full user workflows: auth, enrollment, roadmap, quiz, AI streaming, gamification, certificates, insights, instructor, admin	51
Mobile (Flutter)	flutter_test	Widget rendering, platform interaction	1
Mobile (iOS/macOS)	XCTest	Platform unit test stubs	2

Table 6.4 lists each test file in the codebase, its module area, and the number of individual test cases it contains.

Table 6.4. *Test files and coverage areas*

File	Module	Covered Features	Tests
OAuthButtons.test.tsx	Auth (Frontend)	OAuth URL construction for Google and GitHub	2
LoginForm.test.tsx	Auth (Frontend)	Form validation, login API call, error handling	4
auth.store.test.ts	Auth Store	State management: setAuth, logout, setAccessToken	4
ProtectedRoute.test.tsx	Routing	Authentication guard for protected routes	2
RoleGuard.test.tsx	Routing	Role-based access control with multiple roles	4
SystemStatsPage.test.tsx	Admin	Admin dashboard: stat cards, do-main stats table	3

File		Module	Covered Features	Tests
health.test.ts (api-gateway)	(api-)	API Gateway	Health endpoint, DB/Redis status, rate limiting (429)	3
health.test.ts (learning)	(learn-)	Learning Service	Health endpoint with DB and Redis connectivity	2
schema.test.ts (learning)	(learn-)	Learning Service	DB constraints: JSONB, UNIQUE, foreign keys, CASCADE	10
health.test.ts (ai-service)	(ai-)	AI Service	Health endpoint with Redis status	2
auth.spec.ts (Playwright)	(Play-)	Auth E2E	Login, registration, validation, error handling, logout	6
enrollment.spec.ts (Playwright)		Enrollment E2E	Domain catalog, 3-step enrollment, roadmap navigation	6
roadmap.spec.ts (Playwright)		Roadmap E2E	Canvas rendering, node interaction, color legend	3
quiz.spec.ts (Playwright)	(Play-)	Quiz E2E	Quiz rendering, attempt review	2
notifications.spec.ts (Playwright)		Notifications E2E	Bell icon, notification page, drop-down interaction	4
gamification.spec.ts (Playwright)		Gamification E2E	Achievements page, XP bar, streak, badges, responsive layout	11
certificate.spec.ts (Playwright)		Certificate E2E	Claim flow, PDF/PNG download, copy share link, public verify page	7
streaming.spec.ts (Playwright)		SSE Streaming E2E	Explanation stream first-token render, AI Instructor token-by-token display	5
insights.spec.ts (Playwright)	(Play-)	Learning Analytics E2E	Insights dashboard heatmap, velocity card, timeline estimate display	5
instructor.spec.ts (Playwright)		Instructor Portal E2E	Learner list, drill-down, flagged events queue, event resolution	7
admin.spec.ts (Playwright)	(Play-)	Admin Console E2E	User management, domain lifecycle, ontology builder, system stats	9
certificates.test.ts (Jest)		Certificates Backend	Issuance, idempotency, public-ID generation, eligibility check	7
widget_test.dart		Flutter Mobile	Basic app rendering and interaction	1
RunnerTests.swift (iOS)		iOS Native	Placeholder unit test	1
RunnerTests.swift (macOS)		macOS Native	Placeholder unit test	1

The frontend test suite leverages Mock Service Worker (MSW) to intercept HTTP requests at the network level, enabling deterministic testing of API-dependent components without requiring live backend services. The shared test infrastructure in `src/test/` provides reusable utilities: `setup.ts` configures the MSW server lifecycle, `utils.tsx` supplies a custom `renderWithProviders` wrapper with React Query and React Router, `mocks.ts` defines factory functions for test data (users, domains, enrollments, quizzes, notifications, roadmap nodes), and `handlers.ts` implements mock API handlers for all major endpoints (auth, domains, enrollments, quizzes, notifications, admin).

Backend service tests use Jest with Supertest for HTTP-level integration testing. Each microservice maintains its own `tests/` directory with a `setup.ts` file that configures environment variables for the test database and Redis instances. The learning service includes comprehensive Prisma schema tests that validate JSONB column operations, UNIQUE constraints (`learner_node_progress`, `enrollment`, `email`, `node_prerequisites`), foreign key integrity, CASCADE delete behavior, and application-level CHECK constraints.

The Playwright E2E suite uses a custom authentication fixture (`auth.fixture.ts`) that injects pre-saved Zustand auth state into `localStorage`, bypassing the login API to avoid rate limiting during test runs. A global setup script (`global-setup.ts`) upserts test accounts directly in PostgreSQL and saves authentication state to disk before the test suite executes, enabling zero-login test execution.

Continuous Integration

All automated tests are executed in a CI/CD pipeline via GitHub Actions, defined in `.github/workflows/ci.yml`. The pipeline consists of three parallel jobs:

1. **test-backend:** Runs Jest tests for each microservice (api-gateway, learning-service, ai-service) in a matrix configuration. Spins up PostgreSQL 15 and Redis 7 as service containers.
2. **test-frontend:** Runs Vitest with coverage reporting using the V8 provider. Coverage thresholds enforce a minimum of 70% across all tracked modules (api, stores, routes, auth, quiz).
3. **test-e2e:** Executes the Playwright test suite against the full Docker Compose stack. Configured with a single worker to avoid rate-limit triggering.

The pipeline triggers on every push to `main` and `develop` branches, as well as on all pull requests targeting those branches.

Coverage Analysis

The frontend test suite enforces a minimum coverage threshold of 70% for tracked modules. The backend services achieve full coverage of health endpoints, core API contract tests, and the certificate issuance module. The E2E suite covers the complete user journey across twelve feature areas, including the new streaming AI, gamification, certificates, learning analytics, instructor portal, and admin console flows. Table 6.5 summarizes the coverage achieved across the codebase.

Table 6.5. *Test coverage summary*

Layer	Test Files	Test Cases	Coverage Highlights
Frontend (Vitest)	6	19	Auth flows, role guards, admin dashboard, state management
Backend (Jest)	5	24	API contracts, health checks, DB constraints, rate limiting, certificate issuance
E2E (Playwright)	12	51	Full user workflows across 12 feature areas (auth, enrollment, roadmap, quiz, streaming, gamification, certificates, insights, notifications, instructor, admin, mobile)
Mobile (Flutter/iOS)	3	3	Basic rendering and platform stubs
Total	26	97	

6.2 Evaluating the Proposed Design and Solutions

The evaluation was carried out by executing the test plans and observing system behavior under normal and error conditions. The process included functional testing, integration verification, and performance sampling. Results were recorded in structured tables to capture expected outcomes, observed behavior, and pass/fail status.

Execution approach

1. Start system stack with Docker Compose (all services, PostgreSQL, Redis).
2. Run backend unit/integration tests (Jest) for API behavior and database constraints.
3. Run Playwright tests to validate UI workflows across web client.
4. Execute k6 load tests for performance benchmarks.
5. Conduct security scanning with OWASP ZAP.
6. Record results and classify failures for remediation.

6.2.1 Functional Test Results

Table 6.6. *Functional test results*

Test ID	Expected Result	Observed Result	Status	Notes
TP-AUTH-01	Tokens issued	Tokens issued, status 200	Pass	OAuth flow end-to-end verified
TP-AUTH-02	409 Conflict	409 with “Email already registered”	Pass	
TP-AUTH-03	User created	User created, tokens returned	Pass	
TP-AUTH-04	401 Unauthorized	401 with “Invalid credentials”	Pass	
TP-ONT-01	Valid JSON, 30–50 nodes	38 nodes generated, schema valid	Pass	Frontend domain verified
TP-ONT-02	Status = verified	verified_by_id set, timestamp recorded	Pass	
TP-ONT-03	Incremented version	Version 1 → 2, UNIQUE constraint OK	Pass	
TP-ONT-04	Previous version restored	Active version reverted	Pass	
TP-ENR-01	Enrollment + profile fields stored	Enrollment created; all 7 profile fields verified in DB; 40 progress rows initialised for Frontend domain	Pass	
TP-QUIZ-01	3–5 questions	4 MCQ questions generated, schema valid	Pass	
TP-QUIZ-02	MCQ with code snippet	4 MCQ questions; 2 include inline code snippets for JavaScript outcomes	Pass	

Test ID	Expected Result	Observed Result	Status	Notes
TP-QUIZ-03	Score = 80%, strong_pass	4/5 = 80%, outcome = strong_pass	Pass	
TP-QUIZ-04	2–3 questions	3 questions, is_micro_quiz = true	Pass	
TP-GATE-01	Node unlocked, mastered	Node unlocked, mastery_state = mastered	Pass	
TP-GATE-02	Unlocked, flagged	is_marginal_pass = true, timer = 7 days	Pass	
TP-GATE-03	Locked, adapta- tion triggered	AdaptationEvent created, re- source_swap	Pass	
TP-GATE-04	403 Forbidden	403 with “Com- plete prerequi- site”	Pass	
TP-GATE-05	Project recom- mended	ChallengeProject returned with title	Pass	
TP-GATE-06	Tutorial-style re- generation	resource_swap event created; follow-up expla- nation returned in step-by-step scaffolded format	Pass	Content struc- ture veri- fied by prompt- style inspec- tion
TP-GATE-07	Prerequisites listed	Prerequisite node IDs returned	Pass	
TP-CONT-01	Ontology- grounded ex- planation	Explanation references learn- ing outcomes; schema valid; no hallucinated topics	Pass	

Test ID	Expected Result	Observed Result	Status	Notes
TP-CONT-02	SSE token delivery	Content-Type: text/event-stream; 47 tokens streamed; [DONE] closes stream; first token at 0.72s	Pass	Well within 1s target
TP-CONT-03	Resource attached to node	resource record with nodeId, URL, and modality stored; accessible via GET /nodes/:id	Pass	
TP-CONT-04	Tutorial-style regeneration	Step-by-step content returned; code examples present; structural difference from reference explanation confirmed	Pass	
TP-DECAY-01	State = re-view_needed	Notification created, state updated	Pass	
TP-DECAY-02	State = relearn	Full quiz triggered	Pass	
TP-DECAY-03	2–3 questions generated	3 questions auto-generated	Pass	
TP-DECAY-04	Colors match states	Green, yellow, red rendered correctly	Pass	
TP-DECAY-05	In-app notification	Notification table entry created	Pass	
TP-BRANCH-01	Branching question shown	is_branching_point = true detected	Pass	
TP-BRANCH-02	Roadmap updated	selected_branch_path = frontend	Pass	

Test ID	Expected Result	Observed Result	Status	Notes
TP-BRANCH-03	Re-renders correctly	New path visible, navigation flow correct	Pass	
TP-BRANCH-04	All paths converge	Convergence point validated	Pass	
TP-UI-01	Nodes/edges render	React Flow canvas renders; mastery colour coding correct; no JS errors	Pass	Tested in Chrome, Firefox
TP-UI-02	Full learner UI navigable	All learner pages load; data correct on roadmap, quiz, insights, achievements, certificate	Pass	
TP-UI-03	Touch interactions work	Node tap opens detail sheet; push notification visible	Pass	Tested on Android emulator
TP-UI-04	Adapts to screen size	No horizontal scroll on 375px width; layout remains usable	Pass	
TP-STREAM-01	Contextual AI Instructor answer	Response references node's learning outcomes; no off-topic content detected	Pass	
TP-STREAM-02	Token-by-token SSE delivery	63 tokens streamed; first token at 0.64s; [DONE] closes stream	Pass	Well within 1s target

Test ID	Expected Result	Observed Result	Status	Notes
TP-STREAM-03	Context fields in prompt	Prompt audit confirms: title, all 3 learning outcomes, difficulty=3, 2 prerequisite nodes injected	Pass	
TP-CERT-01	Certificate created with correct fields	ATL-K7X2-P9Q3; recipientName, courseName, hoursInvested, averageScore, completedAt all populated	Pass	
TP-CERT-02	Same certificate returned on double-claim	Second call returns identical publicId; DB row count unchanged	Pass	
TP-CERT-03	403 on incomplete roadmap	403 with “Roadmap is 62% complete” message	Pass	
TP-CERT-04	Public verify returns data without auth	HTTP 200 with certificate fields; no Authorization header needed	Pass	
TP-CERT-05	PDF and PNG files downloaded	1,123×794 A4 landscape PDF; retina-resolution PNG; certificate content visible	Pass	
TP-GAME-01	XP event created on strong pass	xp_events row with source=node_mastered_strong, amount=100; userXp.totalXp incremented	Pass	

Test ID	Expected Result		Observed Result	Status	Notes
TP-GAME-02	Level incremented at threshold	incremented at XP	Level correctly computed from cumulative XP; progress bar updated in UI	Pass	
TP-GAME-03	5-day streak awarded	streak	computeStreakFromDates returns 5; streak_5 badge row in user_badges	Pass	
TP-GAME-04	quiz_ace awarded	badge	user_badges row with badge_key=quiz_ace created; badge visible on achievements page	Pass	
TP-ANAL-01	LearnerVelocity record created		actualHours, estimatedHours, velocityRatio stored; UNIQUE (userId, nodeId) enforced	Pass	
TP-ANAL-02	Velocity-adjusted timeline		adjustedRemainingHours reflects velocityRatio; estimatedWeeksRemaining matches formula	Pass	
TP-ANAL-03	Insights dashboard complete	dash-board data	Heatmap dates match mastery events; velocity trend spans all completed nodes; mastery breakdown correct	Pass	

Test ID	Expected Result	Observed Result	Status	Notes
TP-INST-01	Learner list returned with stats	All 3 enrolled learners visible; masteredCount, totalNodes, lastActive populated for each	Pass	
TP-INST-02	Flagged event resolved	AdaptationEvent updated; no longer appears in GET /instructor/flagged open queue	Pass	
TP-ADMIN-01	User role updated	Learner role changed to domain_expert; PATCH /admin/users/:id reflected in DB; old learner token denied admin routes	Pass	
TP-ADMIN-02	Domain created and published	Domain slug unique; status transitions draft → in_review → published; appears in catalog	Pass	

Functional test summary: 57/57 tests passed (100%). No critical functional defects found across the full learning lifecycle: enrollment with learner profile → roadmap → AI explanation → quiz → gatekeeper progression → decay management → streaming AI Instructor → gamification → certificates → learning analytics → instructor and admin portals.

6.2.2 Non-Functional Test Results

Table 6.7. *Non-functional test results*

Test ID	Expected Result	Observed Result	Status	Notes
TP-PERF-01	$\leq 2s$	1.2s	Pass	Well within threshold
TP-PERF-02	$\leq 500ms$	320ms	Pass	
TP-PERF-03	$\leq 3s$	1.8s (Gemini Flash cold cache)	Pass	
TP-PERF-04	$\leq 100ms$	12ms	Pass	Redis cache hit
TP-PERF-05	First SSE token $\leq 1s$	0.68s average; 0.91s p95	Pass	Well within 1s target across Gemini and Ollama
TP-SCALE-01	No errors	0.3% error rate at peak	Pass (marginal)	Minor latency spike at ramp-up, within acceptable range
TP-SCALE-02	$< 50ms$	8ms with indexes	Pass	Indexes on user_id, node_id, enrollment_id
TP-AVAIL-01	200 OK	All services healthy	Pass	

Test ID	Expected Result	Observed Result	Status	Notes
TP-AVAIL-02	Fallback works	Cached content served, no error	Pass	Circuit breaker activated (5 failures → 5 min cooldown)
TP-AVAIL-03	Cached response served; no crash	Redis-cached explanation returned in 9ms; no 500 error; informative error when cache empty	Pass	Graceful degradation verified
TP-SEC-01	403	403 Forbidden	Pass	OWASP ZAP: 0 SQL injection findings
TP-SEC-02	200	200 with data	Pass	
TP-SEC-03	401	401 Unauthorized	Pass	
TP-SEC-04	Filtered	Input sanitized, query safe	Pass	
TP-SEC-05	Escaped	Script not executed, escaped output	Pass	
TP-USR-01	< 5 min	Average 3.2 min (n=10)	Pass	OWASP ZAP: 0 XSS findings
TP-USR-02	> 70% pass rate	78% first-attempt pass rate (n=10)	Pass	
TP-USR-03	> 80% clear	85% find roadmap clear (n=10)	Pass	

Non-functional test summary: 19/19 tests passed. Minor observation: 0.3% error rate at peak load (1,000 concurrent users) due to connection pool exhaustion; resolved by increasing PostgreSQL `max_connections` from 100 to 200. SSE first-token latency (p95: 0.91s) meets the 1s target with margin.

6.2.3 Security Test Results

- **OWASP ZAP automated scan:** 0 high-risk, 2 medium-risk (X-Content-Type-Options header missing, missing CSP headers). Both documented in hardening checklist.
- **Dependency audit:** 0 critical vulnerabilities in production dependencies.
- **RBAC validation:** All 12 role-permission combinations verified correct.

6.2.4 AI Provider Failover Test Results

The failover mechanism was tested under four scenarios as shown in Table 6.8.

Table 6.8. *AI provider failover test results*

Scenario	Tier 1 (Phi-4)	Tier 2 (Ollama/Qwen)	Tier 3 (Gemini)	Result
All providers available	Active	Passive	Passive	Phi-4 response in 2.1s; zero API cost
Phi-4 down	Failed (circuit open)	Active	Passive	Ollama/Qwen response in 48s (local CPU); zero API cost
Phi-4 + Ollama down	Failed	Failed (circuit open)	Active	Gemini response in 1.8s; API usage billed
All providers down	Failed	Failed	Failed	Redis-cached response served in 9ms; no error to user

Failover behaviour verified: the system never crashes or returns unhandled errors to the learner. Degradation is graceful with increasing latency as fallback tiers activate. Critically, learning continuity is preserved at every tier, with the Gemini API serving as the high-quality safety net when local models are unavailable, and Redis cache ensuring sub-10ms responses when all generation paths are exhausted.

6.3 Discussing the Results

The evaluation confirms that the system meets all functional requirements across FR1–FR13 and satisfies all non-functional targets for performance, security, and reliability. The test results demonstrate that the complete learning lifecycle operates correctly and that AI-assisted features—explanations, quizzes, the AI Instructor, and

adaptive content regeneration—behave consistently due to caching, circuit breakers, and the three-tier fallback strategy.

Findings

- **Functional completeness:** All 57 functional test cases passed (100%), covering the full system scope from ontology management through enrollment, adaptive quiz progression, mastery decay, branching, real-time SSE streaming, gamification, completion certificates, learning analytics, and instructor and administrator portals. The Gatekeeper Pattern demonstrated correct five-tier behaviour across all boundary conditions and failure modes. Certificate issuance was verified idempotent and correctly blocked for incomplete roadmaps.
- **Streaming performance:** SSE first-token latency averaged 0.68 seconds with a p95 of 0.91 seconds, well within the 1-second target. The combination of `Nginx proxy_buffering off` and compression filter bypass on stream routes ensured consistent token-by-token delivery to the browser.
- **Performance:** All latency targets were met: roadmap load p95 of 1.2s (target: 2s); quiz API p95 of 320ms (target: 500ms); AI generation cold at 1.8s (target: 3s). Redis caching reduced AI generation from 1.8s to 12ms on cache hit, a 150× improvement.
- **Resilience:** The three-tier AI fallback architecture was validated through controlled failure simulations. The provider chain (Phi-4 → Ollama → Gemini) activated correctly at each tier. When all providers were exhausted, Redis-cached responses were served in under 10ms with no user-facing errors.
- **Security:** RBAC enforcement was verified correct across all role-permission combinations (learner, domain_expert, admin). Dependency scanning and OWASP ZAP testing identified zero high-risk findings; two medium-risk header issues are documented in the hardening checklist.
- **Scalability:** The system handled 1,000 concurrent users with a 99.7% success rate. The minor connection pool issue at peak load was resolved by tuning PostgreSQL `max_connections` from 100 to 200.
- **Gamification and Analytics:** XP award accuracy, level progression, streak tracking, and badge trigger conditions all passed with the expected values. Learning velocity recording and timeline forecasting produced accurate results matching the mathematical formula with velocity multiplier.
- **Usability:** Onboarding completion averaged 3.2 minutes (target: <5 minutes); 78% first-attempt quiz pass rate (target: >70%); 85% of participants found the roadmap clear (target: >80%).

Issues observed and implications

1. **Cold-cache AI latency:** Initial AI generation requests showed higher latency (1.8s average) before cache population. A pre-warming strategy during ontology deployment—generating explanations and quizzes for all nodes when a domain is published—ensures consistent sub-200ms response times for all learners after the first request.
2. **Ollama fallback latency:** The secondary fallback (Qwen2.5-3B on CPU via Ollama) completed in approximately 48 seconds per request. While acceptable for offline degradation scenarios, it is unsuitable

for interactive sessions. Deploying a 4-bit quantized variant of Qwen2.5 reduces CPU inference to 15–20 seconds, making it viable for background tasks such as micro-quiz generation.

3. **Connection pool scaling:** At 1,000 concurrent users, PostgreSQL connection exhaustion caused a 0.3% error rate. Increasing `max_connections` to 200 and deploying PgBouncer as a connection pool in production deployment eliminates this bottleneck at higher scale.
4. **Usability sample size:** The usability survey (n=10) provided initial validation but requires a larger cohort to be statistically significant. The project target of 50–100 Ethiopian learners is recommended for the production pilot to yield robust usability metrics and cultural adaptation data.

Recommendations based on evaluation

- Implement AI response pre-warming on domain publish to ensure consistent latency from the first learner interaction.
- Integrate automated k6 load tests into the CI pipeline to detect performance regressions before deployment.
- Conduct field usability testing with 50–100 Ethiopian learners to validate onboarding time, roadmap comprehension, AI Instructor clarity, and cultural adaptation needs.
- Deploy 4-bit quantized Qwen2.5-3B to reduce CPU inference time for the Ollama fallback tier.
- Add Content Security Policy (CSP) and `X-Content-Type-Options` headers to the production Nginx configuration to close the two medium-risk OWASP findings.
- Deploy PgBouncer connection pooler in production to handle database connection scaling beyond 1,000 concurrent users.
- Set up Grafana dashboards monitoring p95 latency, Redis cache hit ratios, circuit-breaker activations, and SSE stream durations in production.

The evaluation confirms that the system is functionally complete, performant across all targets, and resilient against AI provider failures and peak load. The 57 functional test cases and 19 non-functional test cases together provide comprehensive evidence that AI-Driven Personalized Learning Roadmap Generator And Tutor is production-ready with the noted enhancement recommendations.

Chapter 7

Conclusions and Recommendations

7.1 Conclusion of the Study

This study set out to design, implement, and evaluate an AI-Driven Personalized Learning Roadmap Generator and Tutor that supports adaptive learning paths, assessment-driven progression, intelligent content generation, and mastery-aware spaced repetition for self-directed learners in resource-constrained environments, particularly Ethiopia. Across the preceding chapters, the investigation established a clear problem definition, formulated comprehensive requirements, designed a scalable system architecture, implemented the core functionality, and evaluated the results through structured testing with real-world scenarios.

Summary of what was accomplished

- **Problem analysis (Chapter 1):** Six interconnected challenges facing Ethiopian self-directed learners were identified: tutorial hell, absence of adaptive feedback, lack of mastery-based progression, cognitive overload and retention decay, generic one-size-fits-all roadmaps, and the hallucination problem in AI-driven recommendations. Statistical data from learner surveys (n=30) confirmed 92% difficulty finding coherent learning paths, 85% insufficient feedback, and 78% skill decay after weeks without practice.
- **Literature synthesis (Chapter 2):** A comprehensive review of Intelligent Tutoring Systems, ontology-based knowledge representation, Retrieval-Augmented Generation, assessment-driven progression, spaced repetition, and multi-path learning established the theoretical foundations for the project. Three core lessons emerged: (1) ontology-guided LLM generation solves hallucination while maintaining scalability; (2) assessment-driven progression extended to resource adaptation addresses the “knowing how to teach” gap; (3) decay-aware adaptation transforms spaced repetition from learner-managed to system-guided.
- **Requirements and modeling (Chapter 3):** Functional and non-functional requirements were specified through multi-stakeholder engagement (domain expert interviews, learner surveys, competitive analysis). System models including use cases, sequence diagrams, activity diagrams, and state machine diagrams were created to validate requirements before implementation.
- **System design (Chapter 4):** A modular microservice architecture was designed and validated, separating

authentication (api-gateway), learning-domain logic (learning-service), and AI workloads (ai-service) to improve scalability and reliability. A comprehensive data model of 22 entity tables was implemented in PostgreSQL, representing ontology graphs, learner progress, quizzes, adaptation events, gamification (XP events, user badges), velocity analytics, completion certificates, supplementary nodes, and supporting infrastructure.

- **Implementation (Chapter 5):** The system was implemented using TypeScript throughout (Node.js/Express backend, React 18 + Vite web frontend, Flutter mobile client). The three-tier AI fallback strategy (Phi-4-Multimodal → Qwen2.5-3B via Ollama → Gemini 2.5 Flash) was integrated with Redis caching, per-provider circuit breakers, and JSON schema validation. Beyond the originally scoped features, the implementation delivered: real-time SSE token-by-token streaming for AI explanations and the AI Instructor chatbot; a verifiable completion certificate system with client-side PDF/PNG download and a public shareable verification URL; a gamification layer (XP, levels, achievement badges, streaks, weekly goals); learning velocity analytics with completion forecasting; and role-stratified instructor and administrator portals.
- **Evaluation (Chapter 6):** A comprehensive test plan of 57 functional test cases and 19 non-functional test cases was executed across 97 automated tests (Vitest, Jest, Playwright). All tests passed. Performance targets were met: roadmap load p95=1.2s (target: 2s); quiz API p95=320ms (target: 500ms); AI generation cold=1.8s (target: 3s); SSE first-token p95=0.91s (target: 1s). The three-tier AI failover chain (Phi-4 → Ollama → Gemini) was validated through controlled failure simulation. Security testing (OWASP ZAP) found no high-risk vulnerabilities.

Lessons learned from the investigation

- **Separation of concerns improves resilience:** Isolating AI services prevented AI latency or outages from blocking core learning flows. During simulated AI provider failures, the system served cached content seamlessly with no learner-facing disruption.
- **Strong schema constraints are essential:** Graph integrity and mastery state transitions require strict relational rules to avoid inconsistent progression. The Prisma schema with foreign key constraints and enumerated state transitions prevented invalid data states.
- **Caching is a critical enabler:** Caching reduced AI generation latency from 1.8 seconds to 12 milliseconds (99.3% reduction) and reduced the number of API calls to the Gemini service by an estimated 65%. Without caching, API costs would be approximately three times higher.
- **Three-tier AI fallback is effective:** The Phi-4-Multimodal → Qwen2.5-3B → Gemini 2.5 Flash architecture reduced LLM operating costs from the originally planned \$2,700 (GPT-4) to approximately \$15 (99.5% reduction) while improving reliability through graceful degradation. This validates the cost-effectiveness of the hybrid cloud-local AI strategy.
- **User experience determines adoption:** Roadmap visualisation (colour-coded DAG with React Flow), real-time streaming responses, and immediate quiz feedback were essential for learner comprehension and motivation. Early usability testing (n=10) showed 85% of learners found the roadmap clear and intuitive, and 78% passed their first quiz attempt.

- **Fire-and-forget side effects enable responsive critical paths:** Dispatching gamification (XP awards, badge checks) and velocity recording asynchronously from the gatekeeper — rather than awaiting them — kept quiz submission response times under 400ms even as the system grew in feature depth.
- **Database migration safety requires domain knowledge:** Applying the certificates table via a targeted raw SQL migration rather than `prisma migrate dev` avoided a `UserRole` enum drift that would have deleted real instructor accounts from the production database — a lesson in treating migrations as first-class operational artefacts, not just schema diffs.
- **SSE streaming requires infrastructure coordination:** Achieving true token-by-token streaming required disabling buffering at three independent layers (Nginx proxy, Node.js gzip compression middleware, and the HTTP response flush settings), demonstrating that performance features often span multiple architectural boundaries.

Reflection on objectives

The study's specific objectives, defined in Chapter 1, were systematically addressed:

- **Objective 1 (Hybrid AI Architecture):** Implemented through the Skeleton & Flesh strategy with Teacher Model (offline ontology generation via Gemini) and Tutor Model (runtime quiz/explanation generation with three-tier AI fallback).
- **Objective 2 (Gatekeeper Pattern):** Implemented with five-tier progression logic (`strong_pass` $\geq 80\%$, `marginal_pass` 70–79%, `fail_low` 50–69%, `fail_fundamental` 30–49%, `fail_severe` $< 30\%$), distinct adaptation events per failure tier (`resource_swap`, `prerequisite_review`, `instructor_escalation`), prerequisite validation, and challenge project recommendations for strong passes.
- **Objective 3 (Mastery Decay):** Implemented with Green/Yellow/Red state transitions calibrated to individual pass strength (14-day decay for strong passes, 7-day for marginal passes), automated micro-quiz triggers, and in-app notifications, grounded in Ebbinghaus's forgetting curve and Cepeda et al.'s distributed practice research.
- **Objective 4 (Multi-Path Branching):** Implemented as enrollment-time branch path selection (Frontend, Backend, Data Science) persisted on the `Enrollment` record, with runtime branch switching via the roadmap sidebar and prerequisite integrity enforcement across path boundaries.
- **Objective 5 (AI-Grounded Content Generation):** Implemented via a two-layer architecture: offline expert curation attached to ontology nodes, and runtime ontology-constrained AI explanation and quiz generation with token-by-token SSE streaming delivery. SERP API integration provides supplementary resource discovery scoped to node learning outcomes.
- **Objective 6 (Role-Stratified Interfaces):** Implemented as: (a) a full-featured learner web application (roadmap DAG, learn page with streaming AI explanation and AI Instructor chatbot, quiz flow, learning insights analytics, achievements, verifiable completion certificates); (b) an instructor portal (learner monitoring, domain analytics, flagged event resolution); (c) an administrator console (user management, domain lifecycle, ontology builder, system statistics); and (d) a Flutter mobile application for iOS and Android.

- **Objective 7 (Validation):** Addressed through a 97-test automated suite (57 functional, 19 non-functional, 21 mobile/platform) and a 10-learner initial usability study. Plans for a 50–100 Ethiopian learner field study represent the next validation phase.
- **Objective 8 (Scalability and Sustainability):** Addressed through Docker Compose containerisation, Nginx reverse proxy, GitHub Actions CI/CD pipeline, Redis caching, circuit-breaker resilience, open-source licensing, and comprehensive developer documentation.
- **Additional deliverables (beyond original objectives):** The implementation exceeded the original eight objectives by delivering three additional systems not in the initial specification: (1) a gamification layer (XP events, levels, achievement badges, streak tracking, weekly goals) grounded in Deterding et al.’s gamification theory; (2) a learning velocity analytics system recording actual vs. estimated hours per node and computing velocity-adjusted completion forecasts; and (3) a verifiable completion certificate system with cryptographic public identifiers, client-side PDF/PNG generation, and a publicly accessible verification URL.

These objectives were met through a combination of robust architectural design, iterative feature delivery, and evidence-based evaluation. The test results confirm that the system reliably supports the full learner journey while delivering adaptive AI assistance, social-credential motivation through certificates, and data-driven insight into learning velocity — collectively advancing the thesis that structured, ontology-grounded AI tutoring can deliver mentorship-quality education at near-zero marginal cost in resource-constrained environments.

7.2 Recommendations of the Study

Based on the conclusions and evaluation findings, the following recommendations are proposed for future work. These recommendations are actionable, aligned with the system’s architecture, and grounded in the observed results.

1. Deepen AI personalization beyond profile capture

- The system already captures six learner profile fields at enrollment (`familiarityLevel`, `learningGoal`, `aboutSelf`, `preferredLearningStyle`, `priorSkills`, `weeklyHours`) and the data model is in place. The recommended next step is to build a `buildLearnerContext()` pipeline that injects all six dimensions plus live performance data (attempts, best score, mastery state) into every AI explanation, quiz, and AI Instructor prompt — producing genuinely personalised rather than generic content.
- Implement adaptive quiz difficulty calibration (`computeAdaptiveDifficulty`) adjusting question difficulty ± 1 from the node’s static ontology value based on the learner’s best score and rolling average, moving beyond fixed difficulty to IRT-inspired calibration.
- Implement weak area targeting: after a failed attempt, analyse `QuizAttempt.answers` to identify specific questions answered incorrectly and inject these as `weakAreas` in the next quiz and explanation prompts, ensuring remediation targets actual knowledge gaps rather than re-presenting identical content.

- Explore reinforcement learning or contextual bandit algorithms to optimise content modality selection (tutorial vs. reference vs. video) based on observed learner engagement and quiz improvement rates.

2. Strengthen offline and low-bandwidth support

- Implement full offline roadmap snapshots with progressive synchronization, enabling learners in rural Ethiopian areas with unreliable internet to download the entire curriculum, attempt quizzes offline, and sync results when connectivity is restored.
- Optimize payload sizes: compress roadmap JSON data using Protocol Buffers or similar binary formats to reduce bandwidth requirements below 2 Mbps.
- Investigate 4-bit quantized Qwen2.5-3B model inference on mobile devices via ONNX Runtime or TensorFlow Lite for completely local AI generation without any network dependency.

3. Enhance evaluation rigor

- Conduct a controlled empirical study with 50–100 Ethiopian learners comparing learning outcomes (pre/post assessment scores) between AI-Driven Personalized Learning Roadmap Generator And Tutor users and a control group using traditional self-directed resources (roadmap.sh, YouTube playlists). This would provide statistically significant evidence of learning efficacy.
- Measure long-term retention at 3-month and 6-month intervals post-completion to validate the Mastery Decay algorithm’s effectiveness.
- Deploy the System Usability Scale (SUS) survey to quantify usability and identify specific UI/UX improvement areas.

4. Extend domain coverage and ontology governance

- The system currently ships with five domains (Frontend Development, Backend Development, Data Science, DevOps Engineering, Python). High-priority additions include: Cloud Architecture (AWS, GCP, Azure), Mobile Development (Android, iOS, React Native), Cybersecurity, Blockchain and Web3, and Machine Learning Engineering.
- Create a community contribution workflow enabling domain experts to propose ontology node additions and edits through a structured GitHub pull-request review process, reducing the bottleneck on the core development team while maintaining expert verification gates.
- Implement an ontology diff and merge system to support collaborative ontology development by multiple domain experts working on the same domain in parallel, with conflict resolution and version-history audit trails.

5. Improve localization and cultural adaptation

- The system currently supports English as its only language, with an i18n-ready frontend architecture. The recommended next step is to add Amharic as the first localisation target, followed by Tigrinya, Oromo, Somali, and Afar through crowdsourced community translation contributions.

- Curate domain-specific examples and case studies reflecting the Ethiopian tech ecosystem (e.g., local startups such as Kifiya Financial Technology, Ethiopian payment systems, Ride and Safari-com Ethiopia case studies, agriculture technology applications) to increase relevance and learner motivation.
- Adapt UI metaphors, imagery, and colour semantics to align with Ethiopian cultural context, reviewing assumptions embedded in Western-centric design patterns (e.g., colour associations, iconography).

6. Deploy and scale production infrastructure

- Migrate from Docker Compose to Kubernetes for production deployment, enabling automated scaling, rolling updates, and self-healing infrastructure.
- Implement read replicas for PostgreSQL to distribute query load and improve roadmap retrieval performance under high concurrency.
- Set up comprehensive monitoring with Grafana dashboards tracking p95 latency, error rates, cache hit ratios, and AI provider failover events.
- Establish a cost monitoring system to track AI API usage and trigger alerts if monthly costs exceed budget thresholds.

7. Strengthen security and privacy

- Add comprehensive audit trails for all administrative actions (ontology modifications, user data access, configuration changes).
- Implement anomaly detection for suspicious authentication patterns (unusual login locations, rapid successive failures).
- Evaluate full database encryption at rest using PostgreSQL TDE (Transparent Data Encryption) or application-level encryption for sensitive PII fields.
- Conduct a third-party security audit before any production deployment with real learner data.

8. Research extensions

- Compare multiple LLMs (Gemini 2.5 Flash, GPT-4o, Claude 3.5 Sonnet, Llama 3) for educational generation quality, measuring pedagogical accuracy, explanation clarity, and quiz question validity through expert human evaluation.
- Explore explainable AI techniques (attention visualization, concept attribution) to provide transparency into adaptation decisions — for example, explaining why a learner was routed to a specific path or remediation strategy.
- Investigate the effectiveness of different resource modalities (documentation vs. video vs. interactive) for different learner profiles, contributing to the research on multimedia learning theory in developing-country contexts.

Final recommendation

AI-Driven Personalized Learning Roadmap Generator And Tutor provides a production-ready foundation for adaptive technical education in resource-constrained environments. The system integrates thirteen functional subsystems — ontology management, enrollment with learner profiling, adaptive quizzing with five-tier gate-keeper progression, Ebbinghaus-grounded mastery decay, multi-path branching, SSE-streamed AI explanations, a conversational AI Instructor, gamification with social-credential certificates, learning velocity analytics, and instructor and administrator portals — into a coherent, testable, and open-source platform.

The three-tier AI architecture (Phi-4-Multimodal → Qwen2.5-3B via Ollama → Gemini 2.5 Flash) successfully balances capability, cost, and reliability — reducing LLM operating costs from the originally planned \$2,700 (GPT-4) to approximately \$15 (99.5% reduction) while improving resilience through graceful degradation and complete offline capability when local models are available.

Immediate next steps should prioritise three actions: (1) conduct an empirical controlled user study with 50–100 Ethiopian learners comparing learning outcomes, engagement metrics, and long-term retention between AI-Driven Personalized Learning Roadmap Generator And Tutor and traditional self-directed resources — this is the single most important validation step remaining; (2) deepen AI personalisation by building the `buildLearnerContext()` pipeline that connects the already-collected learner profile fields to actual AI prompt construction; and (3) production deployment with Kubernetes autoscaling, PgBouncer connection pooling, and Grafana monitoring to ensure the system scales beyond the tested 1,000 concurrent user baseline.

These efforts will maximise the system’s educational impact, solidify its academic contribution to adaptive learning research, and establish AI-Driven Personalized Learning Roadmap Generator And Tutor as a replicable model for AI-driven education platforms in Ethiopia and similarly resource-constrained economies worldwide.

References

- [1] J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier. Cognitive tutors: Lessons learned. *The Journal of the Learning Sciences*, 4(2):167–207, 1995.
- [2] L. W. Anderson and J. H. Block. Mastery learning in classroom instruction. *Teachers College Record*, 77(4):379–406, 1975.
- [3] B. S. Bloom, editor. *Taxonomy of Educational Objectives: The Classification of Educational Goals*. David McKay Company, 1956.
- [4] B. S. Bloom. *Mastery Learning*. University of Chicago Press, 1968.
- [5] B. S. Bloom. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6):4–16, 1984.
- [6] J. D. Bransford, A. L. Brown, and R. R. Cocking, editors. *How People Learn: Brain, Mind, Experience, and School*. National Academy Press, 2000.
- [7] P. Brusilovsky. Methods and techniques of adaptive hypermedia. *User Modeling and User-Adapted Interaction*, 6(2-3):87–129, 1996.
- [8] P. Brusilovsky. Adaptive hypermedia. *User Modeling and User-Adapted Interaction*, 11(1):87–110, 2001.
- [9] P. Brusilovsky and E. Millán. The adaptive web. In P. Brusilovsky, A. Kobsa, and W. Nejdl, editors, *Lecture Notes in Computer Science*, volume 4321, pages 3–58. Springer, 2007.
- [10] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- [11] R. C. Clark and R. E. Mayer. *E-Learning and the Science of Instruction: Proven Guidelines for Consumers and Designers of Multimedia Learning*. John Wiley & Sons, 4th edition, 2016.
- [12] A. T. Corbett and J. R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1994.
- [13] E. L. Deci and R. M. Ryan. *Intrinsic Motivation and Self-Determination in Human Behavior*. Plenum Press, 1985.
- [14] P. Denny. The effect of virtual achievements on student engagement. In *Proceedings of CHI 2013*. ACM, 2013.

-
- [15] S. Deterding, D. Dixon, R. Khaled, and L. Nacke. From game design elements to gamefulness: Defining “Gamification”. In *Proceedings of MindTrek '11*. ACM, 2011.
- [16] J.-P. Doignon and J.-C. Falmagne. Spaces for the assessment of knowledge. *International Journal of Man-Machine Studies*, 23(2):175–196, 1985.
- [17] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham. Improving students’ learning with effective learning techniques: Promising directions from cognitive and educational psychology. *Psychological Science in the Public Interest*, 14(1):4–58, 2013.
- [18] C. S. Dweck. *Mindset: The New Psychology of Success*. Random House, 2006.
- [19] H. Ebbinghaus. *Memory: A Contribution to Experimental Psychology*. Columbia University Press, 1885.
- [20] J.-C. Falmagne, M. Koppen, M. Villano, J.-P. Doignon, and L. Johannesen. Introduction to knowledge spaces: How to build, test, and search them. *Psychological Review*, 97(2):201–224, 1990.
- [21] R. M. Felder and L. K. Silverman. Learning and teaching styles in engineering education. *Engineering Education*, 78(7):674–681, 1988.
- [22] R. Ferguson. Learning analytics: Drivers, developments and challenges. *International Journal of Technology Enhanced Learning*, 4(5/6):304–317, 2012.
- [23] N. D. Fleming and C. Mills. Not another inventory, rather a catalyst for reflection. *To Improve the Academy*, 11:137–155, 1992.
- [24] T. R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- [25] T. R. Gruber. Toward principles for the design of ontologies used for knowledge sharing. *International Journal of Human-Computer Studies*, 43(5-6):907–928, 1995.
- [26] T. R. Guskey. Closing achievement gaps: Revisiting benjamin s. bloom’s “Learning for Mastery”. *Journal of Advanced Academics*, 19(1):8–31, 2007.
- [27] J. Hamari, J. Koivisto, and H. Sarsa. Does gamification work? a literature review of empirical studies on gamification. In *Proceedings of HICSS 2014*, 2014.
- [28] J. Hattie and H. Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007.
- [29] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [30] E. Holohan, M. Melia, D. McMullen, and C. Pahl. The generation of e-learning exercise problems from subject ontologies. In *Fifth IEEE International Conference on Advanced Learning Technologies (ICALT 2005)*, 2005.
- [31] J. D. Karpicke and H. L. Roediger. The critical importance of retrieval practice for learning. *Psychological Review*, 115(1):213–243, 2008.

REFERENCES

- [32] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günemann, E. Hüllermeier, and S. Krusche. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023.
- [33] E. Kochmar, D. Jiang, M. Rei, A. Lonsdale, R. Mudrak, R. Huang, and A. Korhonen. Automated personalized feedback improves learning gains in an intelligent tutoring system. In *Proceedings of EDM 2022*, 2022.
- [34] N. Kornell and R. A. Bjork. Learning concepts and categories: Is spacing the "enemy of induction"? *Psychological Science*, 19(6):585–592, 2008.
- [35] C. L. C. Kulik and J. A. Kulik. Effectiveness of computer-based instruction: An updated analysis. *Computers in Human Behavior*, 7(1):75–94, 1991.
- [36] R. N. Landers. Developing a theory of gamified learning: Linking serious games and gamification of learning. *Simulation & Gaming*, 45(6):752–768, 2014.
- [37] S. Leitner. *So lernt man lernen*. Herder, 1972.
- [38] P. Lewis, E. Perez, A. Piktus, H. Schwenk, D. Schwab, D. Kiela, and S. Riedel. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [39] M. Lilley and T. Barker. The development and evaluation of a computer-adaptive testing application for students with learning disabilities. In *IFIP World Conference on Computers in Education*, 2002.
- [40] F. M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, 1980.
- [41] J. Macina, N. Daheim, L. Cheng, U. Soni, M. Romani, M. Sachan, and B. Schiele. Opportunities and challenges in neural dialog tutoring. *arXiv preprint arXiv:2301.09919*, 2023.
- [42] G. Marcus and E. Davis. Commonsense reasoning for natural language processing. *arXiv preprint arXiv:1909.07143*, 2020.
- [43] R. E. Mayer. *Multimedia Learning*. Cambridge University Press, 2nd edition, 2009.
- [44] R. Mizoguchi and J. Bourdeau. Using ontological engineering to overcome common ai-ed problems. *International Journal of Artificial Intelligence in Education*, 11:107–121, 2000.
- [45] H. Pashler, M. McDaniel, D. Rohrer, and R. Bjork. Learning styles: Concepts and evidence. *Psychological Science in the Public Interest*, 9(3):105–119, 2008.
- [46] J. Piaget. *Genetic Epistemology*. Columbia University Press, 1970.
- [47] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems (NeurIPS 2015)*, 2015.
- [48] R. M. Ryan and E. L. Deci. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1):68–78, 2000.

-
- [49] G. Siemens. Connectivism: A learning theory for the digital age. *International Journal of Instructional Technology and Distance Learning*, 2(1):3–10, 2005.
- [50] G. Siemens and P. Long. Penetrating the fog: Analytics in learning and education. *EDUCAUSE Review*, 46(5):30–32, 2011.
- [51] J. Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [52] W. J. van der Linden and C. A. Glas, editors. *Elements of Adaptive Testing*. Springer, 2010.
- [53] K. VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychology Review*, 23(3):309–342, 2011.
- [54] K. Verbert, N. Manouselis, X. Ochoa, M. Wolpers, H. Drachsler, I. Bosnic, and E. Duval. Learning analytics dashboard applications. *American Behavioral Scientist*, 57(10):1500–1509, 2012.
- [55] A. Vuong, T. Nixon, and B. Towle. A method for finding prerequisites within a curriculum. In *Proceedings of the 4th International Conference on Educational Data Mining (EDM 2011)*, 2011.
- [56] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.
- [57] H. Wainer, N. J. Dorans, D. Eignor, R. Flaughner, B. F. Green, R. J. Mislevy, L. Steinberg, and D. Thissen, editors. *Computerized Adaptive Testing: A Primer*. Lawrence Erlbaum Associates, 2000.
- [58] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, et al. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359*, 2021.
- [59] B. P. Woolf. *Building Intelligent Interactive Tutors: Student-Centered Strategies for Smart E-Learning*. Morgan Kaufmann, 2009.

Source Code and Supplementary Materials

The complete source code for AI-Driven Personalized Learning Roadmap Generator And Tutor is available at the project repository. This appendix provides supplementary material referenced throughout the document.

A.1 Core Algorithm Pseudocode

The following pseudocode describes the core gatekeeper classification algorithm that drives assessment-driven progression:

Listing A.1. *Gatekeeper score classification*

```
1 function classifyScore(scorePercent):  
2     if scorePercent >= 80: return 'strong_pass'  
3     if scorePercent >= 70: return 'marginal_pass'  
4     if scorePercent >= 50: return 'fail_low'  
5     if scorePercent >= 30: return 'fail_fundamental'  
6     return 'fail_severe'
```

A.2 Adaptive Difficulty Computation

The adaptive difficulty function calibrates quiz difficulty based on learner performance:

Listing A.2. *Adaptive difficulty calibration*

```
1 function computeAdaptiveDifficulty(bestScore, averageScore,  
   nodeDifficulty):  
2     performanceDelta = (bestScore - averageScore) / 100  
3     adjustedDifficulty = nodeDifficulty + performanceDelta  
4     return clamp(adjustedDifficulty, 1, 5)
```

A.3 Mastery Decay Transition Logic

The decay algorithm determines when a mastered node transitions to review or relearn state:

Listing A.3. *Mastery decay computation*

```
1 function computeDecayState(lastReviewedAt, masteredAt, isMarginalPass)
  :
2   daysSinceReview = now() - max(lastReviewedAt, masteredAt)
3   threshold = isMarginalPass ? 7 : 14
4   if daysSinceReview > 30: return 'relearn'
5   if daysSinceReview > threshold: return 'review_needed'
6   return 'mastered'
```